



Technische Universität München
TUM School of Computation, Information and Technology

Learning-Based Security Analysis: Contextual Representations and Adversarial Evaluation

Mohammad Reza Norouzian

Vollständiger Abdruck der von der TUM School of Computation, Information and Technology der Technischen Universität München zur Erlangung des akademischen Grades eines

Doktors der Naturwissenschaften (Dr. rer. nat.)

genehmigten Dissertation.

Vorsitz: Prof. Dr. Claudia Eckert

Prüfende der Dissertation:

1. Prof. Dr. First Reviewer
2. Prof. Dr. Second Reviewer
3. Prof. Dr. Optional Third Reviewer

Die Dissertation wurde am DD.MM.YYYY bei der Technischen Universität München eingereicht und durch die TUM School of Computation, Information and Technology am DD.MM.YYYY angenommen.

Mohammad Reza Norouzian

Learning-Based Security Analysis: Contextual Representations and Adversarial Evaluation

Dissertation, Draft

Technische Universität München

TUM School of Computation, Information and Technology

Your Chair or Institute

Boltzmannstraße 3

85748 Garching bei München

Abstract

This dissertation examines two questions in learning-based security analysis. The first concerns representation: which observations of a program or a network a detector should receive, and how they should be structured. The second concerns evaluation, namely how a defence can be measured when an adversary may choose the test data.

Two Android systems form the main contribution. Hybroid combines an embedding of the function-call graph with 13 features of the network flows that an application produces. On CICAndMal2017, fusion added one or two points of detection F_1 over the better single modality, and between minus two and plus three points in malware-category classification. HGANN-Mal lifts the function-call graph to a hypergraph whose hyperedges include the backward slices of security-sensitive call sites, and it learns hyperedge membership weights by attention. On Drebin and CICMalDroid, mean accuracy over five training runs was 3.1 to 10.8 points higher for every hypergraph operator than for the stronger pairwise baseline across all four task and corpus combinations. HGANN-Mal led the non-attentive hypergraph operators by 1.2 to 4.1 points in three combinations and scored 0.1 point lower in the fourth. Since construction and attention changed together, an ablation that isolates attention remains open.

In industrial control systems, the modular framework NADICS supports researcher-guided comparison of learning algorithms through configurable data preparation and fixed model interfaces. A random forest trained on an S7-300 corpus from the IUNO project reached an attack F_1 of 94.50% at a false-positive rate of 0.484% under a session-disjoint split. A GAN-based detector at the same interface flagged 91.89% of anomalous SWaT timestamps at a false-positive rate of 5.08% on later test data.

Within the SPARTA project, an adversary model was translated into a contest on a hidden test set and then into a modular benchmark that combines attacks and defences by configuration. The accuracy-loss score used for ranking assigns zero loss to any constant classifier under label-preserving attacks; under PGD, the defence with zero loss had an attacked accuracy 12.9 points below that of adversarial training. The four contest results do not establish a general relation between clean performance and robustness, so the two should be reported together.

Kurzfassung

Diese Dissertation untersucht zwei Fragen der lernbasierten Sicherheitsanalyse. Die erste betrifft die Repräsentation: Welche Beobachtungen eines Programms oder Netzwerks soll ein Detektor erhalten, und wie sollen sie strukturiert sein? Die zweite betrifft die Evaluation, also die Messung eines Abwehrverfahrens, wenn ein Angreifer die Testdaten wählen kann.

Den Hauptbeitrag bilden zwei Android-Systeme. Hybroid kombiniert eine Einbettung des Funktionsaufrufgraphen mit 13 Merkmalen der Netzwerkflüsse einer Anwendung. Auf CI-CAndMal2017 erhöhte die Fusion den F_1 -Wert der Erkennung um ein bis zwei Prozentpunkte gegenüber der besseren Einzelmodalität; bei der Klassifikation nach Schadsoftwarekategorien lag der Effekt zwischen minus zwei und plus drei Punkten. HGANN-Mal überführt den Funktionsaufrufgraphen in einen Hypergraphen, dessen Hyperkanten auch Rückwärts-Slices sicherheitsrelevanter Aufrufstellen umfassen, und lernt Zugehörigkeitsgewichte mit Attention. Auf Drebin und CICMalDroid lag die über fünf Trainingsläufe gemittelte Genauigkeit jedes Hypergraphoperators in allen vier Kombinationen aus Aufgabe und Korpus um 3,1 bis 10,8 Prozentpunkte über der stärkeren paarweisen Baseline. HGANN-Mal lag in drei Kombinationen um 1,2 bis 4,1 Prozentpunkte vor den Hypergraphoperatoren ohne Attention und in der vierten um 0,1 Prozentpunkte darunter. Da Konstruktion und Attention gemeinsam verändert wurden, fehlt noch eine Ablation, welche die Attention isoliert.

In industriellen Steuerungssystemen unterstützt das modulare Framework NADICS den von Forschenden gesteuerten Vergleich von Lernalgorithmen durch konfigurierbare Datenaufbereitung und feste Modellschnittstellen. Ein Random Forest erreichte auf einem S7-300-Korpus aus dem Projekt IUNO unter sitzungsdisjunkter Aufteilung einen F_1 -Wert der Angriffsklasse von 94,50 % bei einer Falsch-Positiv-Rate von 0,484 %. Ein GAN-basierter Detektor an derselben Schnittstelle erkannte auf späteren Testdaten 91,89 % der anomalen SWaT-Zeitstempel bei einer Falsch-Positiv-Rate von 5,08 %.

Im Projekt SPARTA wurde ein Angreifermodell in einen Wettbewerb mit verdecktem Testdatensatz und dann in einen modularen Benchmark überführt, der Angriffe und Abwehrverfahren per Konfiguration kombiniert. Der zur Rangbildung genutzte Genauigkeitsverlust weist jedem konstanten Klassifikator unter labelerhaltenden Angriffen den Wert null zu; unter PGD lag das verlustfreie Abwehrverfahren in der Genauigkeit unter Angriff 12,9 Prozentpunkte hinter adversarialem Training. Aus den vier Wettbewerbsergebnissen lässt sich kein allgemeiner Zusammenhang zwischen Genauigkeit und Robustheit ableiten; beide sollten gemeinsam berichtet werden.

Acknowledgements

Replace this text with your acknowledgements.

Contents

Abstract	iii
Kurzfassung	v
Acknowledgements	vii
1 Introduction	1
1.1 Research Context	1
1.2 Problem Statement and Scope	2
1.3 Research Questions	3
1.4 Contributions	5
1.5 Publications, Projects, and Individual Contributions	6
1.6 Dissertation Structure	7
2 Background	9
2.1 Android Applications and Their Analysis	9
2.1.1 Android Malware Detection	9
2.1.2 Dalvik Bytecode, Manifest, and Permissions	10
2.1.3 Static, Dynamic, and Hybrid Analysis	10
2.1.4 Obfuscation, Packing, and Repackaging	11
2.2 Graph Representations of Programs	12
2.2.1 Control-Flow and Function-Call Graphs	12
2.2.2 Hypergraphs and Higher-Order Relations	13
2.3 Learning over Graphs	14
2.3.1 Message Passing	14
2.3.2 Attention	14
2.3.3 Hypergraph Convolution	15
2.4 Detection in Networked and Cyber-Physical Systems	15
2.4.1 Anomaly Detection as a Security Task	16
2.4.2 Network Flows as a Unit of Observation	16
2.4.3 Communication Structure in Industrial and IoT Networks	17
2.4.4 Datasets and Measurement Limits	17
2.5 Adversarial Machine Learning	17
2.6 Performance Measures and Evaluation Methodology	18
2.6.1 Observation Units and Confusion Counts	19
2.6.2 Binary Detection Measures and Class Prevalence	19
2.6.3 Multiclass and Multilabel Aggregation	20
2.6.4 ROC Curves and Area under the Curve	21
2.6.5 Performance under Adversarial Input	21

2.6.6	Experimental Protocol and Bias	22
2.7	Summary and Research Gaps	22
3	Multimodal Android Malware Analysis with Hybroid	25
3.1	Research Problem and Contribution	25
3.2	Related Work	27
3.3	Observation and Threat Model	28
3.4	Dataset and Experimental Protocol	29
3.5	Opcode and Basic-Block Representation	30
3.6	Program-Graph Representation	31
3.7	Network-Flow Representation	32
3.8	Multimodal Fusion and Classification	35
3.9	Binary Detection Results	35
3.10	Malware-Category Classification	37
3.11	Modality Comparison	39
3.12	Evaluation Scope and Limitations	39
3.13	Chapter Summary	40
4	Higher-Order Android Malware Analysis with HGANN-Mal	41
4.1	Research Problem and Relation to Hybroid	41
4.2	Related Work	42
4.3	Observation and Threat Model	44
4.4	APK Processing and Function-Call Graph Extraction	44
4.5	Security-Sensitive API Identification and Program Slicing	45
4.6	Node Features	46
4.7	Hyperedge Construction	48
4.8	Hypergraph Attention Model	51
4.9	Datasets, Tasks, and Split Policy	55
4.10	Binary Detection Results	58
4.11	Malware-Family Classification on Drebin	60
4.12	Malware-Category Classification on CICMalDroid	61
4.13	Efficiency Bounds and Future Evaluation	62
4.14	Limitations and Threats to Validity	63
4.15	Chapter Summary	64
5	Anomaly Detection in Industrial Control Systems	65
5.1	Scope, Research Question, and Contributions	65
5.2	Observation Models and Datasets	65
5.3	Exploratory Study: Communication Graphs for IoT Services	66
5.4	The NADICS Framework and IUNO Integration	67
5.4.1	The S7-300 Corpus	71
5.4.2	Detection Results	74
5.4.3	Benign Operation and Alarm Burden	75
5.4.4	Extractor Throughput on Constrained Hardware	76
5.5	A GAN-Based Multivariate Detector for SWaT	76
5.5.1	Model and Training	80
5.5.2	Input Representations and Settings	81
5.5.3	Detection Results	82

5.5.4	Interpretation and Evaluation Scope	84
5.6	Limitations, Validity, and Individual Contribution	85
5.7	Chapter Summary	85
6	Evaluating Adversarial Robustness: Benchmark Design and Metric Analysis	87
6.1	Research Problem and Contribution	87
6.2	Threat Analysis and Evaluation Scope	88
6.3	Evaluation Requirements and Threat Models	88
6.4	Tasks and Evaluation Protocol	89
6.4.1	Prediction Tasks and Permitted Perturbations	89
6.4.2	SAFAIR Contest Scoring and Evaluation Protocol	89
6.5	Data and Evaluation Measures	90
6.5.1	Corpora and Hidden Evaluation Samples	90
6.5.2	Accuracy and Task-Specific Attack Success	91
6.5.3	Aggregation over Attacks	92
6.6	Benchmark Architecture and Reproducibility	92
6.7	Defence Mechanisms and Integration	94
6.7.1	Training and Representation Transformations	94
6.7.2	History and Activation Detectors	94
6.7.3	From an Alarm to a Classification	95
6.8	Results and Metric Analysis	97
6.8.1	Contest Evidence	97
6.8.2	SD Contest Submissions	98
6.8.3	Histopathology Benchmark	98
6.8.4	Sensitivity to the Scoring Rule	100
6.8.5	Comparison with Constrained PDF-Malware Features	101
6.9	Threats to Validity	102
6.10	Individual Contribution and Project Attribution	103
6.11	Chapter Summary	103
7	Cross-Contribution Discussion	105
7.1	Contributions of Contextual Representations	105
7.2	Observation Requirements and Analysis Cost	106
7.3	Evaluation Principles Across the Contributions	106
7.4	Implications for Android Malware Analysis	107
8	Conclusion and Future Work	109
8.1	Answers to the Research Questions	109
8.2	Summary of Contributions	110
8.3	Future Work	110
	References	111
A	Feature Importance of the NADICS Extraction Fields	127

List of Figures

2.1	Android package contents and analysis modes	11
2.2	Pairwise projection of a hypergraph	13
3.1	The Hybroid architecture, redrawn by the candidate from [5].	26
3.2	Opcode frequency across the pre-training corpus. The upper panel names the twenty most common operations, and the lower panel plots all 216 in rank order. The figure describes empirical frequencies, without a statistical test of a power-law fit. Reproduced from [5].	31
3.3	Published design of the program-code branch, redrawn by the candidate from [5].	31
3.4	The network branch, redrawn by the candidate from [5].	33
3.5	Kendall correlations among the retained flow features. Reproduced from [5]. .	34
3.6	Receiver operating characteristic curves for binary malware detection, reproduced from the published vector content of Figure 7 in [5]. The source labels the curves as macro averages and reports their AUC values in the legend.	36
3.7	Binary detection by learner and observation modality, with a panel for each metric. Values are the five-fold means reported to two decimal places in Figure 8 of [5]. The score axis is truncated to 0.86–0.98. Fold-level scores are unavailable, so no error bars are shown. Redrawn by the candidate.	37
3.8	Five-class classification by learner and observation modality. Values are the five-fold means reported to two decimal places in Figure 9 of [5]. The score axis is truncated to 0.78–0.96. Decision-tree recall is 0.86 for code and fused inputs, so their markers coincide. Fold-level scores are unavailable, so no error bars are shown. Redrawn by the candidate.	37
3.9	Per-class receiver operating characteristic curves for gradient boosting on the fused representation, reproduced from the published vector content of Figure 10 in [5]. The legend reports the AUC of each class and a macro-average AUC of 0.976.	38
4.1	Construction of the node feature vector in Stage 3	47
4.2	Hyperedge generation in Stage 4	49
4.3	Class distribution of the two corpora	56
4.4	F1 of the five methods on both corpora and both tasks	59
4.5	Accuracy against the majority baseline and the strongest baselines	59
5.1	Architecture of the NADICS framework. Malicious traffic is generated in a real industrial environment, dissected into protocol-characteristic features, and classified by the anomaly detection engine. The stage structure follows the one reported in the IUNO deliverable [25].	68

5.2	The IUNO test environment at the Volkswagen plant in Wolfsburg. Four servers, three network segments and a Siemens plant network reproduce the production setting in which NADICS ran as the TUM detection tool on the virtualisation host. The network TAPs fitted at the relevant interfaces are omitted from the drawing [42].	70
5.3	Random-forest importance of the 63 extraction fields. (a) The fifteen most important fields, coloured by group. (b) Each group's share of the total importance against its share of the field count. (c) Cumulative share over the ranked fields.	73
5.4	SWaT process and control architecture. Network levels follow the SWaT convention. The field-network symbols indicate sensors and actuators within each stage; the detector consumes their recorded values. Adapted from the supplied diagram and the dataset description [34].	78
5.5	SWaT attack categories and their counts in the dataset paper [34]. Stage and point locations are schematic examples. The arrangement describes attack scope and carries no ranking of detection difficulty.	79
5.6	SWaT collection campaign and the partition used in the GAN study. The campaign follows the dataset paper [34]. After the initial normal-operation exclusion, the first 56,240 attack-period timestamps support training and validation; the later 393,679 timestamps form the test set.	79
5.7	Multivariate GAN detection pipeline. (a) The four networks are trained together on earlier windows containing normal and attack observations. (b) The later 393,679 test timestamps are scored with the trained networks; the latent critic has no part in the score a_t of Equation 5.1. The blue tag marks the input representation, the only factor varied between runs, and orange tags mark development steps that used attack ground truth. The window length is fixed at 60 samples.	80
5.8	Observed signals (blue) and their reconstructions (orange) for two dimensions. The repeated red trace is the aggregate reconstruction discrepancy across dimensions, shared by the two displayed signals. It precedes the combination of Equation 5.1.	81
6.1	Benchmark architecture, redrawn after D7.6 Figure 2, p. 22 [32]. Box labels follow the original. The five modules separate task definitions and data loading from attack execution.	93
6.2	Autoencoder placement in the partner classifier, redrawn after D7.2 Figures 13 and 11, p. 14 [31]. In the top variant (a), the autoencoder r_{η_x} transforms the input before feature extraction. In the middle variant (b), r_{η_z} acts between the feature extractor ϕ_ω and the prediction head c_ψ , as in Equation 6.8.	95
6.3	Partner detection mechanisms, redrawn after D7.2 Figures 15 and 16, p. 16 [31]. Panel (a) uses preceding queries, whereas panel (b) uses internal activations of the classifier. The recorded parameters, the activation features of Equation 6.9 and the per-class detector follow D7.2. The response is the label reversal of Equation 6.10, which D7.6 applies in the benchmark [32]. The final action is part of the defended system and must be included when task accuracy is measured.	96

6.4	Reported clean and transferred-attack accuracies for SD submissions 2 and 3, from D7.6 Sections 2.8.1.2–2.8.1.3, pp. 18–19 [32]. Submission 1 is omitted because D7.6 reports no comparable transferred-attack accuracy for its collapsed model. The 84–85% interval is an approximate reported range, not a confidence interval; these development values are separate from the official contest scores.	99
6.5	Accuracy losses computed from D7.6 Table 2 [32]. Each cell is $100(A_{\text{clean}} - A_{\text{attack}})$ in percentage points; the original accuracies remain in Table 6.4. The separation of attack rows does not imply matched budgets. In particular, zero loss for the activation detector describes its aggregate score, not a demonstrated mechanism or robustness certificate.	100
6.6	Original iterative-FGSM transfer curves from the partner PDF-malware study, D7.5 Figure 15, p. 23 [33]. The horizontal axis is the source’s $\log_{10}(\delta)$ perturbation budget, distinct from the accuracy loss Δ . The published feature-space results contain no uncertainty bands.	102

List of Tables

1.1	Previously disseminated work and project artefacts used in this dissertation. . .	6
1.2	Research questions and the evidence supporting their answers. Each contribution chapter states its experimental scope and limitations.	7
2.1	Decision units and prediction targets	19
3.1	CICAndMal2017 source-corpus composition before extraction failures are excluded. Malware-category counts are recomputed from the official family listing [148]; the 1,700 benign samples are the package/capture pairs used in [5]. . .	29
3.2	The 13 retained network-flow fields, grouped by the quantities they describe [5]. Their encoded values are averaged at application level as in Equation 3.6. . . .	33
3.3	Reported HTTP and TLS flow shares by malware category [5]. The remainder is calculated as 100% minus those shares; its protocol composition is unspecified.	34
3.4	Published binary-detection comparison with four baselines. Performance values are read from Figures 6 and 7 of [5]; their metric-averaging conventions are not fully specified.	35
3.5	Area under the ROC curve per class, gradient boosting on the combined representation, with the pre-exclusion counts of Table 3.1. Values are read from Figure 10 of [5].	38
3.6	Difference between the fused score and the better of the two single-modality scores. Positive values favour fusion. The differences are computed from Figures 8 and 9 of [5], whose values are printed to two decimal places.	39
4.1	The four groups of node feature. The width of the first group is the size of the opcode vocabulary.	46
4.2	Binary detection results (%). Learned-model scores are five-run means on a shared fixed partition per corpus; F1, precision and recall are macro-averaged over both classes. Author-corrected Drebin macro scores supersede the corresponding entries in the source publication [6]. Best values in bold.	58
4.3	Multiclass classification results (%). Learned-model scores are five-run means on a shared fixed partition per corpus; F1, precision and recall are macro-averaged over classes. Drebin uses twenty malware families; CICMalDroid uses five categories, including Benign [6]. Best values in bold.	61
5.1	Observation units in the industrial evaluations and the exploratory IoT study. Labels, sampling processes, and validation protocols differ between the studies.	66
5.2	Composition of the S7-300 corpus and its partitions. Whole capture sessions were assigned to the partitions, four to training and one to each of the other two, and every attack episode lies entirely within one partition. The episode column counts executed attacks, not records.	71

5.3	Duration of the 25 attack episodes in minutes and seconds. Each attack type has three training episodes, one validation episode and one test episode.	72
5.4	Six-class confusion matrix on the 19,841-record test session, with rows giving the true class. Predicted classes are abbreviated: N normal, AB authentication bypass, CS CPU stop and start, RC reconnaissance, DS denial of service, BF brute force. The last three columns give precision, recall and F_1 for the class of that row.	74
5.5	Extraction workloads on the Raspberry Pi 3 prototype. Input frames are the frames presented to the extraction pipeline, and processed frames exclude those dropped before processing. Rates and CPU utilisation are averages over the replay, and CPU utilisation refers to the capacity of the whole device.	76
5.6	Measured timestamp-level performance of the input representations. Each timestamp is counted once and no point adjustment is applied.	83
5.7	Timestamp-level confusion matrix for the final 24-component PCA configuration. Counts refer to individual timestamps after aggregation of overlapping window outputs.	83
6.1	Planned and reported SAFAIR evaluation procedures.	90
6.2	Evaluation units and data provenance. Stated sample sizes describe the printed procedures, not recovered per-example result files.	91
6.3	Reported contest scores, including the worst-perturbed column, from D7.6 Table 1, p. 16 [32]. Task names follow that table. The SD task label conflicts with its detailed attribute-model description; best and worst values are not identified as paired outcomes for one checkpoint under a common attack suite.	97
6.4	Published accuracies for the five partner defence configurations, from D7.6 Table 2, pp. 28–29 [32]. Attack hyperparameters and the evaluated subset are unspecified. Each entry is a reported aggregate, with no dispersion estimate.	99
A.1	Importance of the 63 fields of the NADICS extraction schema in the random forest of Section 5.4.2, in rank order. Importance is the normalised share of the total, and the cumulative column sums the importances down to that rank. The letter of the identifier gives the group of Table A.2, and the bar colours follow Figure 5.3.	127
A.2	Importance by field group. Both shares are given in % of the 63-field total. Importance per field is the group share divided by the number of fields in the group, and the last column gives the rank of the group’s highest field in Table A.1.	129

Introduction

1.1 Research Context

Binary malware detection assigns an application to a benign or malicious class. Multiclass analysis can instead distinguish malware families or broader application categories; the selected label scheme defines the prediction target [1]. Learning-based detectors derive features or structured representations from observable application properties and learn a decision rule from labelled examples.

The reliability of Android malware analysis depends on the observations available to the classifier. DREBIN uses features from an application's manifest and bytecode [2], while MaMaDroid models sequences of abstracted application programming interface (API) calls recovered through static analysis [3]. In graph-based detectors, program structure forms part of the input to representation learning [1]. A systematic review identified 132 studies of deep-learning approaches to Android malware defence published between 2014 and 2021 [4].

This dissertation examines contextual representation and adversarial evaluation, with Android malware analysis at the centre. The term *context* covers two sources of information: relations among program or system elements, and complementary observations combined for one prediction. Hybroid studies context across static code and recorded network traffic [5]; HGANN-Mal studies context within the application package through groups of related methods [6]. The industrial control studies use protocol-specific records and multivariate process windows. A short exploratory study of service relationships in the Internet of Things (IoT) provides a further example of an observation model [7].

Researchers must consider the test population when evaluating a model's predictive performance. Jordaney et al. document changes in malware that make previously trained classifiers unreliable [8]. Pendlebury et al. show how temporally inconsistent partitions and unrepresentative class distributions can inflate reported performance [9]. In an assessment of thirty selected security papers, Arp et al. found at least three of ten methodological pitfalls in each paper [10]. These findings motivate explicit sampling and evaluation assumptions.

Related applications also affect the independence of training and test samples. Drebin contains repackaged applications with duplicated opcode sequences [11]. Zhao et al. found limited duplication effects for the supervised binary detectors they tested, but larger effects for unsupervised malware-family clustering [12].

Adversarial evaluation concerns deliberate changes to the input of a defended system. Athalye et al. identified obfuscated gradients in seven of nine non-certified white-box defences accepted at ICLR 2018; their attacks circumvented six completely and one partially [13]. Carlini et al. specified threat-model and evaluation requirements in 2019 [14]. Tramèr et al. subsequently circumvented thirteen defences, almost all of which had attempted adaptive evaluation [15]. The quality of that evaluation depends on how the attack addresses the particular defence.

For Android, a problem-space attack must produce a valid modified package that retains its malicious functionality [16]. The evaluation must therefore specify which transformations an attack may use and how they change the extracted features. This dissertation compares the predictive performance of different representations and assesses complete systems under adversarial attacks.

1.2 Problem Statement and Scope

The representation problem

Static analysis examines an application without executing it and can inspect code outside the paths exercised by a test harness. Its coverage depends on the available code and the analysis approximations [17]; obfuscation can hinder recovery [18]. Dynamic analysis records triggered behaviour through instrumentation, as in taint tracking [19], but depends on the execution paths reached. Within-system studies have reported benefits from combining static and dynamic observations [20]. Binary and category classification impose different distinctions on the observations: a feature that separates malware from benign applications need not separate malicious categories. A joint representation must also describe both observations at a common unit, because code is recovered from the package while traffic is recorded as individual flows. RQ1 asks how program structure and network behaviour can be represented together, and how their combination compares with each source alone on the same applications, separately for binary detection and malware-category classification.

A function-call graph relates methods through recovered call sites. A hypergraph can record a chosen group of methods as one hyperedge [21]. Replacing each group by pairwise links can lose its identity; a bipartite graph with distinct method and group vertices preserves the memberships [22]. Prior Android work constructs hyperedges from call neighbourhoods and shared permissions [23], and hypergraph attention already provides a mechanism for learned membership weighting [24]. Group construction determines which methods can exchange information through a hyperedge; membership weights then control their influence within that structure. RQ2 asks how such group relations can be represented and learned, and what they add over pairwise graph representations. The distinction between the information retained by a representation and the operations applied to it is developed in Section 2.2.2.

Industrial control systems provide network records and measurements of the physical process. Protocol-specific features describe communication activity [25], while sensor and actuator series record process behaviour [26]. These observations require different feature preparation and decision units. RQ3 asks how a modular learning framework can analyse both forms and

help researchers compare candidate methods for a new dataset. Configurable preparation and a common learning interface support this model-development task. The industrial studies examine this design through separate evaluations of network records and process windows.

The evaluation problem

Attack libraries such as Foolbox and earlier attack/defence competitions provide established evaluation infrastructure [27, 28]. Reuse across prediction tasks requires explicit attacker access and valid perturbation constraints, together with scoring rules that match the prediction objective [14]. RQ4 examines these requirements through the SAFAIR contest and benchmark [29, 30]. Reusable interfaces support a common execution procedure. Reproducing a historical result also requires the evaluated data and trained models, with the complete experimental configuration.

Scope

The principal empirical studies analyse Dalvik bytecode recovered from Android application packages; Hybroid also uses network traffic recorded during execution. The industrial studies use observations from an IUNO project deployment and the Secure Water Treatment (SWaT) testbed. These studies observe communication records or process measurements under their respective collection conditions.

The adversarial-evaluation work concerns facial-image tasks and a partner histopathology benchmark [31, 32]. A short, attributed PDF-malware comparison illustrates constraints on another input domain [33]. The contribution is evaluation design and measurement analysis; the evaluated defence algorithms remain the work of project partners.

The studies are related through their observation and evaluation choices, with separate datasets and protocols. The IoT work did not motivate Hybroid. The Android systems have not been compared on a common corpus or evaluated here against adaptive attackers. Chapter 7 makes the conceptual comparison and identifies requirements for those subsequent experiments.

1.3 Research Questions

Four research questions specify the two themes. The first two concern cross-modal and higher-order Android representations. The third concerns modular anomaly detection in industrial control systems; the fourth concerns adversarial evaluation. Section 2.7 develops their literature-based motivation as gaps G1 to G4.

RQ1: Multimodal Android malware analysis

How can static program structure and observed network behaviour be jointly represented for Android malware detection and category classification, and how does their combination affect predictive performance compared with either source alone?

Gap G1 is examined through Hybroid, which summarises program graphs and recorded flows per application and concatenates the two feature groups. On a common retained cohort of CICAndMal2017 applications, the joint input is compared with each source alone, separately for each learner and prediction task. The category task includes Benign as one of five classes.

RQ2: Higher-order Android malware analysis

How can higher-order relations within static Android program structure be represented and learned, and to what extent do they improve malware detection and classification over pairwise graph representations?

Gap G2 is examined through HGANN-Mal, which learns membership weights over constructed groups of methods, and its comparison with pairwise graph models. Against non-attentive hypergraph models, construction and weighting change together, so an attention-only effect remains unresolved. The tasks are binary detection and dataset-specific classification: malware-family classification on Drebin, and malware-category classification on CICMalDroid, with Benign as one category.

RQ3: Modular industrial anomaly detection

How can a modular learning framework analyse network and process features for anomaly detection in industrial control systems?

Gap G3 is examined through a modular framework for researcher-guided model development on network and process features, with two separate empirical cases. The first is supervised classification of connection records from a labelled corpus of S7comm network traffic, recorded in the IUNO project from Siemens S7-300 programmable logic controllers [25]. The second is a multivariate detector over process measurements from the public Secure Water Treatment (SWaT) testbed [34].

RQ4: Adversarial robustness evaluation

How can an explicit adversary model be translated into a reproducible benchmark for adversarial attacks and defences, and what does such a benchmark reveal about the relation between clean predictive performance and adversarial robustness?

Gap G4 is addressed through benchmark design and analysis of the resulting measurements. The available aggregates support conclusions about scoring and evaluation requirements, but do not establish a general relation between clean performance and adversarial robustness.

1.4 Contributions

Multimodal Android malware analysis. Hybroid represents static program structure and recorded network flows at application level and combines them in one input vector. Its within-system comparison measures the value of fusion on applications for which both observations are available [5]. Fusion has the highest reported detection F_1 for each evaluated learner. For category classification, fusion improves F_1 for the ensemble classifiers, while the decision tree favours static features. These are descriptive comparisons of the published fold means (Section 3.11).

Higher-order Android malware analysis. HGANN-Mal groups methods by call neighbourhood and semantic similarity. Slice-aware hyperedges add relations around security-sensitive API calls, and learned membership weights control aggregation over the groups [6]. Every evaluated hypergraph configuration has higher mean accuracy over five training runs than the pairwise baselines on each task and corpus; HGANN-Mal exceeds the strongest non-attentive baseline in three of four task–corpus combinations. The operator analysis in Section 4.8 also shows how attentive propagation can distinguish groupings with the same fixed propagation matrix.

Modular industrial anomaly detection. NADICS, Network Anomaly Detection for Industrial Control Systems, separates feature preparation from interchangeable learners to support semi-automated model development on new datasets. Researchers can compare candidate algorithms within a shared workflow and use feature-importance analysis to guide input selection. Its protocol-specific extractor and model interface support the IUNO integration [25, 35]. Separate evaluations on S7comm network traffic [36] from Siemens S7-300 programmable logic controllers report attack classification and false-alarm episodes during benign operation. A generative adversarial network (GAN) extension compares representations of multivariate SWaT process data. Their different recall and false-positive profiles relate the representation choice to the operating objective (Section 5.5).

Evaluating adversarial robustness: benchmark design and metric analysis. SPARTA SAFAIR translates attacker assumptions into a contest protocol and a reusable benchmark with task-specific interfaces [37, 32]. The partner histopathology application documents reuse outside the contest’s facial-image tasks. Chapter 6 derives the zero accuracy loss of constant predictions under label-preserving attacks and analyses the effect of reversing a binary prediction after an alarm. These results identify limitations of accuracy-loss ranking and specify the joint outcomes needed to assess a defended classifier. The contest received no independent attack submissions, so its delivered results remain distinct from the planned adaptive competition.

A comparison across the contributions. Chapter 7 distinguishes the acquisition of additional observations from the computational cost of structuring available evidence. Runtime capture supplies an additional observation to Hybroid, whereas HGANN-Mal constructs its groups from static information. The discussion relates these choices to the conditions required for predictive comparison and complete-system adversarial testing, which informs the common-cohort evaluation proposed as future work.

1.5 Publications, Projects, and Individual Contributions

Table 1.1 identifies disseminated publications and project artefacts that support this dissertation, together with author-list positions and project roles. The prose is revised for the monograph; reproduced or adapted figures and tables retain their source citations. Results are traced to published tables and figures or to surviving internal research records, as specified in each contribution chapter.

Table 1.1. Previously disseminated work and project artefacts used in this dissertation.

Item	Venue or type	Year	Position / role	Chapter
Hybroid [5]	ISC, Springer LNCS	2021	1 of 4	3
HGANN-Mal [6]	ISC, Springer LNCS	2025	1 of 2	4
Graph-based Anomaly Detection for IoT Microservices [7]	PAM	2018	4 of 4	5
IUNO work package 4, visual security control room [25]	Project deliverable	2018	contributor	5
IUNO work package 7, security tools [35]	Project deliverable	2018	contributor	5
SPARTA D7.1, AI systems threat analysis [38]	Public deliverable	2021	contributor	6
SPARTA D7.2, preliminary security mechanisms [31]	Public deliverable	2020	contributor	6
SPARTA D7.3, validation and evaluation plan [37]	Public deliverable	2021	editor	6
SPARTA D7.5, AI systems security mechanisms [33]	Public deliverable	2021	contributor	6
SPARTA D7.6, validation and evaluation report [32]	Public deliverable	2022	editor	6
SAFAIR AI Contest development toolkit [39]	Software repository	2021	sole author	6
Adversarial Machine Learning Benchmark Tool [30]	Software repository	2021	developer	6

Individual contribution

The roles below combine the source author lists with the candidate’s account of the work. For Hybroid, the candidate shares first authorship with Peng Xu [5]. He designed the overall pipeline and its traffic-analysis component, and developed the preprocessing and evaluation framework. He conducted the experiments and wrote the paper. His contribution to graph processing was partial.

HGANN-Mal is a joint publication with the candidate’s supervisor [6]. The candidate designed and implemented the method. He conducted the experiments and wrote the paper.

The candidate contributed the service-graph concept and measurement design to the PAM study [7], retained as an exploratory study in Section 5.3. Two companion studies by Pahl and co-authors remain related work; neither paper lists the candidate as an author [40, 41].

Within IUNO, the candidate designed and implemented NADICS and its extractor for the S7comm and PROFINET industrial protocols. He produced the labelled S7-300 attack corpus, with binary labels and five attack classes, documented in the consortium deliverables [25, 35]. An internal deliverable of June 2017 names him among its authors and records the first version of the tool [42]. For SWaT, he designed and implemented the multivariate GAN pipeline and its preprocessing and scoring procedures, performed the experiments and wrote the internal research report.

Within SPARTA, the candidate contributed to the threat analysis and organisation of the SAFAIR contest, and developed the evaluation software. He wrote Chapter 5 of D7.5 and the whole of D7.6 [33, 32]. His section-level contributions to the other deliverables, and his editing of partner text in D7.3, are specified in Section 6.10. The contest toolkit and the modular benchmark are separate public software artefacts [39, 30]. The partner defence algorithms and their experiments retain their attribution; the dissertation adds the mathematical scoring and response-policy analysis.

1.6 Dissertation Structure

Chapter 2 defines the shared foundations for Android analysis and relational learning, with the network and process concepts needed for Chapter 5. It also establishes adversarial terminology and common performance measures, and closes with gaps G1 to G4. Study-specific methods and comparisons with competing systems remain in the contribution chapters.

Table 1.2 maps the research questions to the evidence in Chapters 3 to 6. The two Android chapters come first and carry the main empirical contribution. Chapter 7 relates the studies through their observation requirements and evaluation principles. Chapter 8 answers the research questions within the available evidence and specifies future work. Appendix A records the complete NADICS feature-importance ranking.

Table 1.2. Research questions and the evidence supporting their answers. Each contribution chapter states its experimental scope and limitations.

Question	Chapter	Evidence
RQ1	3	Application-level joint representation; within-system comparisons with each source alone on a common retained CICAndMal2017 cohort.
RQ2	4	Hypergraph construction and operator analysis; common-input comparisons with pairwise and non-attentive hypergraph models on Drebin and CICMalDroid.
RQ3	5	Modular framework and protocol-specific extraction; session-disjoint S7-300 classification and benign monitoring; SWaT representation comparison at timestamp level.
RQ4	6	Contest and partner-benchmark records; mathematical analysis of scoring and detector responses.

Background

Learning-based security analysis converts observations into representations and evaluates a fitted decision rule on a stated population. Its interpretation depends on the evidence available to the analyser and on the experimental protocol [10]. This chapter defines the foundations shared by the contribution chapters. Android applications and program representations appear first because they support the dissertation's main studies. Section 2.4 supplies the network and process foundations for Chapter 5. Comparisons with specific competing systems remain in the contribution chapters.

2.1 Android Applications and Their Analysis

2.1.1 Android Malware Detection

Malware detection assigns a software artefact to a benign or malicious class. Multiclass malware analysis uses a finer label scheme: a corpus may group malicious applications by family or define broader application categories [1]. Each target partitions the applications differently, so its metrics answer a different question. In this dissertation, Hybroid evaluates binary detection and five-class category classification, with Benign as one category. HGANN-Mal evaluates binary detection and malware-family classification on Drebin, together with category classification on CICMalDroid [5, 6].

Signature and heuristic detectors search for specified indicators of malicious activity. Learning-based detectors acquire observable properties of an application, encode them as a numerical or relational representation. They then fit a decision rule to labelled samples [2, 4]. DREBIN, for example, extracts manifest and bytecode features from an Android package and embeds them in a joint vector space. A linear classifier operates on that representation [2]. Graph-based systems add a structural encoding before classification [1]. The prediction concerns an application even when the representation is assembled from instructions, methods or recorded network flows.

The evidence available to a detector remains bounded by its acquisition process. Static analysis depends on the code recovered from the package and on the analyser's approximations; dynamic analysis depends on the paths exercised during observation. Obfuscation and loading at runtime can conceal relevant behaviour, while changes in applications and attack techniques can weaken associations learned from historical data [4]. The following subsections develop these observation limits. Section 2.6.6 addresses temporal change, related samples and other protocol choices that affect reported performance.

2.1.2 Dalvik Bytecode, Manifest, and Permissions

An APK is an installation archive containing compiled code and application resources [43]. Java and Kotlin compilation produces Dalvik Executable (DEX) files. These are packaged as `classes.dex` and, for multidex applications, additional numbered files [44]. Android Runtime (ART) executes the DEX bytecode format also used by its predecessor, Dalvik [45]. Instructions contain an opcode and operand fields, which may identify registers or constants and refer to methods or strings [46]. The opcode embedding in Section 3.5 omits these operands; call-graph extraction can still use their method references.

Android normally isolates applications through separate Linux user identities and processes [43]. Within an application, activities provide user interfaces and services support work without an activity interface. Broadcast receivers respond to events, while content providers expose data through controlled interfaces. These components have distinct lifecycles, with callbacks invoked by the framework. Static analysis must model those entry points and callback relations to recover the corresponding behaviour [47].

The manifest declares application components and requested permissions, together with intent filters and device requirements. Its metadata can be parsed without reconstructing control flow. DREBIN obtains four of its eight feature groups from the manifest and the remainder from disassembled bytecode [2].

Permissions mediate access to protected data and operations. A request in the manifest records a declared requirement; the actual grant depends on the permission and the platform. In the current runtime-permission model, *dangerous* permissions require a grant before protected functionality can be used. *Signature* permissions instead depend on the application’s signing relationship to the permission’s defining application or operating system, and can protect privileged operations [48, 49]. Selecting dangerous permissions therefore selects one class of protected capabilities.

Permission-to-API mappings also depend on platform version. PScout recovered these mappings from framework source across Android 2.2 to 4.0 and documented gaps in the available API specification [50]. More recent work examines permission deprecation as a source of change in permission-based detection [51]. These findings motivate explicit platform assumptions when sensitive APIs are selected, as in Section 4.5.

2.1.3 Static, Dynamic, and Hybrid Analysis

Static analysis examines an application without executing it. It can inspect code outside the paths exercised by a test harness, subject to the code available and the analyser’s approximations. Static methods accounted for 73% of the deep-learning studies reviewed by Liu et al. for 2014–2021 [4]. Figure 2.1 distinguishes package artefacts from execution observations.

Standard toolkits provide disassembly and call-graph construction [52, 53]; FlowDroid adds lifecycle-aware taint analysis [47]. An analysis restricted to recovered DEX code can omit native functionality or runtime-loaded code. More precise reasoning also has computational

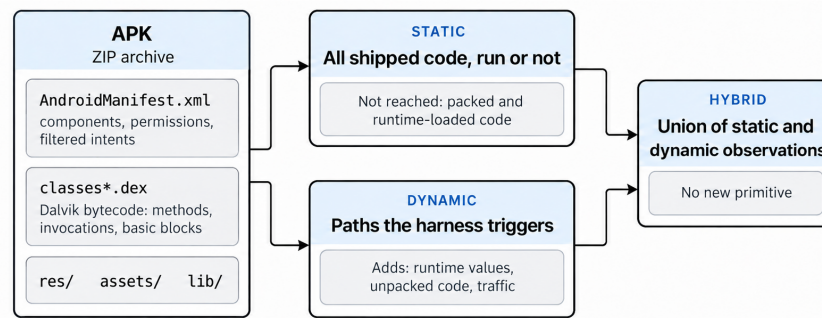


Figure 2.1. Package artefacts and analysis modes. Static analysis reads available code and metadata and infers possible behaviour. Dynamic analysis records behaviour along the execution paths exercised by the harness; network traffic from those executions supplies the additional observation used in Chapter 3.

limits: Moser et al. construct x86 obfuscations whose opaque constants encode instances of the NP-hard 3SAT problem [18].

Dynamic analysis observes an execution through instrumentation. Examples include on-device taint tracking [19] and emulation that reconstructs operating-system and Dalvik-level events [54]. Recorded traffic supplies a further observation channel. Coverage is limited to the paths triggered during the observation period, and applications may recognise an analysis environment and alter their behaviour [55, 56].

Hybrid analysis combines static and dynamic observations. Within-system studies have reported benefits from their combination, with outcomes dependent on the observation types and the stimulation used during execution [57, 20]. Feature-level fusion supplies the derived features together to a classifier, so both observations must first be summarised for the same unit, such as an application. Its additional value is assessed against each modality alone for the same application cohort and prediction task, as in Chapter 3.

2.1.4 Obfuscation, Packing, and Repackaging

Obfuscation transforms code to impede analysis while aiming to preserve its functionality. Legitimate developers use it to protect software: Wermke et al. found developer-applied obfuscation in 24.92% of 1.7 million free Google Play applications [58]. Its use differs across application sources and techniques [59]. Hammad et al. evaluated 29 obfuscation strategies and documented losses in commercial anti-malware detection [60]. A later evaluation of twelve learning-based detectors also found performance losses under obfuscation, temporal change and adversarial perturbations [61].

Packing conceals a payload behind a loader that recovers it during execution. The loader and metadata may remain accessible to static analysis while the original payload requires unpacking. Commercial and custom Android packers use different mechanisms, which limits the coverage of individual unpacking tools [62].

Repackaging reuses an application with modifications; malicious piggybacking inserts additional code into a benign carrier [63]. Such reuse creates code-related samples whose placement across training and test partitions requires attention. Section 2.6.6 addresses the resulting evaluation bias.

2.2 Graph Representations of Programs

A program representation specifies both the entities modelled and the relations between them. The Android chapters use basic blocks and methods at different stages; their graph edges therefore have different meanings.

2.2.1 Control-Flow and Function-Call Graphs

The intra-procedural *control-flow graph* (CFG) used here is a directed graph over the basic blocks of one method. A basic block is a maximal instruction sequence with one entry and no branch except possibly at its end. An edge records a possible transfer of control between blocks [1]. A *call graph* instead has procedures as vertices, with an edge from f to g when a call site in f may invoke g . For Android, this dissertation uses *function-call graph* (FCG) for the method-level object recovered from bytecode [64, 6].

A sound call-graph analysis over-approximates possible calls under its execution model. Practical recovery can introduce infeasible edges and omit unresolved targets, for example through reflection or incompletely modelled callbacks [47]. The recovered graph thus expresses the analyser’s approximation of program structure. Its coverage depends on the toolkit and execution assumptions [52, 53].

CFG embeddings can serve as vertex features in an FCG, so that block-level information and inter-method calls contribute to one representation [1]. Pooling a set of independently embedded CFGs supplies method information without call edges. These constructions must be distinguished when an application vector is described. Other feature choices include abstraction of APIs to packages or families [3], learned CFG features [65, 66], and embeddings of a method’s syntax [67].

Program dependence and slicing. Data dependence relates a value’s definition to uses that it can affect. Control dependence relates execution of a statement to the outcome of a predicate. A program dependence graph represents these relations explicitly. A backward slice identifies statements and predicates that may affect selected values at a specified program point. Interprocedural slicing follows dependencies across procedure boundaries and must account for calling context [68]. A call-graph neighbourhood alone does not supply these data and control dependencies. Section 4.5 uses slicing around sensitive API call sites; the definition here does not establish the scope or precision of that implementation.

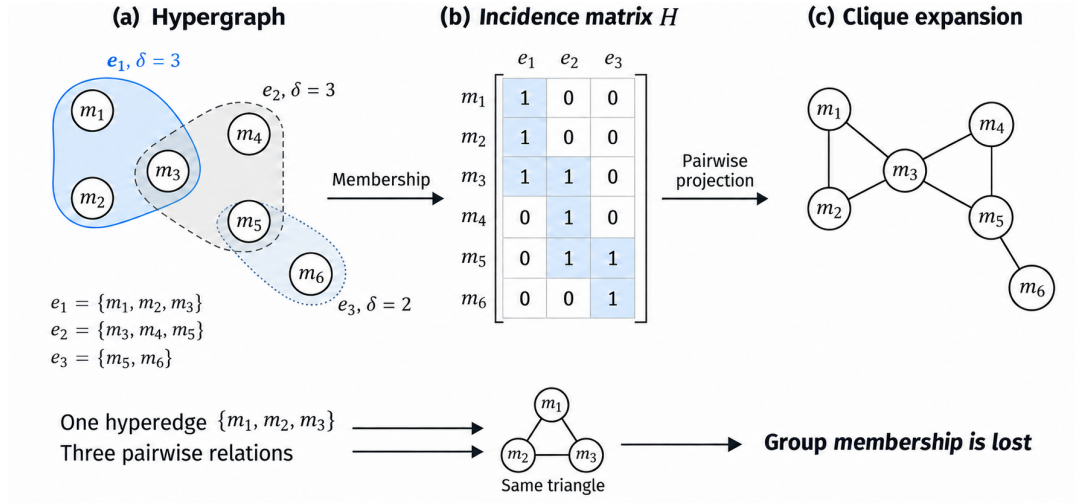


Figure 2.2. The same six methods represented as (a) a hypergraph, (b) its incidence matrix H , and (c) its unweighted clique expansion. A matrix entry is one precisely when the method belongs to the corresponding hyperedge. Hyperedge e_3 joins two vertices and is therefore an ordinary graph edge. The lower example shows that the triangle over m_1, m_2, m_3 arises both from the single hyperedge e_1 and from three separate pairwise relations. The projection preserves pairwise adjacency but does not uniquely determine the original hyperedge memberships [21].

2.2.2 Hypergraphs and Higher-Order Relations

An undirected hypergraph $\mathcal{H} = (\mathcal{V}, \mathcal{E}, w)$ has a finite vertex set $\mathcal{V}$ and a family $\mathcal{E}$ of nonempty subsets of $\mathcal{V}$. Each hyperedge e carries a weight $w(e) > 0$. The incidence matrix $H \in \{0, 1\}^{|\mathcal{V}| \times |\mathcal{E}|}$ has entry $h(v, e) = 1$ exactly when $v \in e$.

The vertex degree and hyperedge degree are

$$d(v) = \sum_{e \in \mathcal{E}} w(e) h(v, e), \quad \delta(e) = \sum_{v \in \mathcal{V}} h(v, e) = |e|.$$

The diagonal matrices D_v , D_e and W contain vertex degrees, hyperedge degrees and hyperedge weights, respectively [21]. An undirected simple graph is the special case in which $\delta(e) = 2$. A hyperedge records group membership; it does not by itself encode call direction or execution order.

Clique expansion replaces each hyperedge by pairwise edges between its members. An unweighted projection cannot distinguish one hyperedge on three vertices from three separate pairwise relations on those vertices. *Star expansion* introduces one vertex per hyperedge and links it to the member vertices [22]. If the two vertex types and their incidence links are retained, this bipartite representation preserves the hyperedge memberships. Figure 2.2 illustrates the loss in the unweighted clique projection.

Retaining a hypergraph and exploiting its group structure are separate properties. Agarwal et al. reduce several spectral learning formulations to graph-expansion operators [22]. For the random walks studied by Chitra and Raphael, *edge-independent* vertex weights give a transition matrix that can be realised on a weighted clique expansion. They also exhibit *edge-dependent*

cases for which this equivalence fails [69]. These results concern the specified walks; other operators require their own analysis.

Wang and Kleinberg characterise ambiguities in reconstructing hypergraphs from unweighted clique projections [70]. Kang et al. evaluate expansion methods through learned embeddings and hypergraph reconstruction, with downstream comparisons [71]; those experiments assess the expansion-and-learning procedure together. A recent preprint organises hypergraph architectures by the patterns they can represent [72]. For program analysis, constructing hyperedges from an FCG adds a chosen grouping of recovered methods. Its value for classification remains an empirical question, distinct from the information retained by the representation.

2.3 Learning over Graphs

Graph learning assigns trainable operations to the structures defined above. The distinction between vertex-level computation and application-level prediction is important for both Android studies.

2.3.1 Message Passing

In a message-passing network, each vertex has a state that is updated from neighbouring states over a fixed number of rounds. Aggregation is invariant to the order in which neighbours are presented [73]. In the GCN formulation of Kipf and Welling, graph degrees determine the normalised neighbour coefficients, while feature transformations remain trainable [74].

A graph-level *readout* combines the final vertex states into a fixed-width representation. Sums or means are simple examples; learned invariant readouts are also possible [73, 75]. Reordering vertices should reorder vertex outputs without changing the graph-level prediction. Shared local operations and an invariant readout allow graphs of different sizes to be processed by the same model. In the Android chapters, the final prediction concerns an application, even when the intermediate learning operates on blocks or methods.

For standard neighbourhood-aggregation networks with corresponding initial node features, graph discrimination is bounded by one-dimensional Weisfeiler–Leman colour refinement. This procedure repeatedly refines a vertex’s label from its current label and the multiset of neighbouring labels. Injective aggregation and readout can attain that bound [75]. Depth introduces a separate concern: repeated normalised propagation can reduce distinctions between vertex representations, the effect termed *oversmoothing* [76].

2.3.2 Attention

Graph attention learns scores for available vertex–neighbour pairs and normalises them within each neighbourhood to weight the messages [77]. Content-dependent weighting is also used in sequence attention [78]. The scoring function determines which relative weightings the model can express.

The original GAT's additive score has a static-ranking restriction: query vertices rank their shared candidate neighbours in the same order. GATv2 changes the scoring function to permit query-dependent rankings [79]. The restriction concerns ordering, not equality of the numerical coefficients. Section 4.8 applies the same argument to the additive hyperedge score used by HGANN-Mal.

Fang et al. analyse scoring capacity under explicit parameter and architecture assumptions and evaluate models on pairwise graph tasks. Their reported means show GAT above the non-attentive baselines on some node-classification datasets and below them on others [80, Table 2]. This supports task-dependent evaluation, rather than a universal ranking of attentive and non-attentive models. The findings do not establish the behaviour of attention over Android hypergraphs.

2.3.3 Hypergraph Convolution

Hypergraph convolution aggregates vertex states within hyperedges and returns the resulting messages to their member vertices. HGNN and HGNN+ specify such propagation through the incidence structure and degree normalisation [81, 82]. Other designs approximate hyperedges by graph edges [83] or learn multiset aggregation functions [84]. Equivariant networks and higher-order Transformers provide further approaches to permutation-aware hypergraph learning [85, 86].

Hypergraph attention assigns learned real-valued weights to incidences [24]. In the HGANN-Mal operator, this changes the strength of existing memberships while their support remains fixed. Learning those weights therefore differs from changing which methods belong to a hyperedge. Earlier Android hypergraph work used call-neighbour and permission-based groups with non-attentive operators [23].

Averaging within hyperedges favours smooth signals. Li et al. analyse the role of high-frequency components in hypergraph learning [87], while HGNNv2 introduces a stability-oriented design based on anisotropic diffusion [88]. These developments address particular properties of propagation. To attribute an empirical difference to attention, its learned weights must be compared with a fixed-weight alternative on the same hyperedges, with a common normalisation convention. Chapter 4 separates its operator analysis from the comparisons of complete configurations.

2.4 Detection in Networked and Cyber-Physical Systems

Network and industrial detectors operate on several observation units. Traffic can be represented by flow records or service interactions; a process detector instead receives measurements of the physical system. Chapter 5 evaluates network records and process measurements, with service interactions retained as an exploratory example.

2.4.1 Anomaly Detection as a Security Task

Signature-based detection matches specified attack patterns, whereas anomaly detection identifies deviations from a reference model of expected behaviour [89, 90]. An anomaly detector may return a binary label or an anomaly score. A threshold converts the score into a decision and therefore determines which deviations are reported [90].

The unit and context of the decision define what constitutes a deviation. A point anomaly is unusual as an individual observation. A contextual anomaly is unusual under a particular condition, such as an operating state, while a collective anomaly arises from a related group whose individual elements need not be unusual [90]. These distinctions matter for the studies in Chapter 5: the SWaT method derives timestamp decisions from multivariate windows (Section 5.5), and the exploratory service-graph method evaluates a relation against learned communication structure (Section 5.3).

An unusual observation can reflect legitimate activity, so an anomaly score requires a security interpretation [91]. Learning regimes also depend on the available labels. Supervised classification uses labelled examples of the target classes. Normal-only training estimates a reference from observations assumed benign, while unsupervised methods work with unlabelled data under assumptions about its composition [90]. Labels used for feature selection or threshold choice make the development procedure partly supervised even when the model's fitting objective uses no class labels.

Local-density methods compare a point with its neighbourhood [92], and isolation methods exploit the short partition paths of isolated observations [93]. Reconstruction methods encode observations and attempt to reproduce them; reconstruction discrepancies provide scores relative to the fitted representation [94, 95]. Relational methods can assess vertex attributes together with graph structure [96]. Each method must specify its normality reference and the unit it scores.

2.4.2 Network Flows as a Unit of Observation

A conventional IP flow groups packets by source and destination addresses and ports, together with the transport protocol. Exporters summarise the traffic through quantities such as duration, packet counts and byte totals; direction and timeout rules determine the exported records [97]. Payload-independent features remain available when application payloads are encrypted, provided the corresponding headers can be observed. Their distribution can still change with protocols or execution conditions.

Flow records are an observation choice, not a universal unit of network behaviour. Service messages and layer-2 industrial traffic require other identifiers. Section 3.7 defines how Hybroid aggregates bidirectional flow records into an application-level vector. Section 5.4.1 defines the windowed industrial connection records used by NADICS; Section 5.3 illustrates service relations in the exploratory IoT study.

2.4.3 Communication Structure in Industrial and IoT Networks

Industrial communications often repeat under a fixed operating mode. Barbosa et al. exploit this regularity through SCADA flow whitelists, while also observing false alarms from rare legitimate activity [98]. Yoon and Ciocarlie model communication patterns between host pairs [99]. Traffic graphs also describe host interactions [100, 101], and enterprise-network studies infer service dependencies from passive observations [102, 103, 104].

Cyber-physical detection can use sensor and actuator series to assess the controlled process [26, 105, 34]. These observations differ from the packets that transport them. In a related task, Maali et al. evaluate *IoT device identification* across operating modes, collection periods and sites [106]. Their transfer results concern device identification; they motivate examining changing observation conditions without supplying a detection rate for industrial anomalies.

2.4.4 Datasets and Measurement Limits

Dataset construction can introduce predictive associations unrelated to the intended threat. KDD 99 contains redundant records [107]; later audits identify construction artefacts in CICIDS2017, UNSW-NB15 and other intrusion datasets [108]. Kus et al. distinguish detection of attack types represented during training from performance on types absent from training [109].

Wolsing et al. review 70 publications and evaluate four deliberately simple industrial detectors against selected comparators on industrial datasets. Their results show that simple process-value tests can be competitive with more complex methods [110]. The comparison supports including appropriate simple baselines under a stated protocol. The general evaluation criteria are developed in Section 2.6.6.

2.5 Adversarial Machine Learning

Adversarial machine learning studies attackers who choose changes to training data or inference inputs with the aim of altering model behaviour, together with methods for resisting those changes [111]. Such strategic manipulation differs from incidental noise or distributional change because the attacker selects the modification to serve an objective. A robustness statement is therefore conditional on the attack goal, the attacker's knowledge and the permitted transformations [14]. Adversarial evaluation tests a learning system under those conditions.

Threat models. A threat model states the attacker's goal and knowledge, together with the capabilities and constraints governing permitted modifications [112, 113]. White-box evaluation gives access to the model's architecture and parameters; a perfect-knowledge formulation can also include the training data [111]. Black-box attacks can use prediction queries or transfer examples from a substitute model, with the available interface stated explicitly [114, 28]. Grey-box access denotes partial knowledge. In the grey-box evasion

setting of Chapter 6, training is fixed; this capability constraint is distinct from knowledge of the training data. A survey of 271 practitioners documents mismatches between academic access assumptions and deployment conditions [115].

Classes of attack. Evasion modifies test-time inputs to change a trained model’s decisions [116]. Poisoning intervenes in training data or the learning process [117, 118]. Training-time backdoor attacks implant behaviour activated by a trigger [119]. Model-extraction attacks instead aim to reproduce a deployed model through access to its prediction interface [120]. Chapter 6 evaluates evasion; the Android studies report predictive performance on historical samples.

Feature space and problem space. A feature-space attack modifies the representation supplied to the learner. A problem-space attack modifies the underlying artefact. Pierazzi et al. formalise constraints on the available transformations and preservation of semantics, alongside plausibility and robustness to preprocessing [16]. For malware, the modified package must retain its malicious functionality as well as remain valid. A successful feature-space perturbation need not correspond to such a package. Android studies illustrate both feature-space attacks [121] and functionality-preserving changes to applications that affect their extracted call graphs [122].

Adaptive attacks and robustness. An adaptive attack is designed against the specific defended system, including its preprocessing and response policy [15]. Gradient masking can obstruct an attack’s optimisation without preventing adversarial errors [13]. Robustness claims therefore require a stated threat model and budget, together with attacks appropriate to the complete defence [14]. FGSM and PGD provide reference gradient-based methods [123, 124].

Reusable attack libraries such as Foolbox provide common model interfaces and attack criteria [27]; the NIPS 2017 competition evaluated attacks and defences submitted by independent teams [28]. Such infrastructure supports empirical evaluation. Its results still depend on input validity and scoring conventions, which Section 2.6.5 defines for the present dissertation.

2.6 Performance Measures and Evaluation Methodology

This section defines the measures used in Chapters 3 to 6, with explicit observation units and aggregation rules. Dimensionless scores are proportions: m corresponds to $100m\%$, and $m_1 - m_2$ to $100(m_1 - m_2)$ percentage points.

Table 2.1. Decision units and targets in the evaluated classification and detection tasks. Attribute accuracy counts the individual label decisions defined in Section 2.6.3.

Study (chapter)	Decision unit	Target
Hybroid (3)	Application	Binary or category label
HGANN-Mal (4)	Application	Binary label; family or category
S7-300 (5)	Connection record	Normal or attack type
SWaT (5)	Timestamp	Normal or anomalous
SAFAIR contest (6)	Image	Identity or binary attributes
Partner benchmark (6)	Histopathology patch	Binary tissue class

2.6.1 Observation Units and Confusion Counts

Let $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$ be a labelled evaluation set with $N > 0$, and let $\hat{y}_i = h(x_i)$ be the prediction of a fitted classifier. For a single-label task with class set $\mathcal{Y}$, the confusion matrix $\mathbf{C}$ records true labels in its rows and predicted labels in its columns:

$$C_{ab} = \sum_{i=1}^N \mathbf{1}\{y_i = a, \hat{y}_i = b\}, \quad a, b \in \mathcal{Y}. \quad (2.1)$$

In binary security detection, 1 denotes the labelled attack or malware class and 0 the normal or benign class. With both axes ordered as $(0, 1)$,

$$\mathbf{C}_{\text{bin}} = \begin{pmatrix} \text{TN} & \text{FP} \\ \text{FN} & \text{TP} \end{pmatrix}, \quad N = \text{TP} + \text{TN} + \text{FP} + \text{FN}. \quad (2.2)$$

True positives and true negatives are correct decisions. A false positive flags a negative observation; a false negative misses a positive observation [125].

Table 2.1 maps the decision units and targets of the contribution chapters. The input representation can have a different granularity: the SWaT model processes windows, then assigns one prediction to each timestamp. Timestamp metrics give longer attacks more weight; episode-level assessment requires a separate matching rule and denominator [126]. Collapsing attack types into one positive class turns errors between those types into correct binary detections (Section 5.4.2).

2.6.2 Binary Detection Measures and Class Prevalence

Accuracy A is the fraction of all observations classified correctly. Precision P is the fraction of positive predictions that are correct, whereas recall R is the fraction of positive observations detected:

$$A = \frac{\text{TP} + \text{TN}}{N}, \quad P = \frac{\text{TP}}{\text{TP} + \text{FP}}, \quad R = \frac{\text{TP}}{\text{TP} + \text{FN}}. \quad (2.3)$$

Recall is also called sensitivity or the true-positive rate (TPR). The F_1 score combines precision and recall through their harmonic mean:

$$F_1 = \frac{2\text{TP}}{2\text{TP} + \text{FP} + \text{FN}}, \quad H(P, R) = \frac{2PR}{P + R} = F_1 \quad \text{when the ratios are defined.} \quad (2.4)$$

The count form omits true negatives and gives $F_1 = 0$ when $TP = 0$ and $FP + FN > 0$. We use the continuous extension $H(0, 0) = 0$ when both defined rates are zero [125, 126].

False-positive and false-negative rates (FPR and FNR) measure class-conditional errors. Specificity, or true-negative rate (TNR), complements FPR; balanced accuracy averages the two class recalls [125, 127]:

$$\begin{aligned} \text{FPR} &= \frac{\text{FP}}{\text{FP} + \text{TN}}, & \text{TNR} &= \frac{\text{TN}}{\text{TN} + \text{FP}} = 1 - \text{FPR}, \\ \text{FNR} &= \frac{\text{FN}}{\text{TP} + \text{FN}} = 1 - R, & \text{BAcc} &= \frac{R + \text{TNR}}{2}. \end{aligned} \quad (2.5)$$

The false fraction among positive predictions is $1 - P = \text{FP}/(\text{TP} + \text{FP})$, with a different denominator from FPR. Zero denominators make ratios undefined unless an explicit replacement or exclusion rule is given.

Let $\pi = (\text{TP} + \text{FN})/N$ be the positive-class prevalence. Writing precision and accuracy in terms of the class-conditional rates gives

$$\begin{aligned} P(\pi) &= \frac{\pi \text{TPR}}{\pi \text{TPR} + (1 - \pi) \text{FPR}}, \\ A(\pi) &= \pi \text{TPR} + (1 - \pi) \text{TNR}. \end{aligned} \quad (2.6)$$

Bayes' rule gives the precision identity [128]. Varying π at fixed TPR and FPR gives a conditional prevalence-sensitivity calculation (Section 4.10). Balanced accuracy removes the explicit prevalence weights, but changes in class-conditional data distributions can still change both recalls.

2.6.3 Multiclass and Multilabel Aggregation

For K mutually exclusive classes, treating class k as positive and the remainder as negative gives classwise measures M_k . Let n_k denote the support of class k and $\pi_k = n_k/N$. Macro and support-weighted averages are

$$M_{\text{macro}} = \frac{1}{K} \sum_{k=1}^K M_k, \quad M_{\text{weighted}} = \sum_{k=1}^K \pi_k M_k. \quad (2.7)$$

Multiclass balanced accuracy is R_{macro} . Macro F_1 gives each class equal weight, whereas weighted F_1 reflects the class distribution. Chapter 5 reports both because normal records dominate the S7-300 data.

Micro averaging pools the one-versus-rest counts. For a single-label task with one prediction per observation and all classes included, it reduces to

$$P_{\text{micro}} = R_{\text{micro}} = F_{1,\text{micro}} = R_{\text{weighted}} = A = \frac{\text{tr}(\mathbf{C})}{N}. \quad (2.8)$$

This identity assumes defined class recalls or an explicit zero-fill rule [127]. Here, macro F_1 means the average of the classwise F_1 scores, which differs from the harmonic mean of macro

precision and recall [129]. Reports must therefore state the class set and zero-denominator rule.

Multilabel prediction assigns several binary labels to each observation. In the facial-attribute task, the reported accuracy averages the J label decisions for each observation:

$$A_{\text{attr}} = \frac{1}{NJ} \sum_{i=1}^N \sum_{j=1}^J \mathbf{1}\{\hat{y}_{ij} = y_{ij}\}. \quad (2.9)$$

Chapter 6 uses $J = 40$; the score does not require every attribute of an image to be correct [32].

2.6.4 ROC Curves and Area under the Curve

For scores s_i whose larger values favour the positive class, thresholding gives $\hat{y}_i(\tau) = \mathbf{1}\{s_i \geq \tau\}$. Varying τ traces the receiver operating characteristic (ROC) points $(\text{FPR}(\tau), \text{TPR}(\tau))$ [125]. With $r(u)$ the linearly interpolated TPR at FPR u , the empirical area under the curve (AUC) is

$$\begin{aligned} \text{AUC} &= \int_0^1 r(u) \, du \\ &= \frac{1}{N_+ N_-} \sum_{i: y_i=1} \sum_{j: y_j=0} \left[\mathbf{1}\{s_i > s_j\} + \frac{1}{2} \mathbf{1}\{s_i = s_j\} \right], \end{aligned} \quad (2.10)$$

where $N_+, N_- > 0$ count positive and negative observations. AUC measures positive–negative pair ranking, with half credit for ties. Perfect and independent random rankings give one and expected 0.5, respectively. One confusion matrix gives only an operating point; AUC specifies neither a deployment threshold nor probability calibration.

Multiclass one-versus-rest curves compare each class with the remainder. Unweighted macro AUC is $K^{-1} \sum_k \text{AUC}_k$; other weighting or curve-averaging conventions must be stated [125]. Hybrid reports per-class ROC AUC (Chapter 3).

2.6.5 Performance under Adversarial Input

An adversarial performance measure requires a fixed threat model and valid perturbed inputs [14]. For an attack a applied to matched, single-label observations, let A_{clean} be the clean accuracy and define

$$A_a = \frac{1}{N} \sum_{i=1}^N \mathbf{1}\{h(a(x_i)) = y_i\}, \quad \Delta_a = A_{\text{clean}} - A_a. \quad (2.11)$$

The SAFAIR score in Chapter 6 uses this signed accuracy difference [32]. For the attribute task, Equation 2.9 supplies both terms. Report clean and attacked accuracy alongside the loss: equal differences can occur at different levels of classification performance.

Attack success measures attainment of a specified goal. One explicit convention restricts evaluation to initially correct examples, $I_{\text{corr}} = \{i : h(x_i) = y_i\}$. For target labels t_i , let $I_t = \{i \in I_{\text{corr}} : t_i \neq y_i\}$. Conditional untargeted and targeted attack-success rates are then

$$\begin{aligned} \text{ASR}_{\text{untargeted}} &= \frac{1}{|I_{\text{corr}}|} \sum_{i \in I_{\text{corr}}} \mathbf{1}\{h(a(x_i)) \neq y_i\}, \\ \text{ASR}_{\text{targeted}} &= \frac{1}{|I_t|} \sum_{i \in I_t} \mathbf{1}\{h(a(x_i)) = t_i\}. \end{aligned} \tag{2.12}$$

Report eligible-set sizes and any alternative eligibility rule; empty sets give undefined rates. These goal-specific rates differ from Δ_a . Chapter 6 develops attribute criteria and per-example attack-suite aggregation as analytical measures, not recovered contest scores. Finite attacks do not certify worst-case robustness [14].

2.6.6 Experimental Protocol and Bias

Malware evaluation should use training samples collected before the test period. The test set should match the conditions expected during deployment [9]. Preprocessing and model selection must exclude test information; data-dependent transformations are fitted within each training fold [10]. Repackaged applications require checks for dependence across partitions [11].

Baseline comparisons should use a common evaluation protocol on shared data partitions and include simple alternatives [10]. Claims about individual components require controlled ablations. Reports must distinguish class averaging from averaging across folds or training runs. For nonlinear measures such as F_1 , the mean of individual scores can differ from the score calculated from pooled counts. Hybroid uses five-fold cross-validation, while HGANN-Mal reports five-run means on a fixed partition. The contribution chapters specify their protocols and the resulting limits on interpretation.

2.7 Summary and Research Gaps

The observation model determines which evidence reaches a detector. Program graphs and hypergraphs structure that evidence, while their learning operators determine how it is combined. Network and process studies make corresponding choices at other observation scales. Evaluation then relates the predictions to a defined population and, where relevant, to permitted adversarial modifications. The following gaps connect these foundations to the research questions in Section 1.3.

G1. Combining program structure with observed behaviour. MARVIN and the study by Onwuzurike et al. report benefits from combining static and dynamic features in within-system comparisons [57, 20]. Those results concern particular observation types and prediction tasks. A different pair first needs a joint representation at a common unit of analysis. Whether the benefit persists then requires comparison of that representation with each modality alone on

the same samples. RQ1 addresses both steps for program structure and network behaviour in binary detection and malware-category classification.

G2. The empirical value of higher-order program representations. Prior Android work uses call-neighbour and permission-based hyperedges for application classification [23]. It remains open how program groups should be constructed and weighted for malware analysis, and how much such representations add over pairwise graphs under a common protocol. Attributing a gain to construction or to weighting would further require varying them separately. RQ2 addresses this question for binary detection and dataset-specific malware classification.

G3. Representations for modular industrial anomaly detection. Communication whitelists and host-pair models use relations observed between devices [98, 99], while process detectors use sensor and actuator measurements [34]. These observation types require different feature preparation and decision units. The design question is how a modular learning framework can accommodate both forms and support researcher-guided comparison of candidate pipelines for a new dataset. This requires configurable data preparation and a common interface for learning methods, with model selection separated from final testing. RQ3 examines the framework design through NADICS and its separate network and process evaluations.

G4. Reproducible and interpretable adversarial evaluation. Attack libraries and public competitions provide established evaluation infrastructure [27, 28]. Reuse across prediction tasks still requires explicit attacker access and valid perturbation constraints. Scoring must match the prediction objective, and attacks must address the complete defended system [14]. The question is how these requirements can be implemented in reusable protocols and which conclusions joint clean and attacked measurements support. RQ4 addresses this benchmark-design and measurement problem.

Multimodal Android Malware Analysis with Hybroid

3.1 Research Problem and Contribution

An Android package can contain code paths that a test never executes, so its traffic capture cannot describe communication those paths might produce. Conversely, flow statistics describe observed communication without identifying the code path that produced it. Static analysis can recover program structure beyond the executed paths, subject to the limits described in Section 2.1.3. For malware classification, the unresolved question is whether combining these partial observations improves predictions over either source alone on the same applications. This possible complementarity motivates Hybroid; its value must be measured for the chosen representations and prediction task.

This chapter addresses RQ1, stated in Section 1.3: How can static program structure and observed network behaviour be jointly represented for Android malware detection and category classification, and how does their combination affect predictive performance compared with either source alone? It examines gap G1 through a joint representation and a comparison on one application cohort, separately for each prediction task and learner.

Program graphs supply context for the code features: control-flow edges relate basic blocks, while call-graph edges relate methods. Hybroid combines learned opcode features with this recovered structure in its static branch [5]. As introduced in Sections 2.2.1 and 2.3.1, graph propagation combines a block's instruction features with neighbouring block states. The code representation is fitted using application labels for malware detection, so training can adapt its features to that task. Its fixed-width output supplies the static input to the final classifier. The network branch supplies flow statistics, such as duration and byte counts, without requiring payload contents. Both branches must describe the same application before their features can be combined.

Figure 3.1 presents the pipeline. Each sample enters as an APK paired with traffic recorded on a physical handset (1); the branches process these observations in parallel (2). The static branch yields a 64-dimensional code vector, and the network branch aggregates flow records into 13 features per application. This common unit permits feature-level fusion: concatenating the vectors gives one classifier access to both feature groups (3). This final supervised learner fits their associations with the target labels (4), with separate models for binary detection and five-class prediction into Benign or one of four malware categories (5). Representation learning in the code branch and final classification thus serve distinct roles; Sections 3.5 to 3.8 define them.

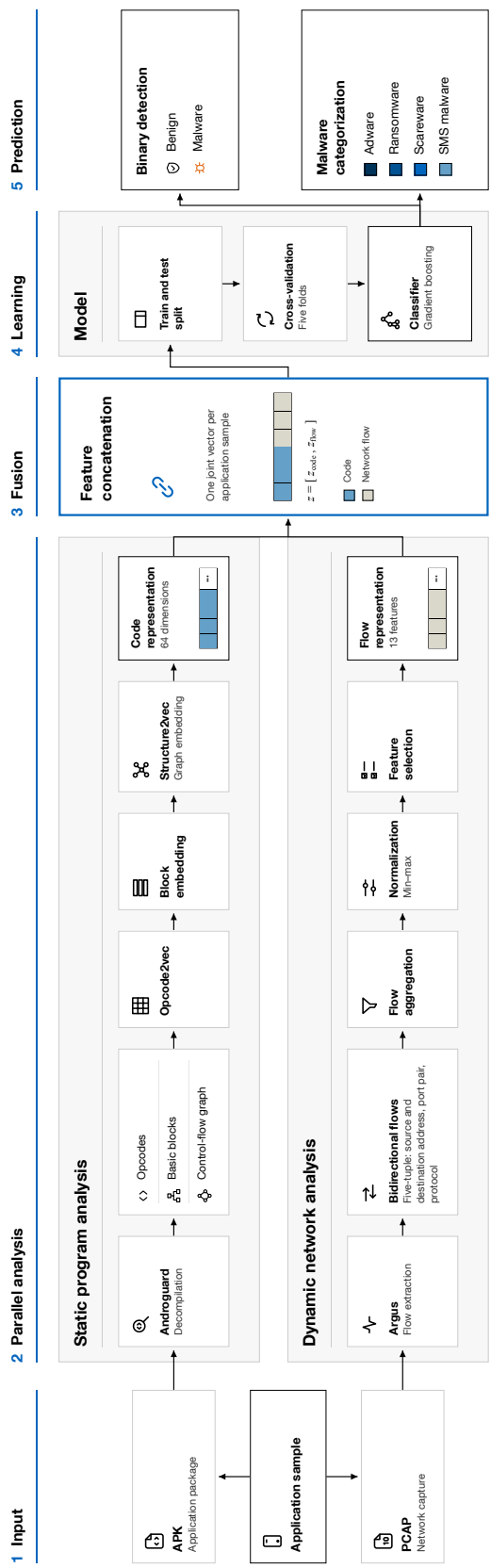


Figure 3.1. The Hybroid architecture, redrawn by the candidate from [5].

The contribution comprises this multimodal pipeline and its within-system evaluation. Section 3.11 compares the joint input with each branch alone for the same learner and task. The reported mean F_1 gains over the better single modality are one or two percentage points for detection; category-classification differences range from minus two to plus three points, depending on the learner. These descriptive results support a task-dependent benefit from fusion. The evaluation leaves the relative merits of alternative fusion mechanisms open.

The work was published at ISC 2021 [5]. The candidate is the first of four authors, with shared first authorship recorded for the first two. He designed the overall pipeline and its network-traffic analysis, together with the framework used for preprocessing and evaluation. He conducted the experiments and wrote the manuscript. His contribution to graph processing was partial, as stated in Section 1.5.

3.2 Related Work

Static Android detectors use features available in the application package. DREBIN combines manifest declarations with features extracted from bytecode, including API calls and network addresses, in a sparse representation classified by a linear model [2]. It is one of the baselines in Section 3.9. API-level detectors include DroidAPIMiner [130], while other approaches combine permissions and API calls [131]. Factorisation machines and static behaviour classifiers also use manifest and API feature vectors [132, 133].

Opcode-based methods describe instructions at a lower level. Canfora et al. evaluate opcode n -grams for malware detection [134]; McLaughlin et al. train convolutional networks on opcode sequences [135]. Hybroid uses opcode embeddings as features within a structured program representation. Basic blocks and their control-flow relations provide the intermediate units described in Sections 3.5 and 3.6.

Graph-based methods differ in both the relations they encode and their vertex features. Adagio maps neighbourhoods of a function-call graph to features for a linear classifier and permits the associated code structures to be inspected [64]. apk2vec learns application representations from several graph views [136]. MaMaDroid models transitions between abstracted API calls as Markov chains; package- or family-level abstraction reduces dependence on individual method identities [3]. Attributed control-flow graphs have also been used for binary-code similarity and malware classification [66, 137]. These methods operate at different program granularities, as defined in Section 2.2.1. This distinction guides the static design in Hybroid: opcode embeddings describe block contents, and graph propagation incorporates neighbouring blocks. The rationale is to use instruction information together with structural context; the modality comparison evaluates that code branch as a whole.

The choice of vertex features remains relevant in later work. Tran et al. augment function-call graphs with function metadata and code embeddings, and report improved classification under distribution shift relative to topology-based features [138]. Their MalNet-derived benchmarks provide evidence for that comparison, rather than a validation of Hybroid's opcode representation. A broader survey describes the extraction and learning choices used in graph-based malware detection [1].

Dynamic network analysis measures communication during execution. Arora et al. use traffic statistics with a rule-based classifier, and Zulkifli et al. evaluate a decision tree on network features [139, 140]. CREDROID examines network activity, including transmitted information [141]. In the broader malware-traffic literature, learned representations operate on raw packets or sessions [142, 143]. The observation unit determines which traffic is represented together and what a prediction label describes. Application-level aggregation in Hybroid permits fusion with the code vector, at the cost of the record order and variation discarded by averaging (Section 3.7).

The CICAndMal2017 study classifies Android malware from flow features and provides the dynamic baseline used here [144]. A later extension combines network-flow and API features [145]. Hybrid methods cover a wider range of observation pairs and fusion mechanisms [146]. MARVIN compares static, dynamic and combined features, while the study by Onwuzurike et al. compares the modalities through a common behavioural model [57, 20]. Both report advantages for the combination in their evaluated settings. These studies establish precedents for hybrid analysis; the observation pair and fusion mechanism determine the design examined in each case.

HGDetector later combines static network-related call-graph features with graphs derived from network traffic [147]. Its experiments use CICAndMal2017 and a filtered CICMalDroid2020 corpus, with gains that depend on the available traffic. This supports examining fusion under explicit observation conditions. Its reported gains are not directly comparable with Hybroid's because the representations and evaluation cohorts differ.

The earlier hybrid studies do not establish whether adding a learned program-graph vector to application-level flow statistics improves classification for the tasks and cohort studied here. RQ1 tests that particular combination against each feature group alone on the same application cohort and with the same learner type. Binary detection and category classification require separate comparisons because their labels distinguish applications in different ways. The observation requirements and evaluation protocol are specified next.

3.3 Observation and Threat Model

The static observation is the Dalvik bytecode recovered from the APK, including its methods and call sites. The dynamic observation is the packet capture associated with an application's execution on a physical handset. Argus converts that capture into bidirectional flow records. The classifier receives features derived from both observations after acquisition and preprocessing [5].

This design supports offline application analysis when an APK and a corresponding traffic capture are available. In the reported evaluation, those inputs were supplied by CICAndMal2017; the candidate's pipeline processed the recorded traffic. A new application would likewise require a capture before the fused classifier could be used. Chapter 4 examines a different observation requirement: analysis from the application package alone.

Table 3.1. CICAndMal2017 source-corpus composition before extraction failures are excluded. Malware-category counts are recomputed from the official family listing [148]; the 1,700 benign samples are the package/capture pairs used in [5].

Class	Families	Samples	Share
Benign		1,700	80.0 %
Adware	10	104	4.9 %
Ransomware	10	101	4.8 %
Scareware	11	112	5.3 %
SMS malware	11	109	5.1 %
Malware	42	426	20.0 %
Total		2,126	100.0 %

The static branch covers the bytecode visible to its analyser. Behaviour implemented in native libraries or code loaded during execution can be absent from that representation. The dynamic branch records network activity during the capture window, which may include consequences of such behaviour, but it supplies neither system-call traces nor decrypted payload contents. It therefore provides evidence about the observed execution rather than all possible executions.

The evaluation is non-adaptive: the malware samples were collected before the detector was developed, and no attacker modifies an application in response to its predictions. Attacker knowledge and perturbation budgets are not specified. The results consequently concern classification of the collected samples, with adversarial robustness remaining a separate evaluation question.

3.4 Dataset and Experimental Protocol

The evaluation uses CICAndMal2017, which supplies application packages and recorded network traffic [144]. Benign applications were collected from Google Play across 2015 to 2017; malicious applications came from VirusTotal and the Contagio archive. Traffic was captured on three physical handsets with scripted user behaviour and live accounts on messaging and social platforms. The capture scenarios cover installation and periods before and after a reboot [148]. These paired observations support the comparison of static and dynamic representations required by RQ1.

Table 3.1 gives the corrected source-corpus composition. The malware counts total 426 across 42 families; the category rows in the Hybrid publication instead sum to 446. The official dataset documentation records 5,065 installed benign applications, of which the Hybrid study uses 1,700 package/capture pairs.

Both analysis branches excluded the 47 graph-extraction failures from the 2,126 source samples and used the same remaining 2,079 applications. This corresponds to processing coverage of 97.8%.

The paper reports five-fold cross-validation with random shuffling and scores averaged over the folds [5]. Implementation used Python with scikit-learn, TensorFlow and Keras, on a server with 32 cores, 128 GB of memory and a 16 GB GPU. Accuracy and F_1 are reported alongside precision, recall and ROC AUC.

A supplementary static-only experiment used 45,592 malicious and 90,313 benign applications from AndroZoo, VirusShare and VirusTotal. Its 135,905 samples were divided at random into 80% for training and 20% for testing [5]. This experiment evaluates the code representation on a larger corpus; it provides no additional modality comparison. Its detection results are reported in Section 3.9.

3.5 Opcode and Basic-Block Representation

Dalvik instructions encode an operation and its operands. Operand fields may identify registers, contain literal values or reference other DEX items, such as methods and strings [46]. Hybroid retains the opcode and omits these operand fields. This abstraction limits the vocabulary to the 216 opcode types used in the study, at the cost of information about the values and objects on which each operation acts.

The opcode vectors are learned with the skip-gram formulation of word2vec [149]. The training corpus contains 18,240,542 opcode occurrences extracted from AndroZoo applications [5]. For a sequence $o_1, \dots, o_L$ over a vocabulary $\mathcal{O}$, let $\mathcal{C}_t$ contain the positions within a context window of o_t , excluding t itself. The canonical objective is

$$\begin{aligned} \max_{\theta} \quad & \frac{1}{L} \sum_{t=1}^L \sum_{j \in \mathcal{C}_t} \log p_{\theta}(o_j \mid o_t), \\ p_{\theta}(o' \mid o) = & \frac{\exp(u_{o'}^{\top} v_o)}{\sum_{r \in \mathcal{O}} \exp(u_r^{\top} v_o)}. \end{aligned} \tag{3.1}$$

Here $v_o, u_o \in \mathbb{R}^l$ are the input and output embeddings, and θ contains their parameters. The objective rewards prediction of local opcode contexts and it defines the learning aim.

Figure 3.2 shows an uneven distribution of opcode frequencies in the pre-training corpus. This pattern does not establish a power-law fit or justify the choice of skip-gram, which learns from local opcode co-occurrence. Once trained, the embedding table can assign vectors to the same opcode types in other applications. Vocabulary coverage alone, however, does not establish classification performance on new applications or independence between the pre-training corpus and evaluation data.

Removing operands assigns the same token vector to calls with the same opcode, even when their callees differ [150]. The analyser can still use method references in those operands to construct call-graph edges. The token vectors thus represent opcode context, while the graph records recovered call relations. Both derive from the same bytecode, so their features should not be treated as independent sources of evidence.

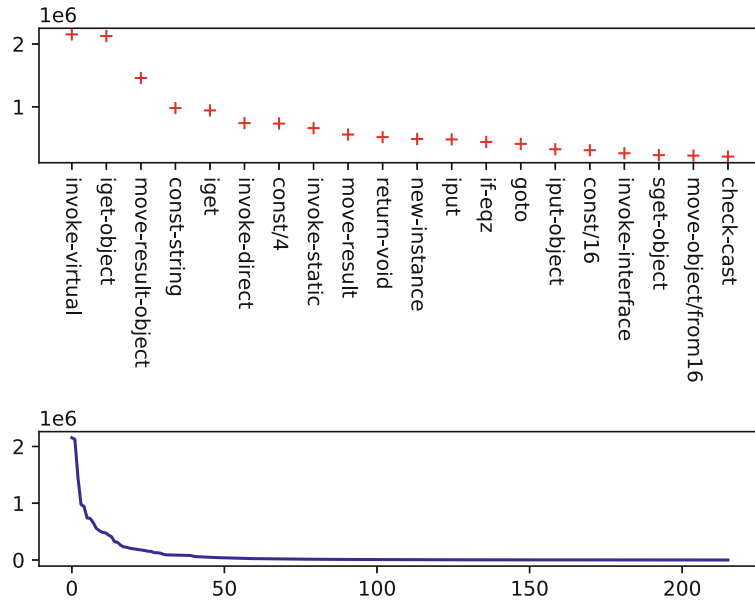


Figure 3.2. Opcode frequency across the pre-training corpus. The upper panel names the twenty most common operations, and the lower panel plots all 216 in rank order. The figure describes empirical frequencies, without a statistical test of a power-law fit. Reproduced from [5].

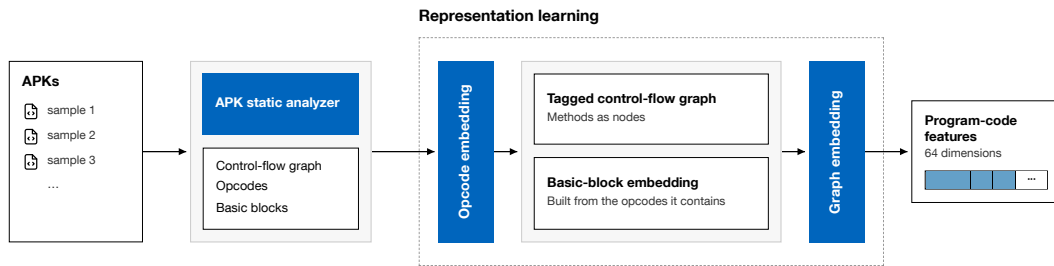


Figure 3.3. Published design of the program-code branch, redrawn by the candidate from [5].

Basic-block features provide the input to the method-level CFG encoder. In the published design, the opcode vectors for instructions in a block are combined by a weighted mean to produce a block vector in $\mathbb{R}^l$ [5].

3.6 Program-Graph Representation

The static branch uses Androguard to extract bytecode and program graphs [52, 5]. A method’s control-flow graph describes relations between its basic blocks. An application-level function-call graph instead relates methods through their call sites. The published design places opcode-derived features within this hierarchy and produces a 64-dimensional application vector (Figure 3.3). The equations below define its documented method-level encoder; the application-level composition is discussed separately.

Static graph recovery depends on the analyser and its treatment of Android execution (Section 2.2.1). It can include infeasible calls and omit calls whose targets are unresolved, for

example through reflection or runtime-loaded code. The representation therefore describes recovered structure, with coverage bounded by the analysis.

Hybroid adapts structure2vec to propagate information between neighbouring basic blocks [65, 5]. Let $G_f = (V_f, E_f)$ be the CFG of method f , with block features $x_v \in \mathbb{R}^l$. Each vertex has a hidden state $\mu_v^{(t)} \in \mathbb{R}^p$, and $N_f(v)$ denotes its neighbour set for propagation. The Hybroid algorithm initialises these states at random and updates them for T rounds:

$$\mu_v^{(t+1)} = F\left(x_v, \sum_{u \in N_f(v)} \mu_u^{(t)}\right), \quad v \in V_f, \quad (3.2)$$

The update function is

$$F\left(x_v, \sum_{u \in N_f(v)} \mu_u^{(t)}\right) = \tanh\left(W_1 x_v + \sigma\left(\sum_{u \in N_f(v)} \mu_u^{(t)}\right)\right), \quad (3.3)$$

where $W_1 \in \mathbb{R}^{p \times l}$ is trainable and $\sigma : \mathbb{R}^p \rightarrow \mathbb{R}^p$ is a fully connected network. For k compatible trainable weight matrices, its stated form is

$$\sigma(h) = P_1 \text{ReLU}(P_2 \cdots \text{ReLU}(P_k h)). \quad (3.4)$$

After propagation, the readout in the publication's Algorithm 1 averages the block states and applies a linear transformation [5]:

$$h_f = W_2 \left(\frac{1}{|V_f|} \sum_{v \in V_f} \mu_v^{(T)} \right), \quad W_2 \in \mathbb{R}^{d_m \times p}, \quad |V_f| > 0. \quad (3.5)$$

The resulting $h_f \in \mathbb{R}^{d_m}$ represents one method. Its hidden states depend on information propagated through at most T edges of the chosen neighbour relation. For $T \geq 1$, $\tanh$ bounds each state coordinate between -1 and 1 , the coordinatewise mean remains in that interval regardless of the number of basic blocks. This controls the direct size dependence of the pooled values without guaranteeing that different graphs remain distinguishable.

The published design subsequently forms a 64-dimensional application representation from the method information and call structure. Its composition operator is not specified, so Equation 3.5 should not be read as an application-level pooling rule. The symbol d_m denotes the unreported method-vector dimension. The encoder can be applied to new graphs with compatible inputs; that architectural property alone does not establish how training and test applications were separated during representation learning.

3.7 Network-Flow Representation

Argus produces bidirectional flow records from each capture [151]. Records are identified through source and destination addresses and ports, together with the transport protocol, and contain roughly forty statistics. Transaction direction and state flags are categorical and are one-hot encoded; the remaining fields are numeric [5].

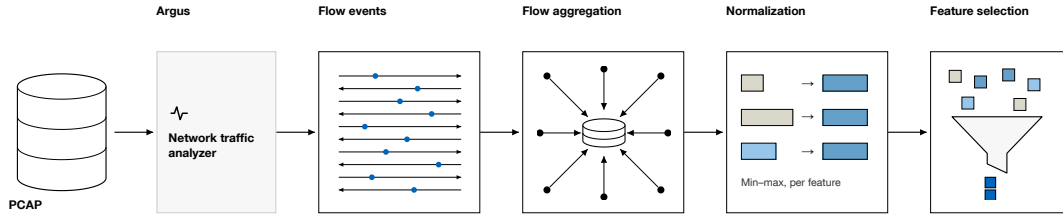


Figure 3.4. The network branch, redrawn by the candidate from [5].

Table 3.2. The 13 retained network-flow fields, grouped by the quantities they describe [5]. Their encoded values are averaged at application level as in Equation 3.6.

Kind	Argus fields before application-level averaging
Volume and duration	mean record duration, total bytes, source bytes
Transport state	destination TCP window, source loss and retransmission flag, destination out-of-order flag, transaction direction
Path and service	source and destination TTL, source and destination TOS byte
TCP sequence state	source and destination TCP base sequence number

Let $r_{ij} \in \mathbb{R}^q$ denote the encoded feature vector for flow record j in the capture associated with application i , where q is the dimension after categorical encoding. For m_i records, the application-level mean is

$$\bar{r}_i = \frac{1}{m_i} \sum_{j=1}^{m_i} r_{ij}, \quad m_i > 0. \quad (3.6)$$

This expresses the aggregation described in the publication [5]. The label applies to the application and its capture; individual flows are not assigned the application’s label as separate training samples. For a one-hot indicator, the mean gives the fraction of records with that category. For a byte-count field, it gives mean bytes per record, not the total volume of the capture.

Reordering the extracted records leaves Equation 3.6 unchanged because addition is commutative. Record order and deviations from the mean are not encoded separately, although means of temporal fields such as duration are retained. Traffic associated with malicious activity may also be averaged together with background traffic in the same capture.

The averaged features are then min-max scaled [5]. For feature k , let $\mathcal{I}_{\text{fit}}$ denote the applications used to fit the scaling bounds:

$$\begin{aligned} \tilde{r}_{ik} &= \frac{\bar{r}_{ik} - a_k}{b_k - a_k}, & b_k &> a_k, \\ a_k &= \min_{i \in \mathcal{I}_{\text{fit}}} \bar{r}_{ik}, & b_k &= \max_{i \in \mathcal{I}_{\text{fit}}} \bar{r}_{ik}. \end{aligned} \quad (3.7)$$

The fitted range maps to $[0, 1]$; values outside that range can map outside the interval.

Feature selection retained the 13 fields in Table 3.2. The publication reports Pearson correlation, an extremely randomised trees estimator and univariate scoring as selection methods [5]. Kendall correlations were then examined for strong pairwise associations (Figure 3.5).

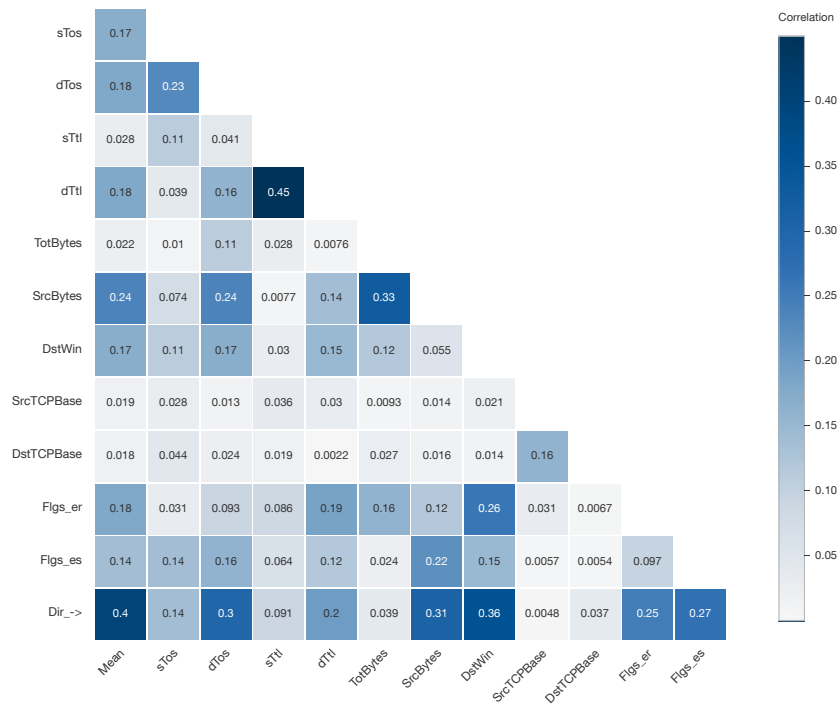


Figure 3.5. Kendall correlations among the retained flow features. Reproduced from [5].

Table 3.3. Reported HTTP and TLS flow shares by malware category [5]. The remainder is calculated as 100% minus those shares; its protocol composition is unspecified.

Category	HTTP	TLS	Unspecified
Adware	52.00 %	8.00 %	40.00 %
Ransomware	29.22 %	0.00 %	70.78 %
Scareware	61.38 %	10.89 %	27.73 %
SMS malware	52.20 %	10.28 %	37.52 %

This checks one form of redundancy; low pairwise correlation does not establish statistical independence or exclude nonlinear dependence.

Table 3.3 reports the observed protocol shares. HTTP accounts for 29.22%–61.38% of flows across the malware categories, and TLS for 0.00%–10.89%.

The network branch uses flow statistics to avoid processing payload contents in the classifier [5]. Application-layer encryption can occur within HTTP or other non-TLS traffic [152], so the TLS share is not a measure of all encrypted content. The evaluation does not separate classification performance by encryption status.

Table 3.4. Published binary-detection comparison with four baselines. Performance values are read from Figures 6 and 7 of [5]; their metric-averaging conventions are not fully specified.

System	Accuracy	F_1	Recall	Precision	AUC
SVM [153]	0.939	0.860	0.856	0.865	0.876
DREBIN [2]	0.954	0.867	0.873	0.861	0.961
Adagio [64]	0.893	0.932	0.913	0.953	0.867
CIC 2017 [144]	0.876	0.872	0.878	0.871	0.914
Hybroid	0.970	0.969	0.970	0.970	0.996

3.8 Multimodal Fusion and Classification

Representation learning and final classification have distinct roles. Within the static branch, a two-layer fully connected network is trained for binary detection, with labels +1 for malicious applications and −1 for benign applications, and a decision threshold of zero [5]. This supervised stage fits the code representation. Its reported output for subsequent classification is a 64-dimensional application vector.

For application i , denote the static vector by $s_i \in \mathbb{R}^{64}$ and the selected flow features by $d_i \in \mathbb{R}^{13}$. Both feature groups form the joint input to the final classifier. This is feature-level fusion: the learner receives both observations, rather than predictions made by separate modality-specific classifiers. The 77 input features need not contribute equally, and their counts do not establish which modality influences the decision. The final learner is trained separately for binary detection and five-class categorisation.

The reported comparisons cover a decision tree, random forest and gradient boosting [5]. Support vector machines and naive Bayes were excluded after preliminary tests. A decision tree partitions the feature space through axis-aligned splits; a random forest combines predictions from multiple trees. Gradient boosting constructs an additive model through sequential tree fits. These are different learning procedures, not a controlled ordering of model capacity. Each is evaluated with static features, flow features and their joint input in Sections 3.9 to 3.11.

3.9 Binary Detection Results

Table 3.4 brings together the binary-detection scores for Hybroid and four external baselines reported in [5].

Hybroid has the highest reported value in each column, with accuracy of 0.970 and AUC of 0.996. The possible post-exclusion class counts imply a calculated accuracy of 79.5%–81.8% for an always-benign rule, which provides a majority-class reference. The baseline table compares complete detection pipelines; its interpretation also depends on preprocessing and tuning, as well as on common metric definitions.

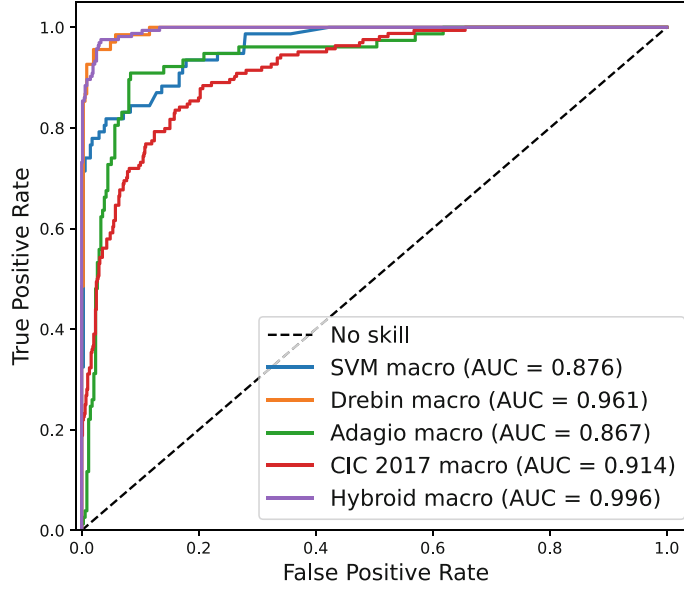


Figure 3.6. Receiver operating characteristic curves for binary malware detection, reproduced from the published vector content of Figure 7 in [5]. The source labels the curves as macro averages and reports their AUC values in the legend.

The missing averaging convention can be illustrated through a confusion-matrix identity. For one binary test set of N applications, let π be the malware proportion, with precision P and recall R defined for malware as the positive class. The false-negative and false-positive counts satisfy $\text{FN}/N = \pi(1 - R)$ and $\text{FP}/N = \pi R(1 - P)/P$, so accuracy satisfies

$$\text{Acc} = 1 - \pi \left[(1 - R) + \frac{R(1 - P)}{P} \right], \quad P > 0. \quad (3.8)$$

Treating Hybroid’s reported accuracy, precision and recall of 0.970 as values from one such matrix would imply $\pi = 0.50$. The evaluated malware proportion is instead between 18.2% and 20.5%, depending on the labels of the excluded applications. This illustrates why the reported row cannot be interpreted as a pooled malware-class confusion matrix. Class-averaged metrics have different meanings, and separately averaged fold metrics need not satisfy the nonlinear identity in Equation 3.8.

Figure 3.6 shows the corresponding ROC curves. DREBIN has the second-highest reported AUC at 0.961. The dynamic CIC 2017 system reaches 0.914, above the SVM and Adagio entries despite its lower accuracy. Baseline ordering therefore depends on the metric. AUC summarises ranking across thresholds; the source does not specify an operational decision threshold [5].

The external comparison includes three static systems and one dynamic system. Their different representations and implementations prevent attributing the score differences to observation modality alone. The within-system comparison below holds the application cohort and learner type fixed while changing the feature groups.

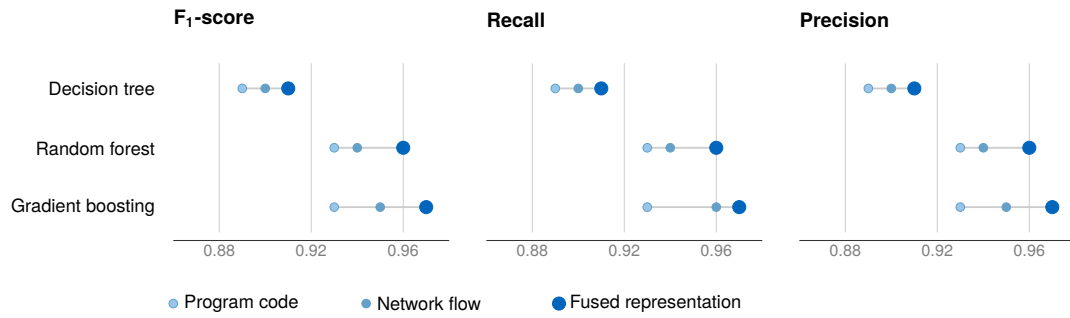


Figure 3.7. Binary detection by learner and observation modality, with a panel for each metric. Values are the five-fold means reported to two decimal places in Figure 8 of [5]. The score axis is truncated to 0.86–0.98. Fold-level scores are unavailable, so no error bars are shown. Redrawn by the candidate.

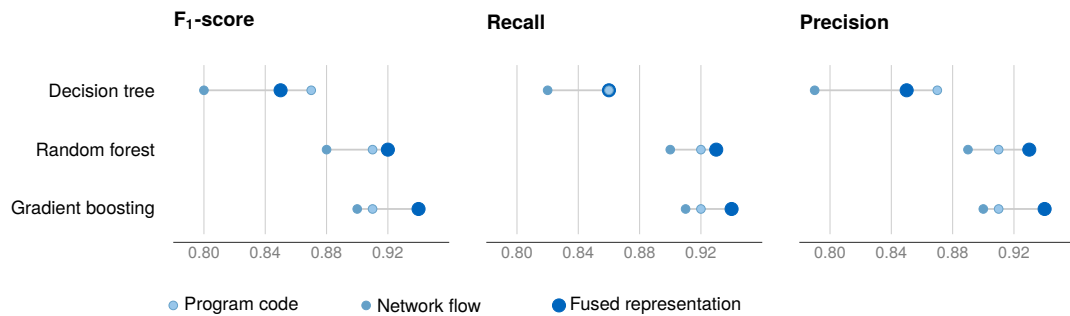


Figure 3.8. Five-class classification by learner and observation modality. Values are the five-fold means reported to two decimal places in Figure 9 of [5]. The score axis is truncated to 0.78–0.96. Decision-tree recall is 0.86 for code and fused inputs, so their markers coincide. Fold-level scores are unavailable, so no error bars are shown. Redrawn by the candidate.

Gradient boosting with the fused representation reaches 0.97 for F_1 , recall and precision (Figure 3.7). Random forest reaches 0.96 and the decision tree 0.91 on those metrics. Flow features alone have higher point values than code features alone for each learner. With code features, random forest and gradient boosting both report 0.93 for all three metrics, so their results tie at the published two-decimal precision [5].

For the supplementary static-only corpus described in Section 3.4, the paper reports 95.0% accuracy and 96.0% F_1 [5]. These values come from the separate 80/20 evaluation on 135,905 applications.

3.10 Malware-Category Classification

The second task assigns each application one of five mutually exclusive labels: Benign or one of the four malware categories in Table 3.1. It is a single-label multiclass task, evaluated separately from binary detection [5].

Figure 3.8 gives the category-classification results. Gradient boosting on the fused representation reaches 0.94 on all three metrics. Random forest follows at 0.92 for F_1 and 0.93 for recall and precision. The decision-tree result differs: the fused representation reaches 0.85 for

Table 3.5. Area under the ROC curve per class, gradient boosting on the combined representation, with the pre-exclusion counts of Table 3.1. Values are read from Figure 10 of [5].

Class	Source samples	AUC
Benign	1,700	0.995
Ransomware	101	0.991
Scareware	112	0.979
Adware	104	0.963
SMS malware	109	0.946
Macro average		0.976

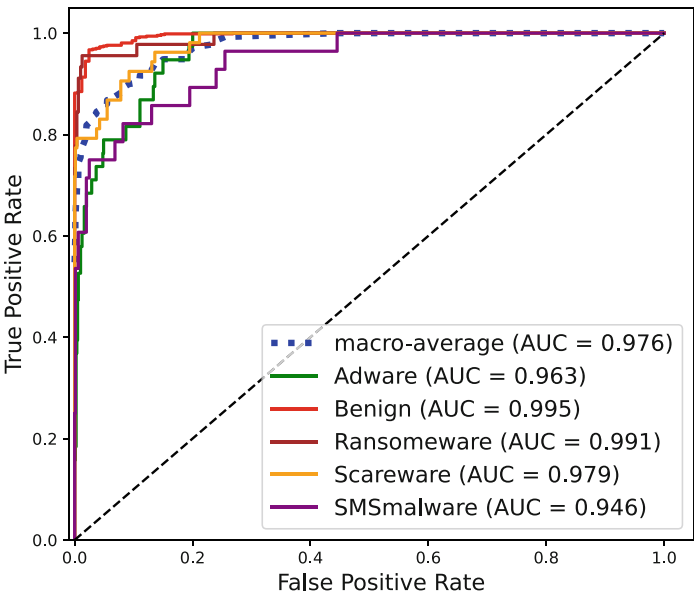


Figure 3.9. Per-class receiver operating characteristic curves for gradient boosting on the fused representation, reproduced from the published vector content of Figure 10 in [5]. The legend reports the AUC of each class and a macro-average AUC of 0.976.

F_1 and precision, below the program-code values of 0.87, while recall is equal at 0.86. The effect of fusion therefore depends on the learner [5].

Table 3.5 gives the per-class ROC AUC for gradient boosting on the joint representation. The source figure reports a macro-average AUC of 0.976 [5]. This is a ranking measure, distinct from five-class accuracy.

Figure 3.9 preserves the published curves. Benign and ransomware have the largest reported AUCs, 0.995 and 0.991; SMS malware has the smallest at 0.946. The five displayed areas have an arithmetic mean of 0.9748, which does not reproduce the reported macro value through rounding alone. The construction of the macro curve is not specified in enough detail to resolve that difference, so both the class values and the reported macro AUC are retained.

Table 3.6. Difference between the fused score and the better of the two single-modality scores. Positive values favour fusion. The differences are computed from Figures 8 and 9 of [5], whose values are printed to two decimal places.

Task	Learner	ΔF_1	Δ recall	Δ precision
Detection	Decision tree	+0.01	+0.01	+0.01
Detection	Random forest	+0.02	+0.02	+0.02
Detection	Gradient boosting	+0.02	+0.01	+0.02
Categorisation	Decision tree	-0.02	0.00	-0.02
Categorisation	Random forest	+0.01	+0.01	+0.02
Categorisation	Gradient boosting	+0.03	+0.02	+0.03

Per-class precision and recall at the five-class decision rule are not reported [5]. Benign applications remain the majority after the exclusions; the aggregate score of 0.94 therefore does not establish equal performance across the classes.

3.11 Modality Comparison

The second part of RQ1 is evaluated through the within-system contrasts in Figures 3.7 and 3.8 [5]. For a fixed learner and task, let $\overline{M}_s$, $\overline{M}_d$ and $\overline{M}_f$ be the reported mean scores for static, dynamic and fused inputs. The contrast against the better single modality is

$$\Delta_M = \overline{M}_f - \max(\overline{M}_s, \overline{M}_d) . \quad (3.9)$$

Positive values favour fusion. Table 3.6 applies this definition to the published two-decimal means, separately for each metric.

The single-modality ordering changes with the task. Network flow has the higher reported F_1 for detection under all three learners: 0.90 against 0.89, 0.94 against 0.93, and 0.95 against 0.93. Program code has the higher value for categorisation: 0.87 against 0.80, 0.91 against 0.88, and 0.91 against 0.90. This reversal is consistent across the three learners reported in the paper [5].

Fusion adds one or two percentage points of F_1 in binary detection. For categorisation, the contrast is minus two points for the decision tree, plus one for random forest and plus three for gradient boosting. For example, gradient boosting improves from the better single-modality score of 0.91 to 0.94 on that task. These comparisons support a modest reported fusion benefit for detection and a learner-dependent result for category classification [5].

3.12 Evaluation Scope and Limitations

Hybroid provides a comparison of code, traffic and fused representations on 2,079 applications, with processing coverage of 97.8%. Using the same retained applications for both branches supports the comparison of modalities. These results concern successful extraction; the 47

applications excluded because of graph-recovery failures motivate further work on obfuscated code and evaluation of a traffic-only fallback [5].

The evaluation uses historical samples under the recorded execution conditions, with traffic coverage bounded by the paths exercised during capture. The main evaluation and the supplementary static-only experiment use random partitions. Testing newer applications in chronological order would assess transfer across changes in malware and network protocols [9]. Ablations of TTL, TOS and TCP sequence fields could assess sensitivity to capture conditions. Resilience to deliberate evasion remains a separate evaluation objective: functionality-preserving APK transformations provide a way to test both branches and the fused classifier under a stated attacker model [122, 154].

The reported means support descriptive comparisons, with metric interpretation discussed in Section 3.9. A fuller replication record would specify the block weights and application-level composition, together with the data used to fit preprocessing and select models. The available description leaves these details and the use of identical folds across configurations unresolved. Explicit metric-averaging conventions and fold-level predictions would support more precise comparisons and estimates of uncertainty.

3.13 Chapter Summary

Hybroid answers the first part of RQ1 with an application-level joint representation. A learned 64-dimensional vector of static program structure is concatenated with 13 features averaged over the recorded network flows of the same application. The joint input supports separate binary and five-class prediction tasks. On CICAndMal2017, gradient boosting with this input achieves a reported detection accuracy of 97.0% and category classification precision, recall and F_1 of 0.94 [5].

The within-system comparisons answer the second part for the evaluated cohort. Fusion improves reported detection F_1 by one or two percentage points over the better single modality. Its category classification contrast ranges from minus two to plus three points across learners. Flow features lead code features in detection, while code features lead in categorisation for each reported learner. These are descriptive point differences under the published protocol [5].

Chapter 4 examines contextual representation within the application package alone. Its hypergraphs encode relations among groups of methods, with no runtime traffic capture required by the analysis pipeline.

Higher-Order Android Malware Analysis with HGANN-Mal

4.1 Research Problem and Relation to Hybrid

This chapter answers RQ2, stated in Section 1.3: How can higher-order relations within static Android program structure be represented and learned, and to what extent do they improve malware detection and classification over pairwise graph representations? It addresses gap G2. Studies have quantified the information lost when group relations are projected onto pairwise edges [71, 70], while other work establishes conditions under which hypergraph propagation is equivalent to propagation on a graph [22, 69]. Zhang et al. evaluated hypergraph representations for Android malware classification [23]. The additional value of learned weighting requires separate evaluation; results on pairwise graph benchmarks show that its benefit depends on the model and task [80].

HGANN-Mal represents an application as a hypergraph over the methods of its function-call graph. Topological hyperedges record call-graph proximity, and semantic hyperedges group methods with similar learned representations. A third family covers methods in backward slices of security-sensitive API calls, with weights based on the access and transmission calls in each slice. An attention operator learns membership weights over the merged incidence structure. The resulting method states are pooled into an application vector, which feeds a classifier with separate binary and class-label heads [6].

The evaluation compares this construction and operator with two pairwise graph networks and two non-attentive hypergraph operators, on two corpora and three tasks under one protocol. Relative to the non-attentive operators, construction and aggregation change together, so their individual contributions remain unresolved; Section 4.13 specifies the ablations needed to separate them.

The chapter retains the source publication’s eleven pipeline stages. Stages 0 to 2 catalogue and unpack the packages and recover the call graph and slices; Stage 3 computes node features. Stages 4 to 6 construct and assemble the hypergraph, followed by mini-batching in Stage 7 and attention in Stage 8. Stages 9 and 10 perform read-out and classification. Sections 4.4 to 4.8 answer the first part of RQ2 through the method, and Sections 4.9 to 4.13 answer the second through the evaluation and analysis.

Relation to Hybroid

Hybroid combines static program information with observed network traffic. HGANN-Mal studies context within the application package through groups of related methods. The former requires an additional runtime observation; the latter constructs its representation from static evidence. Section 7.2 compares these observation requirements.

This is a conceptual comparison between independent studies. Their corpora, class definitions and protocols differ, so the dissertation does not rank their numerical results against one another. Both are the candidate's work; Hybroid is the earlier publication.

The work was published at ISC 2025 [6], with the candidate as first author and his supervisor as second author. The candidate designed and implemented the method, conducted the experiments and wrote the manuscript. Section 1.5 records the individual contribution.

4.2 Related Work

Graph representations of Android applications

Detection over program graphs predates graph neural networks. Adagio embedded call graphs with a neighbourhood hash kernel and classified the result [64], and MaMaDroid abstracted call sequences to package or family level before modelling transitions between them [3]. Their learned successors keep the graph and train the representation over it. The survey that organises this line covers control-flow graphs, call graphs, dependency graphs, heterogeneous application-and-API graphs, system-call graphs and network-flow graphs, and no hypergraph operator appears among them [1].

MsDroid analyses local snippets around security-sensitive APIs. It classifies one subgraph per snippet and combines the decisions at application level. Training uses application labels, and the explanation mechanism identifies important control flows [155]. Both MsDroid and the slice-aware construction in Section 4.7 localise code around sensitive calls. MsDroid extracts separate subgraphs; HGANN-Mal retains one method set and represents each slice-derived group as a hyperedge.

What every member of the line shares is the restriction Section 2.2 describes. An edge relates two methods. A behaviour realised by several methods acting together is representable only as a set of pairs, and the grouping is not recoverable from that set.

Hypergraph learning

HGNN defines a spectral convolution over the incidence structure [81]. HGNN+ generalises propagation through a random-walk transition matrix [82]; the source publication calls this operator HGNNP. Bai et al. introduced learned, real-valued membership weights through

hypergraph attention [24]. Subsequent work includes HGNNv2, which addresses stability through a hypergraph dynamic system with position-aware anisotropic diffusion [88].

Hypergraph-based Android malware analysis

The closest prior Android hypergraph system identified in this review is that of Zhang et al. [23]. Their method builds a function-call graph at method granularity, describes each vertex by six centrality measures and opcode and API-package indicators, and forms hyperedges by two rules: the K -order call neighbours of a function, and the set of functions sharing a permission. It evaluates HGNN, HGNN+, HyperGCN and UniGNN against GCN, GIN and GraphSAGE, on Malnet-Tiny and on Drebin, and it reports its best Drebin result as 97.1% accuracy with HGNN+. The evaluated hypergraph models use fixed hyperedge weights.

Compared with this construction, HGANN-Mal introduces semantic groups and slice-aware hyperedges around security-sensitive call sites. Slice-aware weights depend on the access and transmission calls in each slice, while learned membership scores control aggregation. The experimental baselines are general graph and hypergraph operators implemented within the static-analysis pipeline; the study does not reproduce a complete competing Android detector.

Comparability of published results

Zhang et al. report 97.1% accuracy for HGNN+ on the twenty-family Drebin task over 4,664 samples under an 80/20 split [23], while Section 4.11 reports 95.0% for HGANN-Mal and 93.3% for the same operator on the same task over the same samples under a 70%/20%/10% split. Setting those numbers against each other would compare implementations, feature sets, splits and training regimes at once.

The reported accuracy of HGNN+ differs by 3.8 percentage points between the papers, more than three of the four margins over the baselines in this chapter. Such cross-study differences also motivate common-protocol evaluations of Android detectors [156, 10, 157]. The comparisons below use this study's own protocol.

Evaluation methodology

Methodological work on machine learning in security names the pitfalls that this chapter's evaluation has to be checked against [10], and studies of detector behaviour under obfuscation, drift and adversarial perturbation give the conditions under which published accuracy figures fail to hold [61, 158, 156]. Attacks aimed specifically at graph-based Android detection show what an adversary can do to the representation itself [159, 122, 154]. Section 4.14 states what each of them implies for the results reported here.

4.3 Observation and Threat Model

HGANN-Mal analyses Android applications from their installation packages. The observation unit is one APK: its manifest describes the requested permissions, while its Dalvik bytecode provides the methods and call sites used to construct the program representation. From this evidence, the analysis recovers a function-call graph and identifies code associated with security-sensitive API calls. Hyperedges then group related methods so that the classifier can use context across several methods in one application-level prediction.

This package-based approach supports analysis before an application is executed. An analyst can process a collection of APKs without preparing a device or emulator, scripting user interactions, or recording network traffic. The evidence therefore comes from the code available to the static analyser, independently of which execution paths a test session would trigger. This provides a different observation setting from Hybroid, which combines static evidence with captured network traffic. Section 7.2 discusses these acquisition requirements, and Section 4.13 examines the computational cost of the hypergraph pipeline.

The security objective is to distinguish malicious applications from benign ones and to assign the dataset-specific class labels. On Drebin, the multiclass task identifies malware families; on CICMalDroid, it distinguishes application categories, including Benign. The evaluation uses historical application samples under a *non-adaptive threat model*: the samples are analysed as collected, without modifications designed to evade HGANN-Mal. Within this setting, the experiments assess how higher-order program representations support detection and classification under a common protocol.

The scope of each prediction is the code and call relations recovered from the package. Additional code obtained only at runtime and unresolved reflective calls fall outside that representation; packed or native payloads may also remain unavailable to the DEX analyser. Section 4.14 discusses these coverage limits and the separate question of resistance to deliberate evasion. Chapter 6 develops the evaluation requirements for such adversarial settings.

4.4 APK Processing and Function-Call Graph Extraction

Stage 0 builds a catalogue. Every application package under a root directory is hashed with SHA-256, and the hash is joined against the label list distributed with the corpus, so each sample carries a benign label or a family label before any analysis runs. Working from the hash rather than from the file name matters for a corpus assembled from several sources, since it makes duplicate files visible; Section 4.9 returns to what was and was not done with that information.

In Stage 1 each package is decompressed with `apktool` and the Python `zipfile` module, which yields the manifest and every `classes*.dex` file the package contains. Applications built with

multi-dex are therefore covered, and the bytecode of all of them enters the next stage. The reported runs used apktool, MobSF, Androguard with its DAD decompiler and a ready-made code2vec model.

Androguard [52], which the source runs beside MobSF in Stage 2a, disassembles the DEX files and produces the object the rest of the chapter is built on: a function-call graph $G = (V, E)$ at method granularity, in which a vertex is a method identified by its class, its name and its descriptor, and a directed edge records that one method may invoke another. The same pass emits a flat list of pairs that records, for each method, which framework APIs it calls. That list is what allows a flagged API to be located at its call sites rather than merely counted, and Section 4.5 uses it for exactly that.

The recovery limits in Section 2.2.1 apply to this object. An edge in G records a possible invocation identified by the analyser. Infeasible calls may be included, while unresolved targets or incompletely modelled callbacks may be omitted. The toolkit and execution assumptions therefore affect the vertex and edge sets recovered from an application. These choices affect every construction in Section 4.7, because each hyperedge rule uses G or an artefact derived from it.

4.5 Security-Sensitive API Identification and Program Slicing

Stage 2 decides which parts of an application are worth isolating. In Stage 2a two tools run over each package. MobSF parses the manifest, disassembles the bytecode and reports the permissions an application requests together with the framework APIs it uses. Of the permissions reported, only those whose status is *dangerous* in the platform's own classification are retained, which narrows attention to the capabilities that guard user data and device functions [49].

The APIs reached from those capabilities are then sorted into two classes by a hand-written rule. An *access* API reads something the user would consider private, and the examples the source publication names are contacts, location, short messages and storage. A *transmission* API moves data off the device, with HTTP clients, raw sockets and `sendTextMessage` as its examples. The pairing is deliberate. Data exfiltration is a sequence in which something is read and then sent, and a rule that marks both ends of that sequence gives the construction in Section 4.7 something to join into one group. The source publication describes the classification as a heuristic.

Stage 2b then computes a backward slice for every call site of a marked API. The variables that reach the call site are identified first, and the control-flow graph of the enclosing code is walked backwards, keeping only the blocks and statements that influence those variables. What the stage stores is a compact control-flow graph per slice. The purpose is localisation rather than verification: the slice does not establish that data flows from a source to a sink, in the sense that a taint analysis would [47], and it is not offered as evidence of exfiltration.

Table 4.1. The four groups of node feature. The width of the first group is the size of the opcode vocabulary.

Group	Width	What it records
Opcode histogram	vocabulary	the distribution of Dalvik instructions in the method body
Structural metrics	4	in-degree, out-degree, PageRank and betweenness centrality of the method in G
API indicators	4	whether the method reaches an access API and whether it reaches a transmission API, with the counts of each within the slices attached to it
Semantic embedding	320	a code2vec representation of the de-compiled method body

It marks a neighbourhood of code that participates in reaching a sensitive call, and that neighbourhood becomes one hyperedge.

4.6 Node Features

In Stage 3 each vertex of the call graph receives a feature vector assembled from four groups, listed in Table 4.1 and drawn in Figure 4.1. The groups are concatenated and the result is standardised to zero mean and unit variance across the corpus, giving the feature matrix X with one row per method. The figure labels the blocks h_{op} , s_{struct} , a_{api} and c_{2v} , and the last of them is the embedding written c_v in Section 4.7. With O the opcode vocabulary, the width is $F = |O| + 328$.

The first group describes what a method does at instruction level, and it is the feature the program-code branch of Chapter 3 also rests on, there as a learned embedding of the opcode sequence rather than as a histogram. Operand values are not used, which removes the constants and register numbers that vary between builds of the same code. The histogram can be taken as raw counts or weighted by inverse document frequency across the corpus.

The second group places the method in the call graph rather than describing its body, and it is the only group that changes when the graph changes. The third connects the vertex to the analysis of Section 4.5, so that a method reaching a sensitive API is distinguishable before any hyperedge is built. Both are small, and together they contribute eight of the dimensions.

The fourth group is the largest. code2vec represents a method as a bag of paths through its abstract syntax tree and learns a fixed-width vector from that bag [67]. The published method consumes Java source, and the released model was trained on a corpus of Java source, whereas the input here is Dalvik bytecode recovered from a distributed package. The candidate confirms that the two were bridged by decompilation: each method was decompiled to Java with DAD, the decompiler built into Androguard, and embedded with a ready-made, pre-trained code2vec model. The source publication records neither step, and the width it reports differs from the released model's.

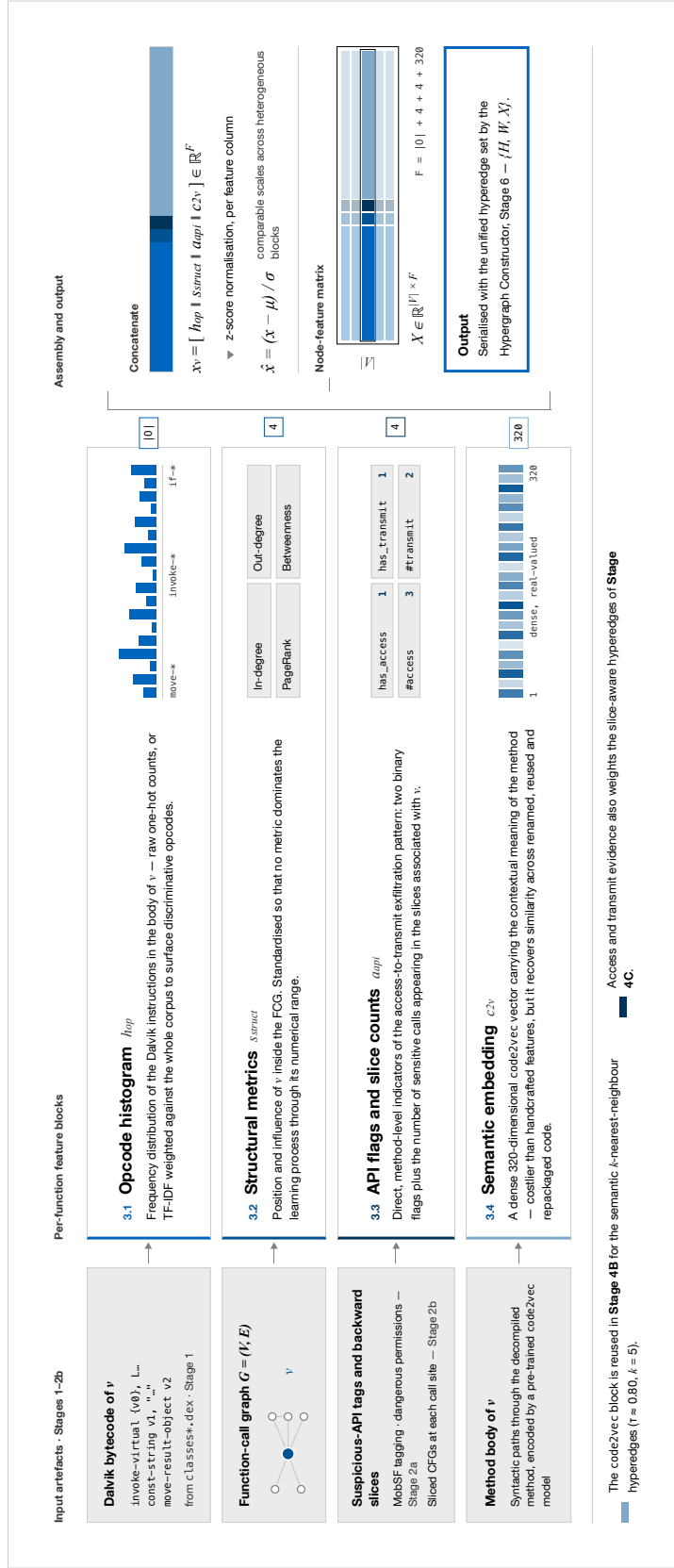


Figure 4.1. Construction of the node feature vector in Stage 3. For each method v of the function-call graph, an opcode histogram, four structural centrality metrics, four suspicious-API flags and counts, and a 320-dimensional code2vec embedding are computed from the artefacts of Stages 1 to 2b. The four blocks are concatenated into x_v and z-score normalised, which yields the node-feature matrix $X \in \mathbb{R}^{|V| \times F}$ consumed by the hypergraph constructor in Stage 6.

The embedding of a method therefore depends on how completely the decompiler recovers its source, as well as on the model. That dependence matters for the construction too. The semantic hyperedges of Section 4.7 are built from cosine similarities between these embeddings, so the decompiler and the model shape one of the three construction rules as well.

4.7 Hyperedge Construction

The lifting from a graph to a hypergraph is where the representation is decided. Stages 4A to 4C apply three rules of different kinds to the same vertex set, and Stages 5 and 6 merge and assemble what they produce. Write $\mathcal{H} = (\mathcal{V}, \mathcal{E}, w)$ for the result, with $\mathcal{V} = V$ the methods of the call graph. Figure 4.2 draws one hyperedge of each kind together with the parameters of its rule. Its panels keep the names H_K , H_{SEM} and H_{API} that the source gives the three lists, which Equation 4.5 writes as the candidate sets $\mathcal{C}_{\text{topo}}$, $\mathcal{C}_{\text{sem}}$ and $\mathcal{C}_{\text{sus}}$.

Topological hyperedges. The source collects, for each method v , its callers and callees up to order K . Let $\vec{d}_G(v, u)$ be the number of call edges on a shortest directed path from v to u , with $\vec{d}_G(v, u) = \infty$ when there is none, and let $\text{dist}_G(v, u) = \min\{\vec{d}_G(v, u), \vec{d}_G(u, v)\}$. Then

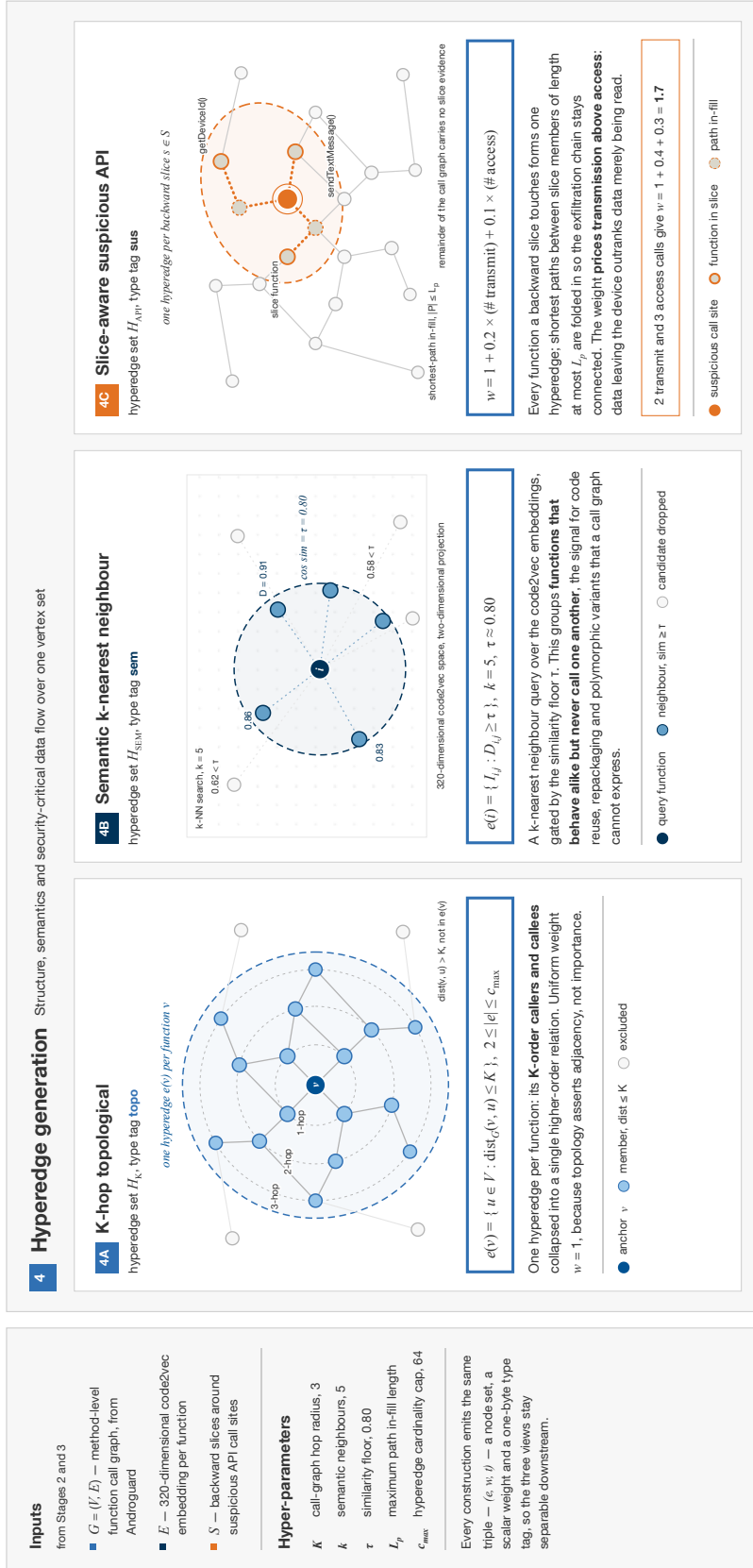
$$e_{\text{topo}}(v) = \{u \in V : \text{dist}_G(v, u) \leq K\}, \quad K = 3, \quad (4.1)$$

holds v together with its callers and callees up to K steps away. This convention follows the wording of the source. A distance measured with call directions ignored would also admit the other callees of a caller and would give larger hyperedges, and the source does not record which convention the implementation used. This is the rule the earlier hypergraph work on Android uses, there in combination with a permission-based rule [23]. It records a neighbourhood as one object rather than as a set of pairs, which is the whole of the higher-order claim at this point: a vertex and its K -hop neighbourhood enter the operator together.

Semantic hyperedges. A nearest-neighbour search over the code2vec embeddings, computed with the FAISS library, groups methods whose representations are close. Write $c_v \in \mathbb{R}^{320}$ for the embedding of method v , the fourth group of Table 4.1; the full feature vector x_v also carries the other three groups, and the search uses the embedding alone. The candidate confirms that the search compares cosine similarities, $\text{sim}(a, b) = a^\top b / (\|a\| \|b\|)$, and that it runs within one application. With $\mathcal{N}_k(v)$ the k methods of the same application, other than v , that score highest under sim ,

$$e_{\text{sem}}(v) = \{v\} \cup \{u \in \mathcal{N}_k(v) : \text{sim}(c_v, c_u) \geq \tau\}, \quad k = 5, \quad \tau \approx 0.80. \quad (4.2)$$

The original algorithm, whose test panel 4B of Figure 4.2 reproduces, names the search output D , a letter that suggests a distance, and keeps a neighbour when $D \geq \tau$. The test agrees with the cosine similarity, since for a distance the inequality would run the other way. The query method is included and is not counted among its k neighbours, so $|e_{\text{sem}}(v)| \leq k + 1$; the source does not say whether its search counted the query itself. Nothing in this rule refers to the call graph, which is the point of including it: two methods that never call one another, and that



sit in different parts of the application, are joined when they do the same kind of work. The source names polymorphic malware and code reuse as the intended cases. Because the search stays inside one application, a semantic hyperedge joins methods of that application only, and code shared across a family can make the hyperedges of different applications alike without joining them.

Slice-aware hyperedges. Each backward slice s from Section 4.5 yields one hyperedge over the methods the slice contains, extended by the methods on short call paths between them. Let F_s be that member set, fixed before any path is added. For a directed path P in the call graph, let $V(P)$ be its methods and $\ell(P)$ its number of call edges, and let P_{uv} be a shortest path from u to v . Then

$$e_{\text{sus}}(s) = F_s \cup \bigcup_{\substack{u, v \in F_s, u \neq v \\ P_{uv} \text{ exists, } \ell(P_{uv}) \leq L_p}} V(P_{uv}). \quad (4.3)$$

Only pairs of original members are joined, so a method that one path adds does not start further paths. The original algorithm loops over the pairs of the set it is enlarging, and freezing F_s is the convention adopted here to make the result independent of the loop order. A pair with no directed path contributes nothing. Where several shortest paths exist, the one chosen decides which methods join. This family carries the only non-unit weight in the hypergraph:

$$w(e_{\text{sus}}(s)) = 1 + 0.2 \# \text{transmit}(s) + 0.1 \# \text{access}(s), \quad (4.4)$$

so a slice that both reads private data and sends it weighs more than one that only reads, and the transmission end weighs twice the access end. The weight is a construction heuristic, so it cannot be read as a probability of exfiltration.

Merging. Each rule proposes candidate triples (e, w, t) of a member set, a weight and a type tag. Collected as a multiset and passed through the post-processing of the source, they give

$$\mathcal{C} = \mathcal{C}_{\text{topo}} \uplus \mathcal{C}_{\text{sem}} \uplus \mathcal{C}_{\text{sus}}, \quad \mathcal{E} = \text{Post}_\eta(\{(e, w, t) \in \mathcal{C} : 2 \leq |e| \leq c_{\text{max}}\}), \quad c_{\text{max}} = 64. \quad (4.5)$$

The lower bound removes singletons, which carry no relation, and with them every slice that has a single member, since such a slice has no pair to join. The upper bound is described in the source publication as necessary for stable message passing. Its prose says that hyperedges are capped, whereas its algorithm rejects a candidate outside the interval. This chapter follows the algorithm: truncation would need a rule for choosing which members to keep, and none is given. Rejection has a consequence of its own. A method whose K -step neighbourhood holds more than 64 methods loses its topological hyperedge entirely, so the topological family omits exactly the methods with the largest neighbourhoods.

The operator Post_η removes duplicates. Two candidates with the same members can carry different weights or tags, as a slice hyperedge and a topological one over the same methods would, and the policy η that decides which weight and which tag survive is not recorded; this chapter does not supply one. Filtering can also leave a method in no hyperedge, for instance an isolated method that belongs to no slice and has no semantic neighbour above τ . Its degree is then zero and the normalisation of Section 4.8 is undefined for it, and the source does not say whether such methods were dropped or handled in another way.

Algorithm 4.1 Construction and assembly of the HGANN-Mal hypergraph. Stages 4A to 4C restore the algorithm of the source publication [6] with the conventions of Equations 4.1 to 4.3 written in; stages 5 and 6 add the post-processing and assembly that its prose describes.

Input: call graph $G = (V, E)$; embeddings c_v ; slices $s \in S$ with member sets F_s ; parameters $K, k, \tau, L_p, c_{\max}$; duplicate policy η

Output: sparse incidence matrix H , weight matrix W , type tags t

Stage 4A: topological hyperedges

```

1:  $\mathcal{C} \leftarrow \emptyset$  ▷ multiset of candidates
2: for  $v \in V$  do
3:    $e \leftarrow \{u \in V : \text{dist}_G(v, u) \leq K\}$ 
4:   if  $2 \leq |e| \leq c_{\max}$  then
5:      $\mathcal{C} \leftarrow \mathcal{C} \uplus \{(e, 1, \text{topo})\}$ 

Stage 4B: semantic ( $k$ -NN) hyperedges
6:  $(I, S) \leftarrow \text{FAISS}(\{c_v\}_{v \in V}, k)$  ▷  $k$  others per method;  $S$  holds cosine similarities
7: for  $v \in V$  do
8:    $e \leftarrow \{v\} \cup \{I_{v,j} : S_{v,j} \geq \tau\}$ 
9:   if  $2 \leq |e| \leq c_{\max}$  then
10:     $\mathcal{C} \leftarrow \mathcal{C} \uplus \{(e, 1, \text{sem})\}$ 

Stage 4C: slice-aware hyperedges
11: for  $s \in S$  do
12:    $e \leftarrow F_s$ ;  $w \leftarrow 1 + 0.2 \# \text{transmit}(s) + 0.1 \# \text{access}(s)$ 
13:   for  $(u, v) \in F_s \times F_s$  with  $u \neq v$  do ▷  $F_s$  is not enlarged
14:     if  $P_{uv}$  exists and  $\ell(P_{uv}) \leq L_p$  then
15:        $e \leftarrow e \cup V(P_{uv})$ 
16:   if  $2 \leq |e| \leq c_{\max}$  then
17:      $\mathcal{C} \leftarrow \mathcal{C} \uplus \{(e, w, \text{sus})\}$ 

Stage 5: post-processing
18:  $\mathcal{E} \leftarrow \text{Post}_\eta(\mathcal{C})$  ▷ policy  $\eta$  not recorded

Stage 6: sparse assembly
19: for  $j = 1, \dots, |\mathcal{E}|$ , with  $(e_j, w_j, t_j)$  the  $j$ -th element of  $\mathcal{E}$  do
20:    $H_{vj} \leftarrow 1$  for every  $v \in e_j$ ;  $W_{jj} \leftarrow w_j$ ;  $t(j) \leftarrow t_j$ 
21: return  $H, W, t$ 

```

Each surviving hyperedge is tagged with its family in one byte, and the incidence matrix H and the diagonal weight matrix W are assembled from the result and serialised per application. Algorithm 4.1 restores the construction procedure of the source publication with the conventions above written into it. It also adds the post-processing and the sparse assembly, which the output line of the original announces and its body omits. The type tag is stored but is not an input to the layer in Section 4.8, which consumes H, W and X . Section 4.13 discusses possible uses of the tags.

4.8 Hypergraph Attention Model

The operator is defined over the structure fixed in Section 2.2.2, whose notation is used here without restatement: the incidence matrix H , the weighted vertex degree $d(v)$ and the hyperedge degree $\delta(e)$ with their diagonal matrices D_v and D_e , and the diagonal weight matrix W . Write $n = |\mathcal{V}|$, and assume for now that $d(v) > 0$ for every method; Section 4.7 records that filtering can violate this. The hypergraph Laplacian is

$$L = I - P_0, \quad P_0 = D_v^{-1/2} H W D_e^{-1} H^\top D_v^{-1/2}, \quad (4.6)$$

and for a signal $f \in \mathbb{R}^n$ over the methods it has the energy

$$f^\top L f = \frac{1}{2} \sum_{e \in \mathcal{E}} \frac{w(e)}{\delta(e)} \sum_{u,v \in e} \left(\frac{f_u}{\sqrt{d(u)}} - \frac{f_v}{\sqrt{d(v)}} \right)^2, \quad (4.7)$$

where the inner sum runs over ordered pairs [21]. The energy is small when the degree-normalised signal varies little inside the constructed groups, and each hyperedge counts in proportion to its weight and in inverse proportion to its size. It describes the operator and is not a term of the training loss. One layer of hypergraph convolution propagates through P_0 :

$$X^{(l+1)} = \sigma \left(D_v^{-1/2} H W D_e^{-1} H^\top D_v^{-1/2} X^{(l)} \Theta^{(l)} \right), \quad (4.8)$$

with $\Theta^{(l)}$ the learned projection of layer l and σ a non-linearity. Read from right to left, the operator gathers vertex states into the hyperedges that contain them, averages within each hyperedge through D_e^{-1} , scales by the hyperedge weight, and returns the result to the member vertices with the symmetric normalisation of $D_v^{-1/2}$. One layer therefore couples every pair of methods that share a hyperedge. The matrix P_0 is positive semidefinite and similar to the row-stochastic matrix $D_v^{-1} H W D_e^{-1} H^\top$, so its eigenvalues lie in $[0, 1]$. Equation 4.8 thus multiplies each eigencomponent of L with eigenvalue λ by $1 - \lambda$, and the components with the highest energy are damped most. This is the smoothing behind the low-pass bias recorded in Section 2.3.3.

Attention over hyperedges

Equation 4.8 weights a vertex's hyperedges by W , which is fixed when the hypergraph is built. Attention replaces the binary membership indicator by a learned real value, so that the strength with which a vertex participates in a hyperedge is trained [24]. The layer index is dropped from here on. Let $X \in \mathbb{R}^{n \times F}$ hold the input states as rows, let $\Theta \in \mathbb{R}^{F \times F'}$ be the projection, and set $Y = X\Theta$, with $y_v \in \mathbb{R}^{F'}$ the row of method v read as a column vector. A hyperedge is first represented by the mean of the projected states of its members,

$$z_e = \frac{1}{\delta(e)} \sum_{u \in e} y_u, \quad (4.9)$$

and the coefficient joining a vertex to one of its hyperedges is the normalised score

$$\alpha(v, e) = \frac{\exp(\sigma(a^\top [y_v \parallel z_e]))}{\sum_{e' \ni v} \exp(\sigma(a^\top [y_v \parallel z_{e'}]))}, \quad (4.10)$$

with $a \in \mathbb{R}^{2F'}$ a learned vector and $\parallel$ concatenation. Collecting the coefficients into H_{att} , whose entry at (v, e) is $\alpha(v, e)$ where H has a one and zero elsewhere, gives the layer actually used:

$$X' = \sigma \left(D_v^{-1/2} H_{\text{att}} W D_e^{-1} H_{\text{att}}^\top D_v^{-1/2} X \Theta \right). \quad (4.11)$$

The degree matrices are those of the binary H , as in the source publication; degrees recomputed from H_{att} would define a different layer. Gradients reach H_{att} as well as X and Θ , so the membership weights are learned. The support is not. $H_{ve} = 0$ implies $\alpha(v, e) = 0$, and a finite score gives every present membership a positive coefficient, so training reweights the

memberships the construction produced and cannot add a method to a hyperedge or remove one.

Equations 4.9 and 4.10 state the additive form of the score, which the source publication leaves as an uninstantiated similarity function and which the candidate confirms is what was implemented. The published formulation also writes the score between two vertices while indexing it by a hyperedge; the form above resolves that by scoring a vertex against a representation of the hyperedge, which is what the implementation computes.

The messages between methods

Expanding Equation 4.11 shows what one method receives from another. The layer runs in two stages. Each hyperedge first forms a message, and each method then combines the messages of its hyperedges:

$$\mu_e = \frac{1}{\delta(e)} \sum_{u \in e} \frac{\alpha(u, e)}{\sqrt{d(u)}} y_u, \quad x'_v = \sigma \left(\frac{1}{\sqrt{d(v)}} \sum_{e \ni v} w(e) \alpha(v, e) \mu_e \right). \quad (4.12)$$

Substituting the first expression into the second gives one matrix of couplings between methods,

$$X' = \sigma(P_{\text{att}} X \Theta), \quad P_{\text{att}}(v, u) = \frac{1}{\sqrt{d(v)d(u)}} \sum_{e \ni v, u} \frac{w(e)}{\delta(e)} \alpha(v, e) \alpha(u, e). \quad (4.13)$$

Two methods interact only through the hyperedges they share, and each shared hyperedge adds one term. In that term the coefficient of the sender and the coefficient of the receiver appear as a product. The construction weight $w(e)$ is a separate factor, so the slice heuristic of Equation 4.4 and the learned coefficients act on a message independently. The coefficients depend on the layer input through Equations 4.9 and 4.10, which makes P_{att} a function of X ; for a fixed input it is a symmetric matrix.

The key z_e and the message μ_e are different aggregations over the same members. The key is an unweighted mean, whereas the message weights each member u by $\alpha(u, e)/\sqrt{d(u)}$, so attention changes the average inside a hyperedge as well as the weighting across the hyperedges of a method. It does not change the sign of any term. Every member enters μ_e with a positive coefficient, so before the non-linearity each output is the projection of a non-negative combination of input states, and one layer cannot form a difference between two members of a hyperedge. The high-frequency component that standard hypergraph convolution discards has been characterised [87]. Attention changes the proportions of that average and keeps every coefficient positive, so the propagation of one layer remains an averaging of member states.

Grouping the sum in Equation 4.13 by hyperedge gives a structural statement:

$$P_{\text{att}} = \sum_{e \in \mathcal{E}} \frac{w(e)}{\delta(e)} \psi_e \psi_e^\top, \quad (\psi_e)_v = \frac{\alpha(v, e)}{\sqrt{d(v)}}, \quad (4.14)$$

with $\alpha(v, e) = 0$ for $v \notin e$. Each hyperedge adds a positive semidefinite matrix of rank one, and because every weight is positive, P_{att} is positive semidefinite for every input. The statement concerns the propagation matrix at a fixed input. It carries no guarantee about the convergence of training.

Because $\alpha(v, e)$ depends on both the method and its hyperedge, two different hyperedge groupings can produce the same fixed matrix P_0 yet different attentive propagation matrices. The reported experiments do not determine whether the learned model used this capacity or whether it caused the measured accuracy differences.

Properties of the score and normalisation

Two further properties bound what the chapter claims for the operator.

The first concerns the ranking that the score induces. Splitting $a = (a_1, a_2)$ across the concatenation writes the argument of σ in Equation 4.10 as $q_v + b_e$, with a query term and a hyperedge key

$$q_v = a_1^\top y_v, \quad b_e = a_2^\top z_e. \quad (4.15)$$

If σ is strictly increasing, then for a fixed layer input and any two hyperedges e_1 and e_2 that contain v ,

$$\alpha(v, e_1) > \alpha(v, e_2) \iff b_{e_1} > b_{e_2}. \quad (4.16)$$

The proof is short. Because σ and $\exp$ are strictly increasing, the numerators of Equation 4.10 are ordered as the keys are, and the two coefficients share one positive denominator. Any two methods therefore rank the hyperedges they share in the same order, and the keys alone fix that order. Methods can still attend to different hyperedges, since each normalises over its own. A non-decreasing activation such as ReLU can merge two hyperedges into a tie but cannot reverse their order between methods. The argument is the one Brody et al. give for the additive score of pairwise graph attention [79], applied to the score of Equation 4.10. It holds for each attention head separately and says nothing about hypergraph attention with other scoring functions. Under either kind of activation, the restriction on wording that the chapter keeps is thus a result for this operator: within one layer, HGANN-Mal can learn which hyperedges matter, and it cannot reverse their order from one method to another.

The query term does not cancel, however. With σ a leaky ReLU of slope 0.2 and keys -1 and 1 , the coefficients are approximately 0.23 and 0.77 for $q_v = 0$, and 0.12 and 0.88 for $q_v = 2$. The order agrees and the values differ. Cancellation is exact when σ is affine on the arguments of one method, as a leaky ReLU is when they share a sign.

The second concerns normalisation. Uniform coefficients do not reproduce Equation 4.8, because the degrees in Equation 4.11 are those of the binary H . Let r_v be the number of hyperedges containing v and $R = \text{diag}(r_v)$. Equal scores give $H_{\text{att}} = R^{-1}H$ and hence

$$P_{\text{att}} = R^{-1}P_0R^{-1}, \quad (4.17)$$

which reduces to P_0/r^2 when every method lies in r hyperedges. An ablation that replaced the learned coefficients by uniform ones would therefore change the scale of propagation as well

as its weighting. A comparison with the non-attentive operator needs a stated normalisation convention before a difference can be read as an effect of attention.

Read-out and classification

Stage 9 pools the final method states into a fixed-size application vector g . The source publication describes mean pooling and learned attention pooling as alternatives. Batches use block-diagonal incidence and weight matrices with stacked node features; an application index keeps propagation and pooling separate for each APK. For a fixed hypergraph, reordering methods or hyperedges leaves the application vector unchanged. This invariance concerns storage order; changes to code or order-dependent construction choices can still alter the hypergraph.

In Stage 10, a shared multi-layer perceptron with two hidden layers, ReLU activations and dropout maps g to two classification heads. The binary head uses a sigmoid to estimate the malware probability, while the class head uses a softmax over the dataset’s labels: twenty malware families for Drebin, or five application categories, including Benign, for CICMalDroid.

The source combines binary cross-entropy (BCE) and multiclass cross-entropy (CE), with weights λ_{bin} and λ_{fam} . To account for samples with only one applicable label, a label-aware formulation averages each term over its eligible training set:

$$\mathcal{L} = \frac{\lambda_{\text{bin}}}{|\mathcal{D}_{\text{bin}}|} \sum_{i \in \mathcal{D}_{\text{bin}}} \text{BCE}(z_{\text{bin},i}, y_{\text{bin},i}) + \frac{\lambda_{\text{fam}}}{|\mathcal{D}_{\text{fam}}|} \sum_{i \in \mathcal{D}_{\text{fam}}} \text{CE}(z_{\text{fam},i}, y_{\text{fam},i}), \quad (4.18)$$

where z denotes logits, y denotes targets, and the non-empty sets $\mathcal{D}_{\text{bin}}$ and $\mathcal{D}_{\text{fam}}$ contain samples with binary and class labels, respectively. The class-labelled set includes benign applications on CICMalDroid; on Drebin, it contains only members of the twenty selected malware families. This formulation specifies how missing class targets can be handled; the source does not document that handling.

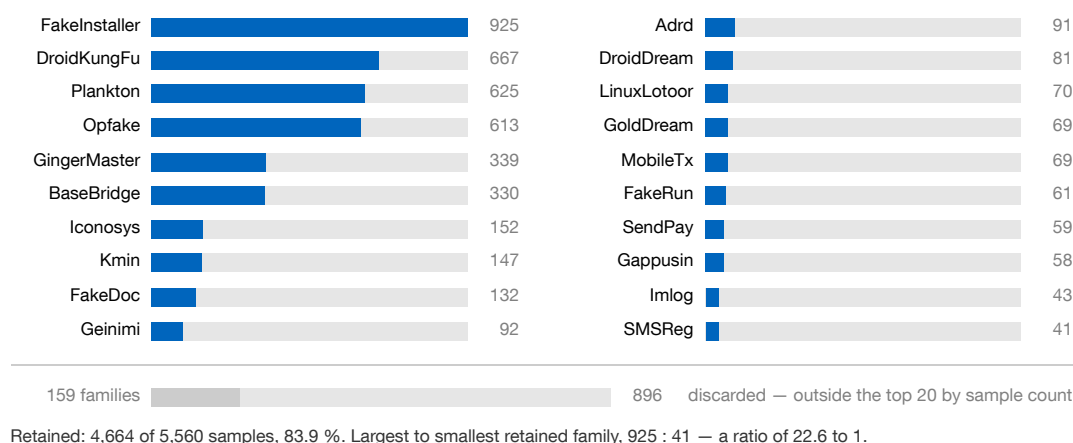
4.9 Datasets, Tasks, and Split Policy

The evaluation uses two corpora with different label taxonomies: Drebin for malware-family classification and CICMalDroid for malware-category classification. Binary detection is evaluated on both, with an additional benign collection for Drebin.

Drebin

Drebin is the malicious part of a larger collection gathered between August 2010 and October 2012 from the Google Play Store, from Chinese and Russian alternative markets, from Android websites and malware forums, and from the whole of the Malgenome project [2]. Every sample in that collection was submitted to VirusTotal and judged by ten antivirus scanners; an application was counted as malicious when at least two of the ten flagged it, applications

(a) Drebin — the twenty retained families



(b) CICMalDroid2020 — five categories

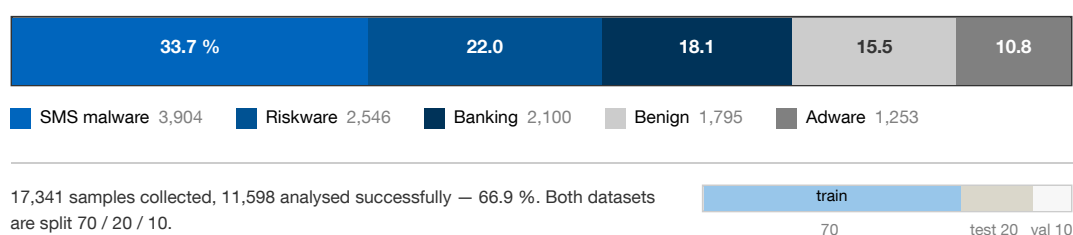


Figure 4.3. Class distribution of the two corpora. (a) The twenty largest Drebin families, the classes of the malware-family classification task. The 159 smaller families, with 896 samples, take no part in that task and remain in binary detection. (b) The five CICMalDroid 2020 categories of the published distribution [160]; Benign is the negative class of binary detection on this corpus. The long tail of Drebin is the reason the source publication reports an F1 score, which it labels Macro-F1, alongside accuracy.

labelled adware were then removed, and 5,560 malicious samples remained. The family names are taken from the labels of one of the ten scanners. The source publication of this chapter counts 179 families in the set [6], a figure the dataset paper does not state. Family sizes are heavily skewed, so the classification task uses the twenty largest, which cover 4,664 of the samples and are drawn with their sizes in Figure 4.3(a). The 159 smaller families hold the other 896 samples, 16.1% of the set, and they are left out of the family task only, since binary detection uses all 5,560 malicious samples.

The 123,453 benign packages used in the original DREBIN evaluation are not part of the distributed collection. For this study, the benign samples were drawn at random from AndroZoo [161]. AndroZoo collects Android applications by crawling Google Play, the Chinese markets Anzhi and AppChina, and several smaller repositories; the crawl began in late 2011 and has continued since, and the collection held more than three million applications when it was first described. Every application in it has been submitted to VirusTotal, and what the collection stores per application is the number of antivirus products that flagged it, together with the compilation date of its bytecode, its size and the market it came from. About 9,470 benign applications were used against the full set of 5,560 malicious ones, giving a corpus of roughly 15,030 in which malware is 37.0%.

Turning that stored count into a benign label requires a threshold, and the threshold is a decision rather than a property of the data. The AndroZoo authors report the flagged share at one antivirus product and again at ten, and state that antivirus results have to be treated with care; the two thresholds give very different populations.

CICMalDroid

CICMalDroid 2020 was collected between December 2017 and December 2018 and holds five categories, four malicious and one benign. Its benign class is not a separate acquisition: applications that fell into none of the four malicious categories were taken as benign and then checked against VirusTotal. Its published distribution, drawn in Figure 4.3(b), holds 11,598 applications, which is not the number collected: 17,341 applications were gathered, 13,077 executed successfully in the dataset authors' sandbox, and the labelled distribution retains 11,598 after the analysis records of the remainder failed to parse [160]. This chapter uses the published distribution, so it inherits a selection made by dynamic analysis. The categories are far more even than the Drebin families, since the largest holds 3.1 times as many samples as the smallest.

Evaluation tasks

Binary detection separates malicious from benign, and it is reported on both corpora. On Drebin the negative class is the AndroZoo sample described above; on CICMalDroid it is the 1,795 benign applications of Figure 4.3(b) against the remaining 9,803.

The malware-family classification task assigns a malicious application to one of the twenty Drebin families. No benign application takes part, and the task presupposes that the sample is malicious.

The malware-category classification task assigns an application to one of the five CICMalDroid categories, one of which is Benign. It therefore includes the detection decision within a wider one.

The tasks have different class priors and majority-class baselines. The corpora also differ in their treatment of adware. Drebin removed every application its scanners labelled adware, while CICMalDroid keeps Adware as one of its four malicious categories. A sample of the kind Drebin discarded is therefore a positive case on CICMalDroid and absent from Drebin, which limits comparisons between the two classification tasks.

Split and repetition policy

Each corpus was divided at random into 70% for training, 20% for testing and 10% for validation. The partition was shared by all methods and held fixed across repetitions. Each configuration, including GCN, GraphSAGE, HGNN, HGNN+ and HGANN-Mal, was trained five times with different random seeds. The tabulated learned-model scores are arithmetic

Table 4.2. Binary detection results (%). Learned-model scores are five-run means on a shared fixed partition per corpus; F1, precision and recall are macro-averaged over both classes. Author-corrected Drebin macro scores supersede the corresponding entries in the source publication [6]. Best values in bold.

Method	Drebin and AndroZoo				CICMalDroid			
	F1	Prec.	Rec.	Acc.	F1	Prec.	Rec.	Acc.
GCN	92.2	93.0	91.5	92.8	90.6	90.6	90.6	90.9
GraphSAGE	92.1	92.1	92.5	93.3	91.0	91.1	91.1	91.5
HGNN	96.6	96.8	96.3	96.8	94.2	94.8	94.3	94.6
HGNN+	96.1	95.9	96.1	97.1	95.9	96.1	96.5	97.7
HGANN-Mal	98.2	98.2	98.2	98.3	96.2	96.2	97.6	97.6
Majority class	63.0				84.5			

means over these five runs. These repetitions sample training randomness conditional on the fixed partition.

At that test fraction the smallest Drebin family, with 41 samples, places about eight samples in the test partition in expectation, so a per-family score for the smallest classes would rest on a handful of decisions. The training and test partitions draw from the same collection period. No deduplication or hash-aware grouping was applied, although Drebin contains repackaged near-duplicates.

4.10 Binary Detection Results

Binary detection tests whether the application-level representation separates malicious APKs from benign ones. Table 4.2 compares HGANN-Mal with the pairwise graph models GCN and GraphSAGE and the hypergraph models HGNN and HGNN+. The evaluation follows the shared partitions defined in Section 4.9; all learned-model scores are means over five training seeds. Precision, recall and F_1 are macro-averaged across the malicious and benign classes, so each class contributes equally. Figures 4.4 and 4.5 place these binary results alongside the multiclass results reported in the following sections.

Drebin and AndroZoo

HGANN-Mal achieved the highest mean scores on this corpus: 98.3% accuracy and 98.2% macro- F_1 . Relative to the strongest pairwise result for each metric, these are gains of 5.0 percentage points in accuracy over GraphSAGE and 6.0 points in macro- F_1 over GCN. The non-attentive hypergraph models also scored above both pairwise baselines. Among them, HGNN+ reached the higher accuracy, 97.1%, while HGNN reached the higher macro- F_1 , 96.6%. HGANN-Mal exceeded these values by 1.2 and 1.6 points, respectively.

The author-supplied five-run summary matrix gives a more concrete account of the detection errors. For the 3,006 test applications, it reports 1,085 true positives and 27 false negatives, with 24 false positives and 1,870 true negatives. These entries correspond to detection of

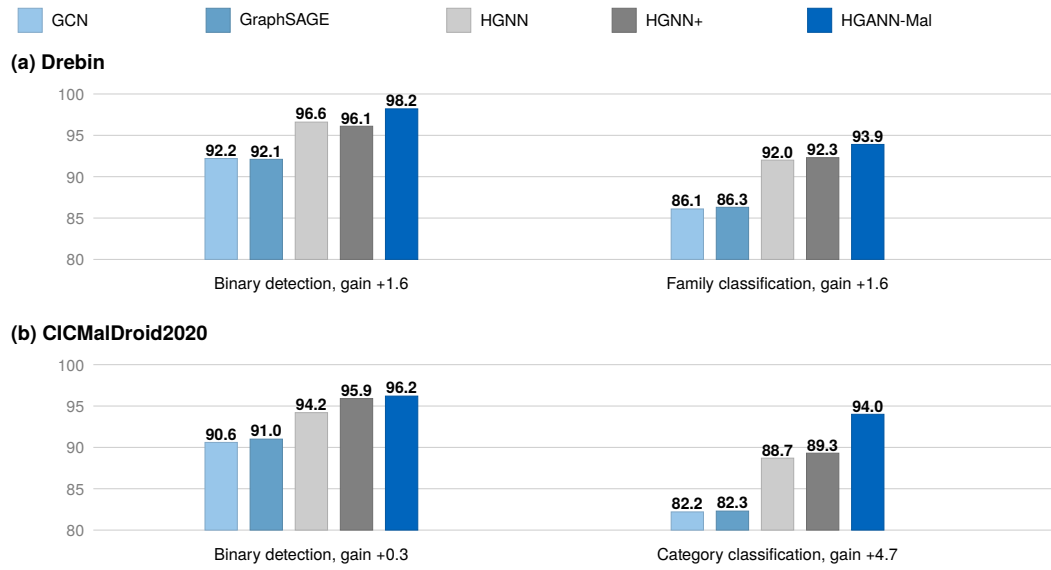


Figure 4.4. Mean Macro-F1 over five training seeds on each fixed split, from Tables 4.2 and 4.3. The vertical axis is truncated at 80 so that differences among the hypergraph methods remain visible. The gain beside each task is the margin of HGANN-Mal over the strongest baseline.

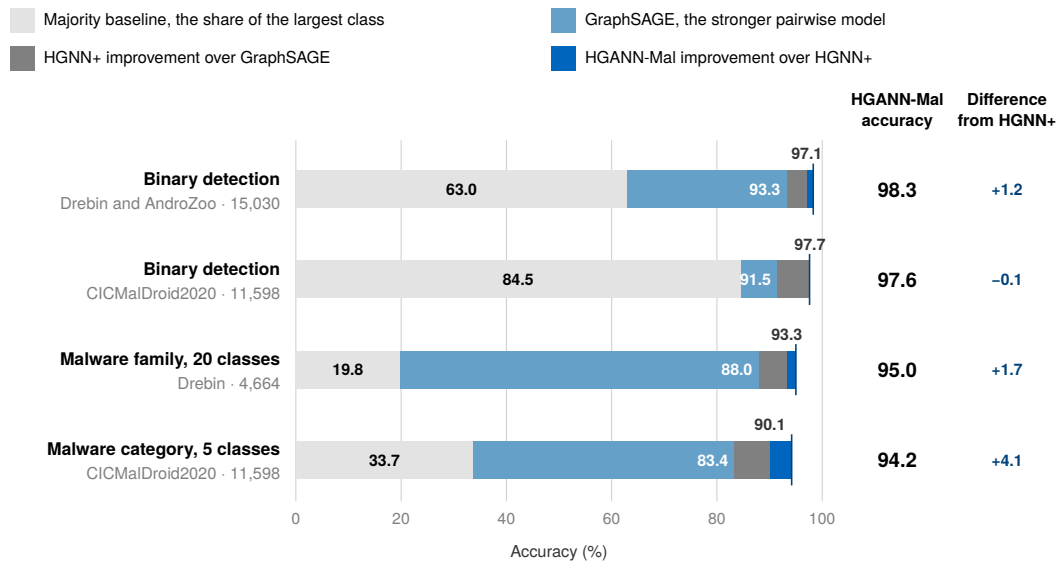


Figure 4.5. Mean accuracy over five training seeds on each fixed split, from Tables 4.2 and 4.3, divided at the majority baseline, at GraphSAGE and at HGNN+. The vertical tick marks the accuracy of HGANN-Mal. On CICMalDroid binary detection it lies 0.1 points below HGNN+.

97.6% of the malicious applications and a false-positive rate of 1.3% among benign applications. Malware-specific precision is 97.8%, and malware-specific F_1 is 97.7%. Equal averaging over both classes gives the rounded macro scores of 98.2% reported in Table 4.2. Compared with the HGNN summary matrix, missed malware decreases from 63 to 27 applications, while false alarms decrease from 33 to 24. The improvement therefore appears in both types of detection error.

CICMalDroid

On CICMalDroid, HGANN-Mal reached 97.6% mean accuracy and 96.2% macro- F_1 . These exceed GraphSAGE, the stronger pairwise baseline on both metrics, by 6.1 and 5.2 percentage points. All hypergraph models again scored above the pairwise models in accuracy and macro- F_1 . Within the hypergraph group, HGANN-Mal recorded the highest macro- F_1 , 0.3 points above HGNN+, whereas HGNN+ recorded the highest accuracy at 97.7%, 0.1 points above HGANN-Mal. The ranking thus depends on the measure: accuracy weights applications equally, while macro- F_1 weights the two class-specific F_1 scores equally. Across both corpora, the binary results support higher-order modelling within the evaluated static-analysis pipeline.

Class balance and interpretation

The two corpora place different weights on detection errors. Malware accounts for 37.0% of the Drebin–AndroZoo corpus and 84.5% of CICMalDroid. Predicting only the majority class would therefore yield accuracies of 63.0% and 84.5%, respectively; HGANN-Mal exceeds these reference values by 35.3 and 13.1 percentage points.

The Drebin summary also permits a sensitivity calculation for a lower malware prevalence. Holding its malware recall and false-positive rate fixed, Equation 2.6 gives a malware-specific precision of 89.5% at 10% prevalence, compared with 97.8% in the test cohort.

4.11 Malware-Family Classification on Drebin

Family classification identifies which known malware family an application belongs to. The evaluation uses 4,664 malicious applications from the twenty largest Drebin families, whose sizes are shown in Figure 4.3(a). Each application is assigned one of these twenty family labels. The left panel of Table 4.3 reports the mean scores over five training seeds on the shared fixed partition described in Section 4.9.

HGANN-Mal achieved the highest mean scores on all four measures, with 95.0% accuracy and 93.9% macro- F_1 . Its macro-precision and macro-recall were 94.1% and 93.8%, respectively. GraphSAGE, the stronger pairwise baseline in accuracy and macro- F_1 , reached 88.0% and 86.3%. The corresponding gains for HGANN-Mal are 7.0 and 7.6 percentage points.

Both non-attentive hypergraph models also exceeded the pairwise baselines. HGNN+ obtained the stronger results within this group, with 93.3% accuracy and 92.3% macro- F_1 . HGANN-Mal

Table 4.3. Multiclass classification results (%). Learned-model scores are five-run means on a shared fixed partition per corpus; F1, precision and recall are macro-averaged over classes. Drebin uses twenty malware families; CICMalDroid uses five categories, including Benign [6]. Best values in bold.

Method	Drebin, 20 families				CICMalDroid, 5 categories			
	F1	Prec.	Rec.	Acc.	F1	Prec.	Rec.	Acc.
GCN	86.1	85.9	86.2	87.8	82.2	82.0	82.4	82.7
GraphSAGE	86.3	86.7	86.2	88.0	82.3	82.2	82.6	83.4
HGNN	92.0	92.1	92.4	92.0	88.7	88.8	88.8	89.0
HGNN+	92.3	92.5	92.5	93.3	89.3	89.5	89.8	90.1
HGANN-Mal	93.9	94.1	93.8	95.0	94.0	94.1	94.0	94.2
Majority class	19.8				33.7			

improved on these scores by 1.7 and 1.6 percentage points, respectively. The same ordering appears in Figure 4.4. These results support the use of higher-order program representations to distinguish malware families from static application code in this evaluation.

Family sizes range from 41 to 925 applications. The largest family, FakeInstaller, accounts for 19.8% of the selected corpus, which provides the reference accuracy for a rule that assigns every application to that family. Accuracy weights each application equally, so larger families contribute more to the result. Macro- F_1 gives each family equal weight through the mean of its family-specific F_1 scores [129]. The gains on both measures therefore show an advantage under both weighting conventions. The aggregate scores describe performance across the twenty selected families; assessment of individual families, particularly the smallest ones, requires their class-specific scores and test-set counts. The corpus labels and the treatment of repackaged applications are described in Section 4.9.

4.12 Malware-Category Classification on CICMalDroid

The CICMalDroid task assigns each application to one of five categories: Adware, Banking malware, SMS malware, Riskware or Benign. The evaluation uses the 11,598 applications described in Section 4.9. It assesses both recognition of benign applications and distinction among the four malware categories. Predicting the wrong malware category counts as an error even when the application is recognised as malicious. The right panel of Table 4.3 reports the mean scores over five training seeds on the shared fixed partition.

HGANN-Mal achieved the highest mean scores across all four measures: 94.2% accuracy and 94.0% macro- F_1 , with macro-precision of 94.1% and macro-recall of 94.0%. GraphSAGE, the stronger pairwise baseline in accuracy and macro- F_1 , reached 83.4% and 82.3%, respectively. The corresponding gains for HGANN-Mal are 10.8 percentage points in accuracy and 11.7 points in macro- F_1 . Both non-attentive hypergraph models also exceeded the pairwise baselines on these measures.

Among the non-attentive models, HGNN+ obtained the higher accuracy, 90.1%, and macro- F_1 , 89.3%. HGANN-Mal exceeded these scores by 4.1 and 4.7 percentage points, respectively. These are its largest gains over the strongest non-attentive baseline among the four task-dataset combinations in this chapter, as shown in Figures 4.4 and 4.5. On the binary task for the same corpus, the corresponding macro- F_1 gain was 0.3 points, while accuracy was 0.1 points lower (Section 4.10).

SMS malware is the largest category, with 33.7% of the corpus. Assigning every application to this category would give the majority-class reference accuracy in Table 4.3. Macro- F_1 complements accuracy by giving each of the five categories equal weight, including Benign. The reported gains therefore hold both when applications contribute equally to accuracy and when categories contribute equally to macro- F_1 , under the evaluation conditions defined in Section 4.9.

4.13 Efficiency Bounds and Future Evaluation

Ablation priorities

The current evaluation changes two parts of HGANN-Mal together: the method groups (hyperedges) and the weights that set each method's participation in those groups. Its results measure the complete system. To test attention alone, future work could keep the same groups and compare fixed with learned membership weights. Both versions need matched normalisation, because uniform coefficients also change propagation scale (Equation 4.17). A separate comparison with topological groups alone, followed by removal of one group type at a time, would show the value of the additional groups.

Efficiency

The sparse representation provides explicit storage bounds. Let n and m count methods and retained hyperedges, let $\mathcal{S}$ be the slice set, and let $\nu = \text{nnz}(H)$ count stored memberships. Algorithm 4.1 proposes at most two candidates per method and one per slice. With at most $k + 1$ members per semantic hyperedge and $c_{\max}$ for the other types,

$$m \leq 2n + |\mathcal{S}|, \quad \nu \leq (c_{\max} + k + 1)n + c_{\max}|\mathcal{S}| = 70n + 64|\mathcal{S}|, \quad (4.19)$$

where the numerical bound uses $k = 5$ and $c_{\max} = 64$. Stored memberships therefore grow linearly with the numbers of methods and slices. For one attention head with input width F and output width F' , sparse propagation requires

$$T_{\text{layer}} = O(nFF' + \nu F'), \quad M_{\text{layer}} = O(nF + (n + m)F' + \nu + FF') \quad (4.20)$$

time and memory. The incidence products in Equation 4.12 avoid forming the potentially dense method-to-method matrix P_{att} .

Construction has separate costs. With $|E|$ call edges, breadth-first searches for topological groups take $O(n(n + |E|))$ time. Exhaustive semantic search compares $O(n^2)$ pairs of 320-dimensional embeddings; a tighter bound depends on the unspecified FAISS index. Slice-path searches take $O(\sum_s |F_s|(n + |E|))$, with F_s defined in Section 4.7. Candidate generation precedes size filtering, and centrality computation and code embeddings add further work. These are analytical bounds; end-to-end runtime measurements remain future work.

Interpretability and extensions

Slice-aware hyperedges identify code near security-sensitive API calls. An analyst could inspect this code and the weights assigned to its groups when investigating a classification. The hyperedge ranking in Equation 4.16 could help select groups for inspection. The source publication reports no learned weights or tests of their value as explanations, so this use remains a proposal.

Each hyperedge also has a tag that records how it was formed: from call-graph proximity, similar code, or a slice around a sensitive API call. The current model stores the tag but does not use it during propagation. A future model could learn one weight per type or include the tag in the attention score. To let different methods prefer different types, the effect of the tag must depend on the method's features. Giving every method the same type-specific bonus would retain the shared ranking of Equation 4.16 [79]. These extensions require evaluation.

4.14 Limitations and Threats to Validity

Comparison scope. Common call graphs and node features support the comparison of graph and hypergraph models under shared evaluation partitions. The measured gains concern the complete HGANN-Mal configuration; the individual contribution of attention remains a question for the ablations in Section 4.13. The baselines assess general learning operators within this pipeline. Comparisons with dedicated Android detectors would extend this evidence to a broader range of detection systems.

Evaluation coverage. Drebin and CICMalDroid provide distinct application collections and classification tasks. The five-run means describe performance on fixed random partitions; individual run scores are unavailable for quantifying variability. Evaluation on later application releases would require time-ordered splits [9]. Grouping repackaged applications before partitioning would also address the potential overlap between training and test code identified in Section 4.9.

Static evidence and robustness. Package-based analysis supports classification before execution, with coverage determined by the bytecode and call targets recovered by the analyser (Section 4.3). Additional runtime or native-code analysis could broaden this coverage. The experiments evaluate applications as collected; dedicated tests of obfuscation and detector-directed evasion remain future work. Published attacks on call-graph detectors provide relevant candidates for such tests [159, 122, 154]. Their application to HGANN-Mal would assess robustness under the explicit adversary assumptions described in Chapter 6.

4.15 Chapter Summary

This chapter addressed RQ2 through HGANN-Mal, which represents groups of related methods within an Android application. Hyperedges capture call-graph proximity and similarity between method representations. Slice-aware groups add context around security-sensitive API calls, and attention learns how strongly each method participates in its groups. The resulting application representation supports binary detection and prediction of family or category labels from static code. The mathematical analysis established properties of this attention operator and bounds for storage and sparse propagation.

The evaluation compared graph and hypergraph models on Drebin and CICMalDroid under shared fixed partitions. Across the four task–dataset combinations, every hypergraph configuration exceeded the stronger pairwise baseline by 3.1 to 10.8 percentage points in mean accuracy over five training seeds. HGANN-Mal exceeded the strongest non-attentive hypergraph baseline in three of these settings, with accuracy gains of 1.2 to 4.1 points. On CICMalDroid binary detection, HGNN+ achieved 97.7% accuracy and HGANN-Mal achieved 97.6%. The largest gains for HGANN-Mal occurred in CICMalDroid category classification.

These results support higher-order modelling of static program context within the evaluated configurations. Section 4.13 identifies the experiments needed to measure the separate contributions of construction and attention. In the dissertation, HGANN-Mal complements Hybroid through its focus on relations within the application package, while Hybroid combines program structure with observed network traffic. Chapter 7 examines what these two approaches contribute to contextual security analysis.

Anomaly Detection in Industrial Control Systems

5.1 Scope, Research Question, and Contributions

This chapter answers RQ3, which asks how a modular learning framework can analyse network and process features for anomaly detection in industrial control systems (Section 1.3), and addresses gap G3. The framework, NADICS, supports researcher-guided model development through configurable feature preparation and comparison of interchangeable learning components. In the German Industrie 4.0 security project IUNO (Section 5.4), its network application uses a protocol-specific extractor for S7comm and PROFINET. A GAN-based extension analyses multivariate SWaT process windows with elements of MAD-GAN and TadGAN.

The two industrial applications use different datasets and decision units. The network evaluation classifies packet-derived connection records, while the SWaT study derives timestamp decisions from process windows. They share a learning interface, but their results are assessed under separate experimental protocols.

Released code and IUNO deliverables document the original framework [25, 35]. The SWaT study is recorded in an unpublished internal report written by the candidate. Because that report is a research record, it is not a bibliography item. The candidate designed the method, implemented the experimental pipeline, ran and analysed the experiments, and wrote the report. Section 5.5 presents its empirical evaluation.

Section 5.3 retains a short exploratory IoT study as an example of relation-based monitoring [7]. It was separate from NADICS and provides supporting context; the answer to RQ3 rests on the industrial framework and its evaluations.

5.2 Observation Models and Datasets

NADICS (Network Anomaly Detection for Industrial Control Systems) is the detection framework described in Section 5.4 [25]. It receives tabular records. Its network path takes connection features derived from the industrial protocols S7comm and PROFINET, or from conventional network traffic. The SWaT path consumes the sensor and actuator values of a six-stage water-treatment process, sampled once per second [34, 26].

The industrial evaluations use different decision rules. The S7-300 evaluation (Section 5.4.1) uses labelled classification. The GAN study (Section 5.5) fits mixed normal and attack windows,

Table 5.1. Observation units in the industrial evaluations and the exploratory IoT study. Labels, sampling processes, and validation protocols differ between the studies.

Study	Unit observed	Representation	Decision
NADICS in IUNO	packet-derived connection record	protocol and flow features	binary or attack-class label
SWaT GAN study	sensor and actuator sample	multivariate time window	anomalous or normal timestamp
IoT service graph (exploratory)	inter-service message	vertices and whitelisted relations	known or unknown relation

combines reconstruction and critic scores, and applies an adaptive threshold to assign a binary prediction to each timestamp. An attack present during fitting may enter its learned reference distribution. Table 5.1 also includes the service-relation unit of the exploratory study below.

5.3 Exploratory Study: Communication Graphs for IoT Services

The collaborative study presented at PAM 2018 examined communication whitelisting for IoT microservices [7]. Its monitor operates at the Knowledge Agent, the entry point of the Virtual State Layer middleware through which services exchange data [162]. Each computing node stores a graph whose vertices are service addresses and whose edges record observed communication partners. This observation model captures service relations at the middleware level, in contrast to the packet-derived records used in the industrial evaluation.

A learning phase begins when a service starts and treats the relations observed during that period as normal. A later relation absent from the graph raises a notification. The site owner can approve it, after which the monitor adds it to the allowed relations. This update procedure permits new legitimate partners during operation, subject to the assumption that the initial learning period represents benign behaviour. The prototype uses concrete local addresses; site-independent service models remained a proposed extension [7].

The source reports a data-structure memory estimate of 33.7 KB for twenty services with ten relations each. Measurements with ten to forty services give an added latency of about 6 ms over an inter-service round trip of 24 ms, although the repetition count and the summary statistic behind these values are unspecified. A short graph-update trace records service start-up and the addition of a service, followed by a notification for a previously unseen connection [7].

The evidence documents prototype operation and resource costs. Detection quality was not measured against a labelled attack set, so the study provides neither attack recall nor a false-positive rate. The decision rule identifies unfamiliar communication partners. A new legitimate relation may therefore raise an alert, while malicious activity that preserves the permitted relations remains outside the rule’s scope.

5.4 The NADICS Framework and IUNO Integration

NADICS, Network Anomaly Detection for Industrial Control Systems, is a Python framework built at the Chair of IT Security of the Technical University of Munich to detect attacks against PLC and SCADA installations from network traffic. The candidate developed it to help researchers construct a suitable detection pipeline for a new environment or dataset. Its design goal was semi-automated model development: researchers define the prediction task and candidate configurations, while the framework coordinates repeated training and comparison. The intended selection process includes data preparation and hyperparameter tuning alongside the choice of learning algorithm.

The framework is organised in three modules. Preprocessing prepares the feature table through encoding and scaling, with optional resampling for class imbalance. Feature-importance analysis guides the researcher's choice of input variables. The learning module holds the estimators behind one interface, so that classical models and neural models can be exchanged by name without the pipeline being rewritten: the released repository contains the scikit-learn estimators, the IUNO deliverable additionally reports a deep neural network and a convolutional network [25], and the generative adversarial detector of Section 5.5 was added later at the same interface for multivariate process data. Postprocessing produces evaluation reports and supplies detections to site-specific alarm handling. The S7comm and PROFINET extractor existed as a separate project artefact, so the released repository takes a feature table as its input.

The IUNO deliverable records the need to produce labelled traffic from a real industrial environment [25]. For a new dataset, researchers must establish its feature meanings and the available labels before choosing compatible methods. The layout and model registries allow these choices to change without a separate training and reporting script for each algorithm. The objective is to identify a configuration suited to that dataset and a stated criterion, such as attack recall subject to a tolerable false-positive rate. The preferred configuration is conditional on the candidates considered and the evaluation protocol.

IUNO, the German national reference project for IT security in Industrie 4.0, ran from 2015 to 2018. Its consortium joined industrial companies such as Volkswagen with research institutes, among them Fraunhofer AISEC, Fraunhofer SIT, DFKI and TUM [42]. The partners built security tools and tested them in four use cases [35]. NADICS was developed for the use case *Visual Security Control Room*. This control room captured data streams from production in real time and displayed the security state of the production network in one view, so that operators could act on a deviation before the process came to a halt [25].

Figure 5.1 shows the pipeline as it was deployed in that use case. Two data artefacts separate its three stages. The first stage produced the corpus. Attacks of five classes (authentication bypass, CPU stop and start, reconnaissance, denial of service and brute force) were executed in a real industrial environment against Siemens S7-300 controllers that exchanged S7comm traffic [36] with a control server. The traces were labelled both in binary form and by attack class [25]. In the second stage, the public analysers TShark, Bro-IDS and Argus were combined with dissectors written for S7comm and PROFINET [163], and the feature extractor reduced

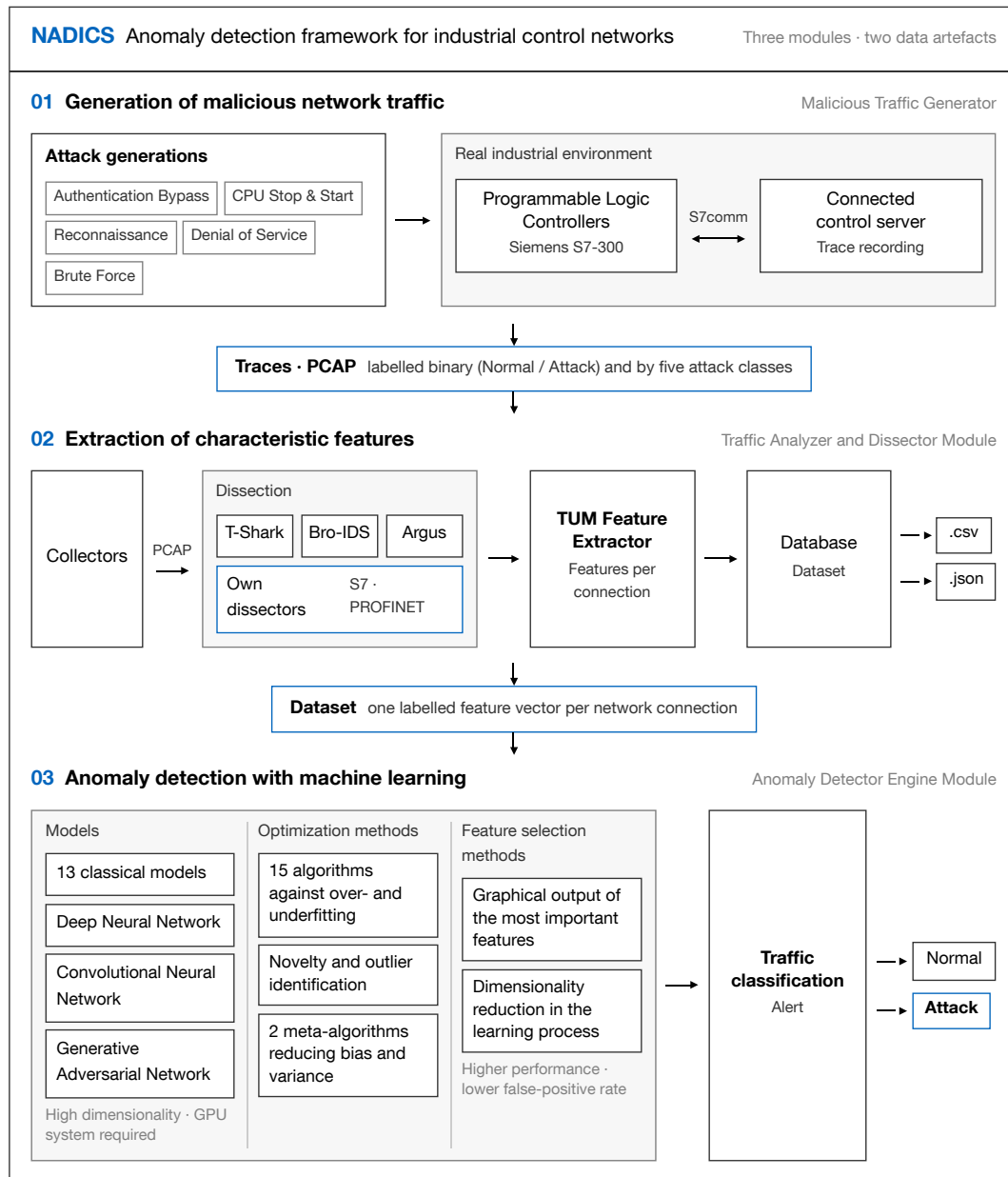


Figure 5.1. Architecture of the NADICS framework. Malicious traffic is generated in a real industrial environment, dissected into protocol-characteristic features, and classified by the anomaly detection engine. The stage structure follows the one reported in the IUNO deliverable [25].

each network connection to one labelled feature vector, written as CSV or JSON [42, 25]. The third stage is the framework proper. It reads that dataset, fits one of the registered estimators, applies the resamplers and the outlier tests, and reports which features carried the decision. Detections reached the control room as JSON event messages over the authenticated message bus of the IUNO aggregation system [25]. The response to an alert was left to the control room operators [35].

The modules map onto these stages at the dataset. Preprocessing spans the second stage and the input side of the third, while learning and postprocessing both sit in the third stage. Corpus generation lies outside the framework. A layout file specifies the input columns and target labels, together with the feature subset and the string fields that require encoding. Model registries map names to estimators. The original repository contains thirteen classifiers and four novelty or outlier detectors, with fifteen resamplers and two boosting meta-estimators. The registry named clustering supplies the novelty and outlier methods. Supervised learners use `fit` and `predict`; the outlier routines also use score functions or `fit_predict`, according to the estimator interface [164].

The archived runner accepts a researcher-specified list of classifiers and reports their predictive scores together with training and prediction times. Researchers can request feature-importance estimates from an Extra-Trees ensemble and revise the selected columns in the layout. The `learning.py` module defines a `GridSearchCV` helper with five-fold cross-validation over pre-defined parameter grids for decision trees, random forests and k -nearest neighbours, with scoring left at the estimator default. These components document the intended automation of experiment execution and bounded parameter search.

Model selection for a new environment requires a development protocol separate from final testing. Each validation split must fit data-dependent preprocessing and feature selection on its training portion; resampling must leave validation observations unchanged. After selecting a configuration, researchers assess it on held-out data and define how its predictions should be converted into operational alerts. This workflow reuses the analysis components while retaining domain-specific decisions about labels and acceptable alarm burden. The industrial experiments below evaluate configured detectors; they do not measure an advantage of automated pipeline selection over manual development.

Figure 5.2 shows where the framework ran. The environment was built at the Volkswagen plant in Wolfsburg from hardware of the kind used in regular production [42]. A Hirschmann industrial switch carries three segments, the intranet, the demilitarised zone and the plant network, and manually configured Eagle routers separate them. The first of four servers runs the WinCC and OPC UA clients that originate test traffic, together with a Nagios instance and the database of the aggregation system. The second is the virtualisation host for the detection tools contributed by AISEC, DFKI, SIT and TUM, and the third serves the visual control room to the intranet. The fourth combines the WinCC server with the collector, which gathers traffic from the plant network and passes it, bundled and pre-filtered, to the detection tools, so NADICS was one of four detectors reading the same collector output. Siemens controllers occupy the plant network behind a Scalance switch. The collector observed that network passively at the plant switch, never in line with the traffic, which matches the detection-only design: an alert could reach the control room, but nothing in the path could stop a packet.

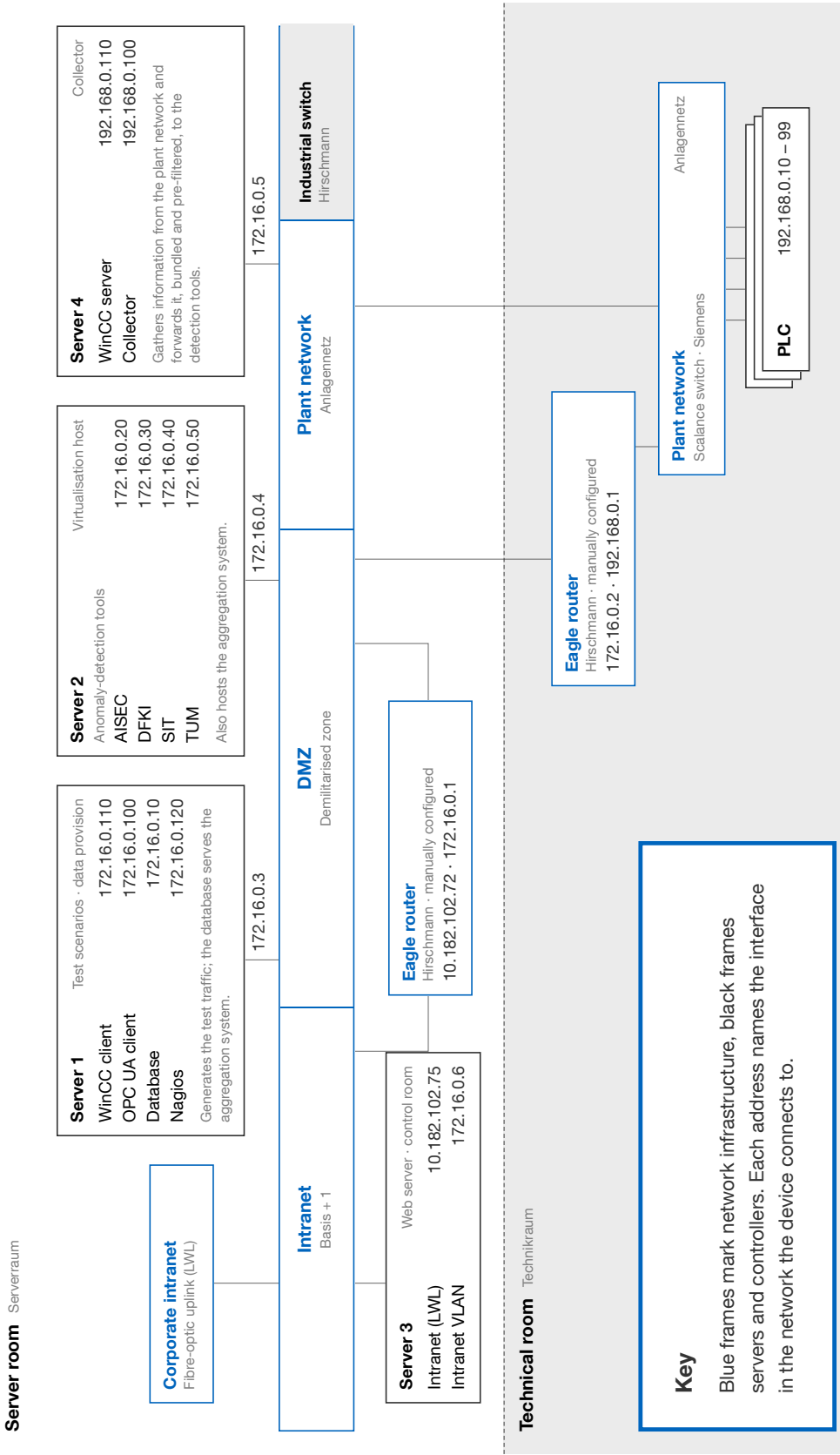


Figure 5.2. The IUNO test environment at the Volkswagen plant in Wolfsburg. Four servers, three network segments and a Siemens plant network reproduce the production setting in which NADICS ran as the TUM detection tool on the virtualisation host. The network TAPs fitted at the relevant interfaces are omitted from the drawing [42].

Table 5.2. Composition of the S7-300 corpus and its partitions. Whole capture sessions were assigned to the partitions, four to training and one to each of the other two, and every attack episode lies entirely within one partition. The episode column counts executed attacks, not records.

Class	Corpus	Train	Validation	Test	Episodes
Normal	113,751	76,556	18,587	18,608	
Authentication bypass	1,058	441	269	348	5
CPU stop and start	593	359	90	144	5
Reconnaissance	1,803	1,070	366	367	5
Denial of service	1,230	765	278	187	5
Brute force	725	445	93	187	5
Total	119,160	79,636	19,683	19,841	25
Capture time (h:min:s)	24:00:00	16:01:57	3:56:27	4:01:36	

The IUNO deliverables describe the tool and its environment but print no measured result for it [25]. The values below come from the experimental records of the project and cover the corpus and its feature schema, the detection results on the held-out session, the alarm burden over a benign monitoring period, and the throughput of the extractor on the prototype collector.

5.4.1 The S7-300 Corpus

The corpus was recorded in 2018 in the environment of Figure 5.2, where six Siemens S7-300 controllers exchanged routine polling and control traffic with WinCC-based services. Complete selected channels were kept, together with ARP and DCP traffic and the connection attempts belonging to those channels. No random packet sampling was applied.

Recording opened with a two-hour clean baseline, which was excluded from the corpus totals. That interval fixed the communication relationships, the polling behaviour and the protocol activity expected under the operating modes present at the time. Six capture sessions followed, with 24 hours of labelled traffic in total. The collector recorded 12,616,875 Ethernet frames in that period, an average of 146.03 frames per second. The captured frames hold 2,317,074,962 bytes, and the six capture files occupy about 2.519 GB in the classic PCAP format.

Twenty-five controlled attack episodes were executed during the sessions, five repetitions of each of the five classes the deliverable names [25]. Background communication continued while they ran. Table 5.3 lists the duration of every episode. The shortest lasted 3 min 07 s and the longest 9 min 55 s, and together they cover 2 h 51 min 05 s. Episodes also differed in the channels they touched and in how often a connection was re-established. Labels were assigned from the execution logs and from inspection of the packet traces, and only the communications an episode actually affected received an attack label, so background traffic concurrent with an attack keeps the normal label. Two attack types were never run against one channel at the same time.

Extraction produced 119,160 feature records. Of these, 113,751 are normal and 5,409 carry one of the five attack labels, an attack share of 4.54%. Table 5.2 gives the composition. The training sessions hold 79,636 records, the validation session 19,683 and the test session

Table 5.3. Duration of the 25 attack episodes in minutes and seconds. Each attack type has three training episodes, one validation episode and one test episode.

Class	Training episodes	Validation	Test	All five
Authentication bypass	5:33, 4:01, 5:50	7:16	9:34	32:14
CPU stop and start	7:03, 8:23, 3:07	4:04	7:39	30:16
Reconnaissance	6:42, 9:40, 7:28	9:55	8:27	42:12
Denial of service	6:41, 9:39, 7:07	7:46	5:38	36:51
Brute force	8:05, 3:16, 4:06	4:15	9:50	29:32
Total				171:05

19,841. Each attack type contributes three training episodes and one episode to each of the remaining partitions, and because episodes differ in duration and in the channels they touch, the attack records of a class are not divided between the partitions in a fixed proportion. An episode, its recovery interval and the windows that depend on it stay inside a single partition, and no rolling window crosses a partition boundary. The attack counts must not be read as a count of attacks. A single episode of a few minutes generates many records, and the corpus averages about 216 affected records per episode.

The extraction schema has 63 fields in five groups: 18 flow fields, 13 timing fields, 18 protocol fields, nine relational fields and five process fields. Forty-six are numerical, eleven categorical and six binary, and encoding of the categorical fields expands the input dimensionality of the model. The flow group follows the output of Argus and Bro-IDS, which the extractor wrapped. Protocol fields cover S7 message and function codes, addressed memory areas, address entropy and error codes, together with COTP and OPC UA fields. Relational fields describe the communication partners, the age of an edge, and the number of distinct destinations and ports observed in the preceding minute. The process group reads one preconfigured anchor tag per channel, whose identity and units are held as metadata, so a value delta is never computed across unrelated tags. Unobservable protocol fields carry an explicit missing state, and an absent OPC UA security policy is not encoded as the policy *None*.

Figure 5.3 shows the importance of each field in the random forest of Section 5.4.2. The forest was fitted to the training partition; the impurity reductions from splits on each field were summed across its trees, normalised over all 63 fields, and ranked. Panel (a) shows the fifteen highest, and Appendix A lists all 63. *s7_function* leads at 11.9%, because a stop or a start is an ordinary S7 operation whose function code is visible in the request, and *role_violation* follows at 8.0%, because a write from a host whose role does not write is anomalous even over an established connection. Both signals have limits: legitimate maintenance issues the same function codes, and a compromised engineering station writes from a role that is permitted to write.

Panel (b) sets each group’s share of the total importance against its share of the fields. Protocol semantics holds the largest share at 36.4%, although it also holds the most fields. The comparison that carries information is the share per field: the nine relational fields average 2.38% each, against 0.74% for the eighteen flow fields and 0.60% for the five process fields. Only the process fields receive less importance per field than the flow statistics. That is consistent with the position the schema takes: an industrial detector built on Argus and Bro

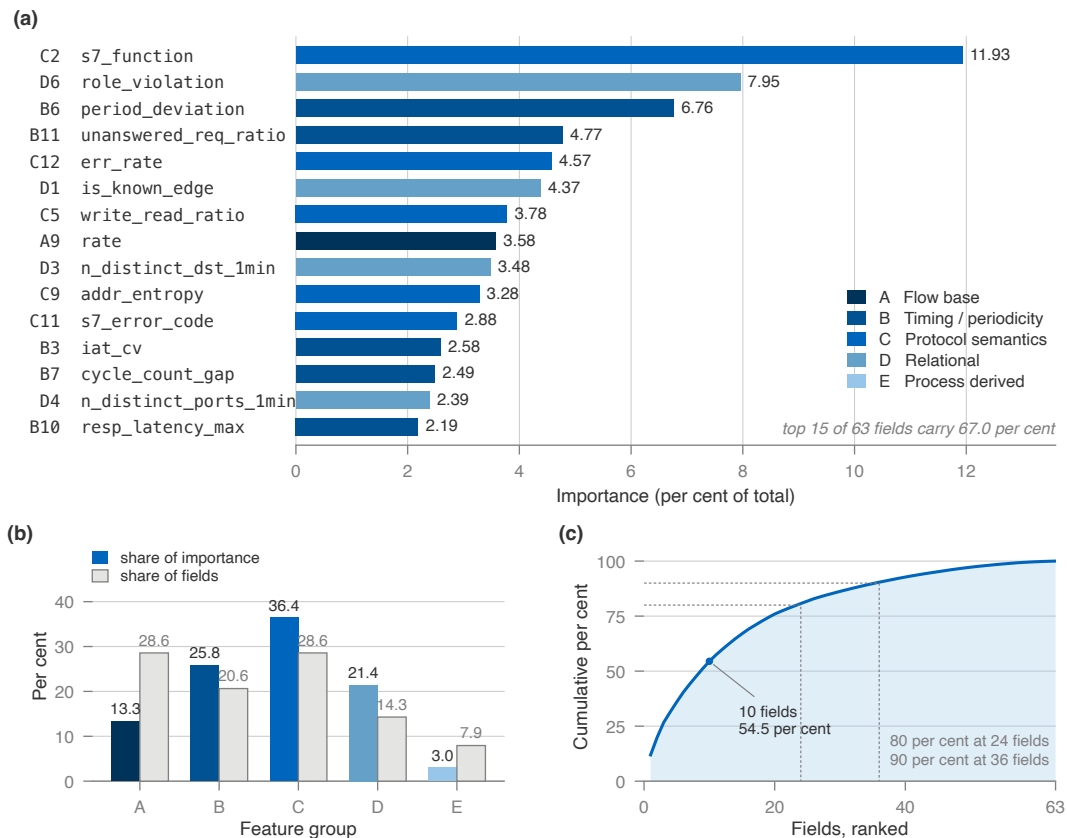


Figure 5.3. Random-forest importance of the 63 extraction fields. (a) The fifteen most important fields, coloured by group. (b) Each group's share of the total importance against its share of the field count. (c) Cumulative share over the ranked fields.

Table 5.4. Six-class confusion matrix on the 19,841-record test session, with rows giving the true class. Predicted classes are abbreviated: N normal, AB authentication bypass, CS CPU stop and start, RC reconnaissance, DS denial of service, BF brute force. The last three columns give precision, recall and F_1 for the class of that row.

True class	Predicted class						Score (%)		
	N	AB	CS	RC	DS	BF	Prec.	Rec.	F_1
Normal	18,518	21	5	31	18	15	99.74	99.52	99.63
Authentication bypass	22	292	8	12	1	13	88.75	83.91	86.26
CPU stop and start	3	1	134	2	3	1	90.54	93.06	91.78
Reconnaissance	9	7	0	337	6	8	84.25	91.83	87.87
Denial of service	5	0	0	4	178	0	85.99	95.19	90.36
Brute force	9	8	1	14	1	154	80.63	82.35	81.48

flow features alone discards most of what separates an S7 attack from S7 polling. Panel (c) gives the cumulative share. Ten fields reach 54.5% of the total importance, twenty-four reach 80% and thirty-six reach 90%, so the tail of the schema is thin and a reduced schema is plausible. NADICS exposes an importance ranking for exactly this purpose [25].

One record covers a non-overlapping ten-second window of an activity-bearing connection or channel, and a shorter final record is written when a session closes. PROFINET RT traffic is keyed at layer 2, since it carries no transport-layer connection. The windowing refines the per-connection unit of Figure 5.1 so that a change during a long-lived session can be scored while the session is still open. Channels that fall silent emit no record.

Three definitions were adjusted so that no record depends on information from after its own window closes. The duration field measures the exported segment and not the eventual lifetime of an open connection, while a separate field retains the time elapsed since the underlying connection opened. The ratio of unanswered requests counts only those requests whose one-second response deadline had already expired at window closure, and pending requests are excluded from both the numerator and the denominator. Trailing source and timing state is held over 60 seconds, and the period features are computed from past observations. Encoders, imputation and scaling were fitted on the training partition alone, and the classifier together with its preprocessing was then held fixed for validation and testing.

5.4.2 Detection Results

The classifier is a random forest of 300 trees with a maximum depth of 16 and a minimum leaf size of five. Class weighting was applied per bootstrap sample, and the training data were not resampled. Two decisions were read from the same fitted model: a binary verdict of normal against attack, and an assignment to one of six classes, the normal class and the five attack types. Every figure below comes from the held-out test session, which holds 18,608 normal and 1,233 attack records.

Table 5.4 reports the six-class confusion matrix together with the per-class scores. Collapsing the same predictions into the binary decision gives 18,518 true negatives, 90 false positives, 48 false negatives and 1,185 true positives. Binary accuracy is 99.30%, attack precision 92.94%

and attack recall 96.11%, for an attack F_1 of 94.50%. The false-positive rate over normal records is 0.484% and the balanced accuracy 97.81%. On the six-class task, accuracy reaches 98.85%, the macro- F_1 89.56% and the weighted F_1 98.86%.

Accuracy is the least informative of these numbers. Normal records make up 93.79% of the test partition, so a detector that never fired would already score 93.79%, and the gap between the weighted and the macro F_1 , 98.86% against 89.56%, measures how much of the six-class figure the normal class carries. Per-class recall is highest for denial of service at 95.19% and lowest for brute force at 82.35%, with authentication bypass close behind at 83.91%. Most errors of these two classes fall on other attack classes. Authentication bypass loses 22 of its 348 records to the normal class and 34 to the other attack types, while brute force loses nine of its 187 records to the normal class and 24 to the other attack types, 14 of them to reconnaissance. Both attacks reuse the protocol operations of legitimate engineering access, so a window holding only a few such operations offers little that separates them from ordinary traffic or from each other.

All of these scores are computed over records. They show that the extracted features separate the five executed attacks from the surrounding traffic of this plant under a session-disjoint split.

5.4.3 Benign Operation and Alarm Burden

A second exercise measured what the detector does when nothing malicious happens. It ran in the Volkswagen environment with four active PLCs and ordinary supervisory traffic. A separate two-hour baseline preceded 24 hours of benign monitoring. Over that day the collector recorded 8,472,913 frames, an average of 98.07 frames per second. The captured frames hold 2,186,039,471 bytes, about 2.322 GB in the classic PCAP format, and the extractor emitted 83,847 records. Engineering work and planned device restarts took place during the period and count as benign.

The detector classified 83,740 records as normal and 107 as attacks, all of them false, a record-level false-positive rate of 0.128%. Positives were merged into alarm episodes before presentation to the control room: positives that share a source, a destination and a predicted attack class belong to one episode, and an episode closes once 60 seconds pass without a further positive. One notification opens an episode, and later positives update it. Nine alarm episodes were raised over the day, 0.375 per hour on average, and each was assigned a benign cause. Engineering workstation activity accounts for 43 positive records in three episodes and OPC UA reconnections for 28 records in two. Brief communication interruptions produced 23 records in two episodes, and planned device restarts 13 records in two.

No attack was executed during this period, so every alarm it raised was false. Its 0.128% record rate lies below the 0.484% of the test session, whose normal records also include the recovery intervals of executed attacks. The recovered causes share a structure that matters for deployment: each is a legitimate departure from routine polling, so the alarm burden of this design follows the rate at which such events occur in the plant.

Table 5.5. Extraction workloads on the Raspberry Pi 3 prototype. Input frames are the frames presented to the extraction pipeline, and processed frames exclude those dropped before processing. Rates and CPU utilisation are averages over the replay, and CPU utilisation refers to the capacity of the whole device.

Quantity	Light load	Moderate load	Overload
Replay duration (s)	600.14	599.87	600.09
Input frames	599,827	1,498,463	2,100,731
Dropped frames	4	361	245,897
Processed frames	599,823	1,498,102	1,854,834
Frame loss (%)	0.00067	0.02409	11.70531
Input rate (frames/s)	999.48	2,497.98	3,500.69
Processed rate (frames/s)	999.47	2,497.38	3,090.93
Feature records	5,842	5,887	5,781
Mean CPU utilisation (%)	22.6	43.1	50.4
Peak extractor memory (MiB)	302	381	436
95th-percentile emission delay (ms)	96	234	817

5.4.4 Extractor Throughput on Constrained Hardware

The extractor was also exercised on a Raspberry Pi 3 Model B. The deliverable of June 2017 names Raspberry Pi boards as the prototype platform [42]. The board carries four cores at 1.2 GHz, 1 GB of RAM and 100 Mbit/s Ethernet. Only the extraction stage ran on it, comprising selected-field capture and dissection of S7comm and PROFINET traffic, the bounded rolling feature computations and emission of CSV or JSON. Model fitting and inference remained on the server, and archival of full packet captures to the SD card was excluded. Each load replayed traffic over 100 configured communication channels with varying channel activity for about ten minutes.

Table 5.5 reports the three loads. At the moderate load, 1,498,463 frames were presented in 599.87 seconds and 361 were dropped, a loss of 0.024% at a processed rate of 2,497.38 frames per second. This input rate is about 17 times the average frame rate of the corpus capture. The extractor used 43.1% of the CPU capacity of the device on average and at most 381 MiB of memory. The 95th percentile of the delay from the close of a window to the emission of its record was 234 ms, a delay that excludes the ten seconds in which the window accumulates. The light load lost four of its 599,827 frames at 22.6% CPU utilisation. The overload row does not describe a supported rate: 11.71% of the input was dropped, and the processed rate stayed at 3,090.93 frames per second against an input rate of 3,500.69. Mean CPU utilisation reached 50.4% there, and the 95th-percentile emission delay rose to 817 ms. The three runs emitted 5,842, 5,887 and 5,781 records, fewer than the 6,000 that 100 continuously active channels would give in ten-second windows over ten minutes, because channel activity varied.

5.5 A GAN-Based Multivariate Detector for SWaT

Industrial process recordings form multivariate time series: each timestamp associates sensor measurements with actuator states, and successive observations describe the evolution of the plant. An anomaly may appear as an unusual value or as a sequence that conflicts with the

expected relationships between variables. A measurement within its usual range can therefore be anomalous in its temporal or process context. MAD-GAN motivates joint modelling of these dependencies for industrial intrusion detection, while TadGAN addresses anomalous sequences across several time-series domains [165, 166]. Process monitoring can expose the physical effects of an attack, although a deviation alone does not establish its cause. Scarce labelled attack examples motivate methods that learn reference behaviour from recorded operation.

A generative adversarial network (GAN) learns a data distribution through two neural networks with opposing training objectives [167]. The generator transforms samples from a latent distribution into synthetic observations. The discriminator learns to distinguish recorded observations from generated samples, and the generator is updated to make that distinction harder. For time series, the generated objects are sequences of measurements. Here, *adversarial* describes the training relationship between the networks. The discriminator's training labels identify recorded and generated data; they are not attack annotations.

For anomaly detection, the learned distribution provides a reference against which new observations can be assessed. MAD-GAN uses long short-term memory (LSTM) networks to model multivariate windows and combines discrimination with reconstruction error [165]. Reconstruction requires a latent representation from which the generator can reproduce the observed sequence; MAD-GAN searches for this representation at inference. TadGAN learns an encoder for this mapping and adds a cycle-consistency loss that penalises differences between an input and its reconstruction. Its Wasserstein critics supply real-valued scores for matching distributions during training [166]. Reconstruction error and discriminator or critic output can then supply evidence of deviation. Their usefulness depends on the behaviour represented in training and on the subsequent scoring rule.

The present study combines multivariate process windows with the encoder and cycle-consistent training of TadGAN. It examines how five input representations affect detection and false alarms on a fixed SWaT evaluation cohort. The experiment concerns a detector developed with some attack observations, as specified below; its learned reference therefore includes both normal and attack behaviour. This distinction matters when comparing its results with methods trained only on normal operation.

The detector operates on process measurements from the Secure Water Treatment (SWaT) testbed. SWaT comprises six treatment stages, with programmable logic controllers coordinating sensors and actuators and a supervisory network connecting the controllers to supervisory control and data acquisition (SCADA) (Figure 5.4). The process dataset contains 51 sensor and actuator variables recorded at a nominal frequency of one observation per second [34]. These variables describe the physical behaviour of the plant, allowing the detector to consider changes in individual measurements together with relationships among measurements across the treatment process.

The dataset documentation describes 36 attacks distributed across four categories (Figure 5.5): 26 single-stage, single-point attacks; four single-stage, multiple-point attacks; two multiple-stage, single-point attacks; and four multiple-stage, multiple-point attacks [34]. These categories describe the scope of the interventions. They do not imply that all attacks produce

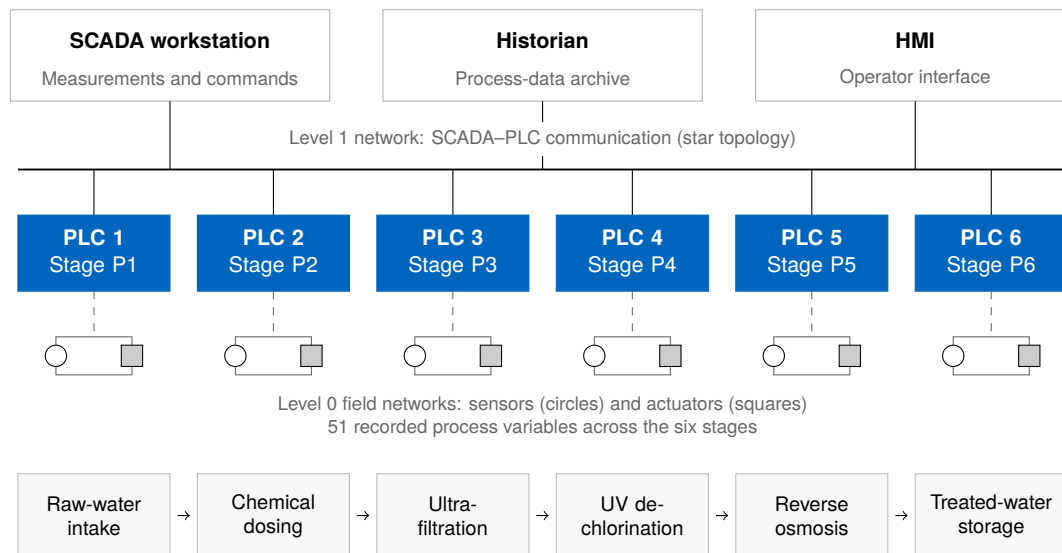


Figure 5.4. SWaT process and control architecture. Network levels follow the SWaT convention. The field-network symbols indicate sensors and actuators within each stage; the detector consumes their recorded values. Adapted from the supplied diagram and the dataset description [34].

equally observable process changes. An intervention affecting one sensor or actuator may also influence measurements elsewhere in the plant, while an attack with little physical effect may remain difficult to identify from process data alone. The aggregate calculations in this section concern anomalous timestamps and do not establish the number of individual attacks detected.

The source collection campaign comprises seven days of normal operation followed by four days containing attacks [34]. Preprocessing removes the first 21,600 observations from the normal-operation trace, which cover the initial six-hour stabilisation period (Figure 5.6). The empirical study uses all 449,919 timestamps of the attack-period recording. Its first 56,240 timestamps are divided between the training and validation partitions, which also receive separate portions of the normal-operation recording. The later 393,679 attack-period timestamps form the test set. They comprise 51,453 anomalous and 342,226 normal timestamps, so anomalous observations account for 13.07% of the test cohort.

All three partitions contain normal observations and labelled attack examples. Each attack episode is confined to one partition, and no window crosses a partition boundary or a discontinuity in the recordings. Partitioning precedes imputation, scaling, feature selection and window construction. Earlier observations support model development, while the later observations form the test cohort.

The GAN objective does not require class annotations during gradient-based fitting. Nevertheless, the complete procedure includes supervised feature selection and model selection, and training includes attack observations. The resulting evaluation therefore concerns a detector developed with prior exposure to attacks. It does not establish detection performance for previously unseen attack types or a wholly unsupervised development process.

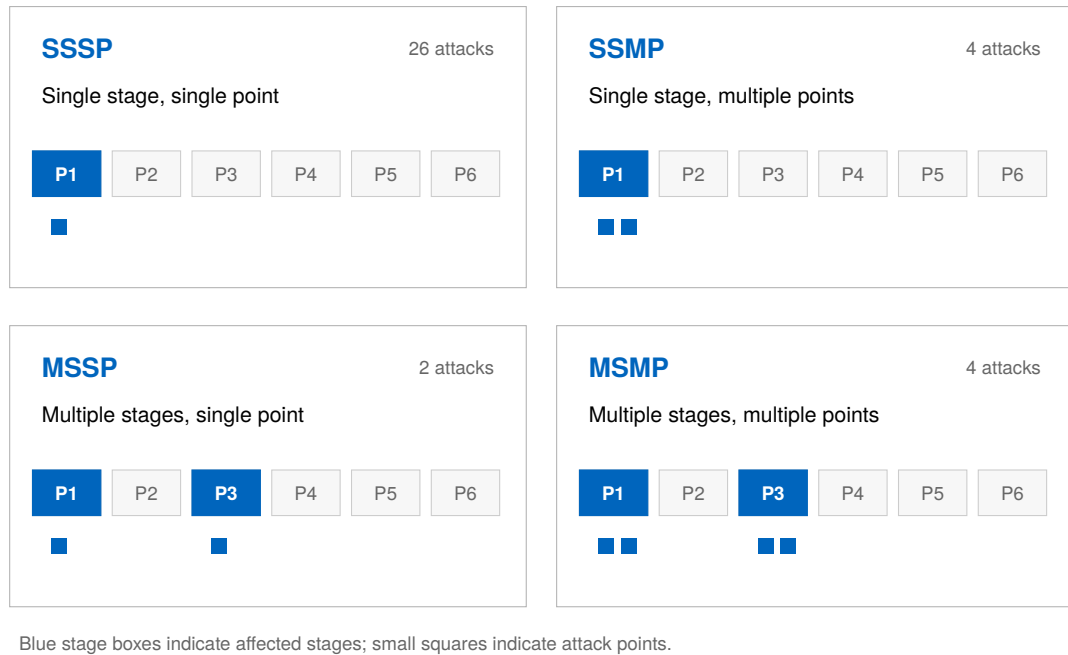


Figure 5.5. SWaT attack categories and their counts in the dataset paper [34]. Stage and point locations are schematic examples. The arrangement describes attack scope and carries no ranking of detection difficulty.

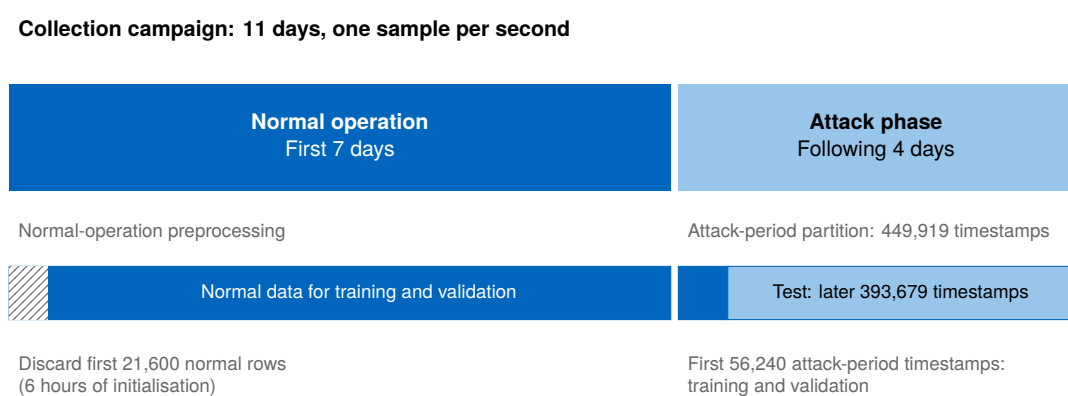


Figure 5.6. SWaT collection campaign and the partition used in the GAN study. The campaign follows the dataset paper [34]. After the initial normal-operation exclusion, the first 56,240 attack-period timestamps support training and validation; the later 393,679 timestamps form the test set.

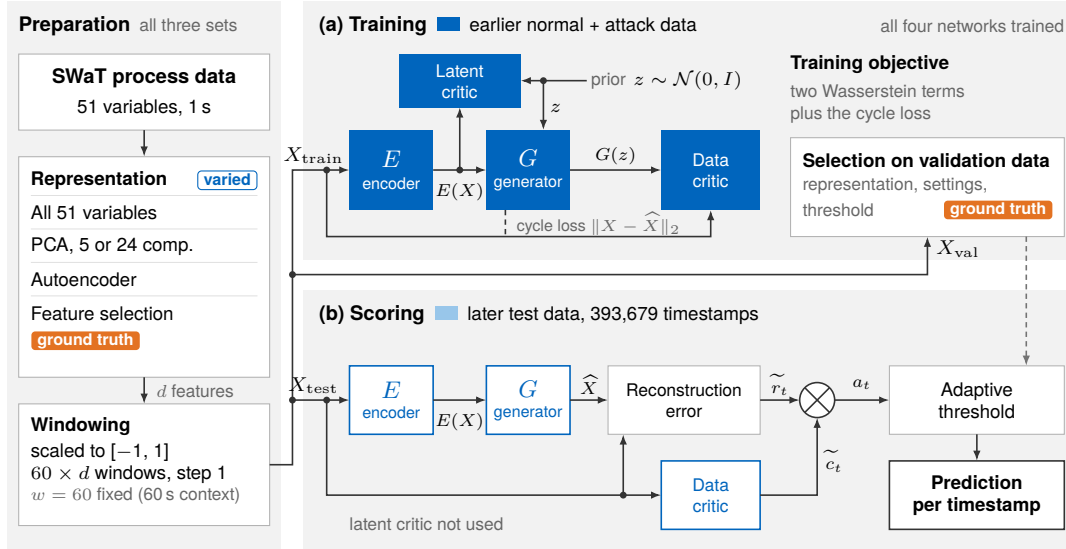


Figure 5.7. Multivariate GAN detection pipeline. (a) The four networks are trained together on earlier windows containing normal and attack observations. (b) The later 393,679 test timestamps are scored with the trained networks; the latent critic has no part in the score a_t of Equation 5.1. The blue tag marks the input representation, the only factor varied between runs, and orange tags mark development steps that used attack ground truth. The window length is fixed at 60 samples.

5.5.1 Model and Training

The model contains four jointly trained networks: an encoder, a generator, a data critic and a latent critic (Figure 5.7). For an input window X , the encoder E maps the observed process sequence to a latent representation $E(X)$. The generator G maps this representation back to the observation space, producing the reconstruction

$$\hat{X} = G(E(X)).$$

Latent samples z are drawn from a standard normal distribution. The data critic compares observed windows with generated windows $G(z)$, encouraging the generator to reproduce the distribution of the training sequences. The latent critic compares encoded observations $E(X)$ with samples from the latent prior. A cycle-consistency term penalises discrepancies between an observed window and its reconstruction,

$$\mathcal{L}_{\text{cyc}} = \mathbb{E}_X[\|X - G(E(X))\|_2].$$

Training combines the two Wasserstein objectives with this reconstruction constraint. The encoder provides a direct mapping from process observations to latent representations, while the cycle-consistency term encourages those representations to retain information required to reconstruct the input. The data critic supplies an additional measure of how closely an observed sequence resembles the learned training distribution. The latent critic contributes to training and is not included in the anomaly score at inference [166].

At inference, each test window is encoded and reconstructed using the fitted model. Discrepancies between the observed and reconstructed sequences provide a reconstruction score

(Figure 5.8). The data critic supplies a second score for the observed window. Before combination, the scores require a common orientation, such that larger values indicate greater deviation from the reference behaviour. Their normalisation must also prevent a product of two negative standardised values from being interpreted as strong anomalous evidence. The reported score therefore uses non-negative, consistently oriented normalised components:

$$a_t = \tilde{r}_t \tilde{c}_t. \quad (5.1)$$

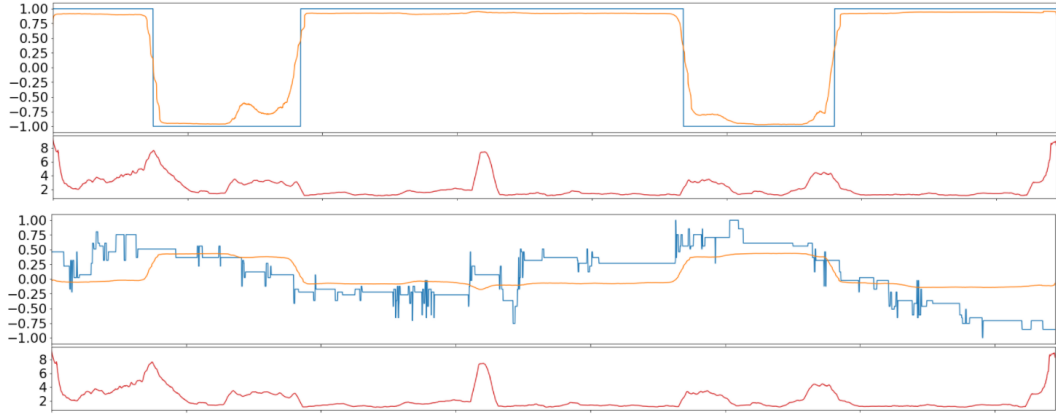


Figure 5.8. Observed signals (blue) and their reconstructions (orange) for two dimensions. The repeated red trace is the aggregate reconstruction discrepancy across dimensions, shared by the two displayed signals. It precedes the combination of Equation 5.1.

Score transformations are fitted on development data and fixed during evaluation. Reconstruction discrepancy and critic output jointly indicate deviation, without providing a calibrated attack probability. Normal operational changes can also produce high reconstruction error.

For each timestamp, overlapping-window reconstructions are averaged per variable before calculating the multivariate reconstruction discrepancy; critic outputs are averaged separately. Only windows within the same partition contribute. Each timestamp receives one combined score and prediction and is counted once in the confusion matrix.

An adaptive threshold converts scores into binary predictions and may merge adjacent anomalous intervals. Its historical context is distinct from the networks' 60-second input window. Because aggregation includes observations later than the timestamp being scored, the results describe offline detection; online performance and real-time detection delay remain unevaluated.

5.5.2 Input Representations and Settings

The representation comparison includes the complete set of 51 process variables, two principal component analysis (PCA) configurations, an autoencoder representation and a selected feature subset. The full representation retains the original measurements and actuator states. It provides the broadest input coverage but also exposes the model to redundant relationships and normal operational variation across the process.

PCA projects the measurements onto k orthogonal components. Let $\lambda_1 \geq \dots \geq \lambda_{51} \geq 0$ denote the eigenvalues of the covariance matrix estimated from training data. The retained variance fraction is

$$\rho_k = \frac{\sum_{j=1}^k \lambda_j}{\sum_{j=1}^{51} \lambda_j}.$$

The study compares $k = 5$ and $k = 24$, with $\rho_{24} \approx 0.99$. Retained variance does not quantify attack-relevant information; low-variance directions may still distinguish attacks.

The autoencoder learns a nonlinear representation for reconstruction before GAN processing. Feature selection combines correlation analysis with chi-squared selection, recursive feature elimination, LASSO and Random Forest importance. Supervised selectors use training labels only. Any non-negative transformation required for chi-squared selection is fitted separately from GAN scaling. The aggregate results do not specify the selected variables or the autoencoder bottleneck dimension.

All preprocessing is fitted on training data and reused for validation and testing. GAN inputs use a training-derived scale with reference range $[-1, 1]$. Values outside the training range remain unclipped to preserve deviations.

For feature vectors $\mathbf{y}_t \in \mathbb{R}^d$, each window is

$$X_t = [\mathbf{y}_t, \dots, \mathbf{y}_{t+59}]^T \in \mathbb{R}^{60 \times d}.$$

At the nominal sampling rate, this gives a 60-second context with a one-second stride. An uninterrupted sequence of $N \geq 60$ observations yields $N - 60 + 1$ windows; performance metrics count timestamps after aggregation.

The final configuration uses Adam with learning rate 5×10^{-4} , latent dimension 20, batch size 64 and 50 epochs. All variants share the evaluation cohort and counting convention. Attribution of performance differences to the input representation requires control of network capacity and tuning effort.

The reference computing platform comprises an AMD EPYC 7551P processor with 32 cores, 125 GB of RAM and a TITAN X GPU with 12 GB of memory. These specifications describe the reported execution environment. They do not establish training duration, inference throughput or memory consumption. Reducing the input dimension from 51 variables to 24 components constitutes a 52.94% reduction in input dimensionality; computational savings would require direct measurement.

5.5.3 Detection Results

Table 5.6 presents the measured performance of the five input configurations. Each row uses the same class totals: 51,453 anomalous timestamps and 342,226 normal timestamps. The measures follow the definitions in Section 2.6.2, with an anomalous timestamp as the positive class. All table entries are calculated from integer confusion counts and rounded to two decimal places.

Table 5.6. Measured timestamp-level performance of the input representations. Each timestamp is counted once and no point adjustment is applied.

Input representation	Precision (%)	Recall (%)	F_1 (%)	False-positive rate (%)
All 51 variables	70.22	94.34	80.51	6.02
PCA: 5 components	75.20	86.54	80.47	4.29
PCA: 24 components	73.11	91.89	81.43	5.08
Autoencoder representation	72.05	86.32	78.54	5.03
Selected feature subset	70.91	84.10	76.94	5.19

Table 5.7. Timestamp-level confusion matrix for the final 24-component PCA configuration. Counts refer to individual timestamps after aggregation of overlapping window outputs.

Actual class	Predicted anomalous	Predicted normal	Total
Anomalous	47,279 (TP)	4,174 (FN)	51,453
Normal	17,391 (FP)	324,835 (TN)	342,226
Total	64,670	329,009	393,679

The complete representation has the highest recall, at 94.34%, together with the highest false-positive rate, at 6.02%. This profile represents a detector that identifies a larger proportion of anomalous observations while also flagging more normal observations. The five-component PCA configuration has the highest precision, at 75.20%, and the lowest false-positive rate, at 4.29%, accompanied by lower recall of 86.54%. In these results, stronger compression therefore corresponds to fewer false-positive timestamps and more missed anomalous timestamps.

The 24-component PCA configuration provides an intermediate balance, with precision of 73.11%, recall of 91.89% and an F_1 score of 81.43%. Relative to the complete representation, its precision is 2.89 percentage points higher and its recall is 2.45 percentage points lower. Its F_1 score is approximately 0.92 percentage points higher. This configuration has the highest reported F_1 and is used for the detailed analysis below. The numerical difference does not establish a statistically reliable advantage or a preference under every operational cost model.

The autoencoder representation has a precision of 72.05% and recall of 86.32%, corresponding to an F_1 score of 78.54%. The selected feature subset has precision of 70.91%, recall of 84.10% and F_1 of 76.94%. These values show lower detection coverage for the two configurations. They do not identify a causal mechanism: loss of relevant input information, differences in representation learning and changes in model capacity would require separate investigation.

The final 24-component PCA evaluation produced the confusion matrix in Table 5.7: 47,279 of the 51,453 anomalous timestamps are detected and 17,391 of the 342,226 normal timestamps are flagged as anomalous.

From these counts, precision is $47,279/64,670 = 73.11\%$ and recall is $47,279/51,453 = 91.89\%$. F_1 equals $94,558/116,123 = 81.43\%$, the false-positive rate $17,391/342,226 = 5.08\%$, and accuracy $372,114/393,679 = 94.52\%$.

The recall of 91.89% means that the detector flags approximately 92 of every 100 anomalous timestamps. It does not mean that 92% of attack episodes are detected. Long attacks contribute more timestamps than short attacks, so the same timestamp-level recall could arise from different distributions of detection across attack episodes. Precision of 73.11% means that approximately 27% of flagged timestamps fall within recorded normal periods. This proportion differs from the false-positive rate of 5.08%, whose denominator includes all normal timestamps.

Accuracy is interpreted alongside the class distribution. Since normal observations account for 86.93% of the cohort, a rule assigning every observation to the normal class would achieve 86.93% accuracy while detecting no anomalous timestamps. The measured accuracy of 94.52% therefore provides supplementary information, whereas recall, precision and F_1 directly describe the balance relevant to identifying the anomalous class.

No point adjustment is applied. A timestamp inside an attack interval counts as a true positive only if that timestamp is predicted anomalous. Detecting one observation does not automatically convert all other observations in the interval into true positives. Contiguous positive predictions may be grouped into alarm intervals for display or operational analysis, but grouping does not retrospectively alter the timestamp-level classifications used in the matrix.

5.5.4 Interpretation and Evaluation Scope

The 24-component PCA configuration combines the highest measured F_1 of 81.43% with a reduction from 51 input variables to 24 components. It detects 91.89% of anomalous timestamps at a false-positive rate of 5.08%. Within this SWaT evaluation, these results support PCA as a compact representation with a useful balance between detection coverage and false alarms.

The full-input and five-component PCA configurations have similar F_1 scores but different error profiles. The full input detects a greater proportion of anomalous observations, while five-component PCA flags fewer normal observations. Representation choice therefore offers a practical way to address different monitoring priorities: broader detection coverage when missed anomalies carry greater cost, or fewer false alarms when unnecessary investigation is more costly.

For RQ3, the study illustrates how the modular workflow supports comparison of process-window representations on a common evaluation cohort. The resulting error profiles provide a basis for choosing a configuration according to the intended monitoring objective. This complements the S7-300 network evaluation with an application to physical process measurements.

5.6 Limitations, Validity, and Individual Contribution

Scope of the industrial evaluation. The studies provide evidence from one S7-300 installation and one SWaT campaign. The S7-300 evaluation separates capture sessions, while SWaT uses later observations for testing. Each S7-300 attack class contributes one test episode, whose records are correlated; the separate benign monitoring day provides additional evidence on false alarms. Evaluation across further installations and operating modes would test transfer.

Operational assessment. The SWaT errors total about 4.83 hours of flagged normal observations and 69.6 minutes of missed anomalous observations. Assessment of individual attacks would match predicted intervals to attack episodes under a stated overlap rule and report detection delay alongside missed episodes. Grouping false alarms and reporting their rate per day of normal operation would clarify operator workload. Deployment precision also depends on the proportion of anomalous observations.

Further validation. The measured comparisons concern configurations within the proposed workflow. Comparisons with simple baselines would test their added value, as motivated by prior industrial detection studies [110, 168]. Repeated training runs would quantify variability, with uncertainty estimates that account for temporal dependence. Controlled ablations could isolate the contributions of reconstruction and critic scores. Complete experimental settings and runtime measurements would support reproduction and deployment planning.

Individual contribution. The candidate designed and implemented NADICS, its model interface and protocol extractor, and produced the labelled S7-300 corpus. For SWaT, he developed the multivariate GAN pipeline and preprocessing variants, conducted and analysed the experiments, and wrote the internal report. His service-graph contribution covered the concept and measurement design. This account rests on his attestation and the surviving artefacts; the IUNO deliverables record authorship at document level [25, 35].

5.7 Chapter Summary

RQ3 asks how a modular framework can analyse network and process features for anomaly detection in industrial systems. NADICS provides a researcher-guided workflow for adapting data preparation and comparing candidate learners through fixed interfaces. Its modular design supports the selection of a configuration for a given dataset. On the S7-300 corpus recorded inside IUNO the framework reaches 94.50% attack F_1 and 89.56% macro- F_1 over six classes at a record false-positive rate of 0.484%, and a benign day with four active PLCs raised nine alarm episodes, all of them false. For SWaT process windows, the GAN evaluation reports 91.89% timestamp recall at a false-positive rate of 5.08% with 24 PCA components. The full input gives the highest recall and five components the lowest false-positive rate, so the preferred representation depends on the relative cost of missed anomalies and false alarms.

The framework and its two industrial applications address the representation and modular-analysis aspects of gap G3. Their results remain specific to the evaluated installation or SWaT

campaign; transfer to other installations and robustness against adaptive attacks remain untested. Chapters 6 and 7 examine requirements for broader evaluation.

Evaluating Adversarial Robustness: Benchmark Design and Metric Analysis

6.1 Research Problem and Contribution

This chapter addresses RQ4: How can an explicit adversary model be translated into a reproducible benchmark for adversarial attacks and defences, and what does such a benchmark reveal about the relation between clean predictive performance and adversarial robustness? The question concerns gap G4, which links adversary assumptions to the validity of the resulting measurements.

Adversarial machine learning studies how deliberate changes to training data or inference inputs can cause a learning system to fail, together with methods for resisting such changes [111]. A poisoning attack intervenes during training, whereas an evasion attack modifies an input presented to an already trained model. Defences may expose the model to adversarial examples during training, transform its inputs or internal representations, or detect and respond to suspected adversarial inputs. This chapter concerns evasion attacks and the evaluation of these three defence strategies. The broader attack taxonomy and terminology are defined in Section 2.5.

An empirical robustness estimate depends on the permitted perturbations and the attacks used to search for them [112, 14]. An unsuccessful search may reflect resistance to the tested attack, or a failure of the attack to exploit the model. For example, Athalye et al. identified obstructed gradients in seven of nine non-certified defences accepted at ICLR 2018 and circumvented six completely and one partially [13]. Evaluators therefore need to examine the measurement procedure alongside the defended classifier.

The SPARTA SAFAIR work produced two complementary evaluation mechanisms: an adversarial machine-learning competition and a reusable benchmark framework [37, 32]. The competition evaluated submitted defences on hidden test samples. The benchmark provided common interfaces for evaluating trained models against reference attack implementations, with task-specific components for different prediction problems. Its versioned source code is publicly available on GitHub [30].

Using the project records, this chapter reconstructs both evaluation mechanisms and assesses whether the reported measurements are consistent with the documented attack objectives, attacker access and perturbation constraints. The empirical evidence comprises the contest submissions and the application of the benchmark to partner defences on histopathology

images. The chapter then derives properties of the accuracy-loss score and the binary detector response policy. These analytical results establish what the published aggregates support and what additional observations would be required to distinguish competing explanations. A short comparison with the project's PDF-malware study examines how feature constraints alter the evaluation problem.

SPARTA was funded under Horizon 2020 grant agreement No. 830892. Its seventh work package, SAFAIR, concerned the security of artificial-intelligence systems. The candidate wrote the whole of D7.6, which reports the contest and benchmark, and Chapter 5 of D7.5 [32, 33]. Section 6.10 distinguishes this contribution from the partner methods and experiments discussed below.

6.2 Threat Analysis and Evaluation Scope

SPARTA's threat analysis links attack techniques to affected ML assets and the adversary's knowledge [38, Sections 3.3.2–3.3.6]. Applied to the evaluations in this chapter, these distinctions locate the attack within the ML life cycle. Poisoning concerns training data or the learning process; evasion concerns inputs supplied to a trained model.

The evaluations examined here concern evasion. The contest specified attacks that induce a chosen identity or errors in selected facial attributes [37, Section 3.3]. The histopathology benchmark reports classification accuracy on clean and perturbed images [32, Section 3.1.3]. The affected asset is therefore the inference input, and the measured outcome is prediction correctness under attack. Sections 6.3 and 6.4 specify attacker access and permitted perturbations, which determine the conditions under which these measurements can be interpreted.

6.3 Evaluation Requirements and Threat Models

The threat model specifies the attack objective, the information available to the attacker and the operations permitted on the input. Section 2.5 defines the white-box, black-box and grey-box terminology used throughout the dissertation. Under its grey-box convention, some model information is known and the training process cannot be influenced. Knowledge of training data and the ability to alter those data are separate assumptions.

The contest granted knowledge of the training corpus, withheld the defence technique and model architecture, and allowed gradient-based updates against the deployed model [37, 29]. Access to gradients is an oracle capability. The term grey box alone therefore leaves the experimental interface underspecified. The conditions stated in Section 6.4, including the query interface and input budget, define the contest adversary used here. This is an academic evaluation setting; a survey of 271 practitioners found that academic knowledge assumptions often grant more information than attackers have in deployment [115].

D7.3 adopted the evaluation checklist of Carlini et al. It called for attacks adapted to the complete defence, explicit attack hyperparameters and comparisons with transfer or gradient-

free attacks. It also requested perturbation-budget curves and ablations [37, 14]. These requirements provide a basis for assessing the delivered results. Participation by independent attack teams can broaden the search, but the scoring rule and the validity of each generated input require separate checks.

6.4 Tasks and Evaluation Protocol

6.4.1 Prediction Tasks and Permitted Perturbations

The contest defined two tasks, each with an attack track and a defence track: targeted face re-identification and attribute alteration. In targeted face re-identification, the attacker modifies a face image so that the classifier assigns the image to a specified target identity. In attribute alteration, the attacker modifies an image so that the classifier misclassifies a nominated subset of its forty binary facial attributes. The corresponding defence tracks evaluated model robustness against the same perturbations [37]. A participant could attack one task and defend the other, but could not enter both sides of the same task.

Let f_θ denote the model scores and h the resulting class decision. For an image x in a specified input coordinate system, define the feasible perturbations by

$$\mathcal{S}_\varepsilon(x) = \{x' \in \mathcal{X}_{\text{valid}} : \|x' - x\|_\infty \leq \varepsilon\}. \quad (6.1)$$

Here $\mathcal{X}_{\text{valid}}$ includes the permitted pixel range and quantisation. For pixel values scaled to $[0, 1]$, the contest budget was $\varepsilon = 8/255$, corresponding to a maximum change of eight intensity levels per channel in the discrete 0–255 encoding. The rules required discrete output images, deterministic and stateless submitted models, and an upper limit of four hours for generation of 1,000 perturbed examples [32]. The report identifies two 12 GB NVIDIA GPUs, a Titan X Pascal and a GeForce GTX, with a 32-core AMD CPU and 128 GB RAM. Container execution used Ubuntu 18.04 and CUDA 10.1. These are reported resource limits, not measured per-example runtimes.

The budget must be interpreted before any model-specific normalisation. For example, a linear rescaling from $[0, 1]$ to $[-1, 1]$ doubles the numerical distance. A reproducible evaluation therefore records both the input coordinate system and preprocessing.

6.4.2 SAFAIR Contest Scoring and Evaluation Protocol

The SAFAIR contest ranked submissions by the change between clean and perturbed accuracy,

$$\Delta = A_{\text{initial}} - A_{\text{final}}. \quad (6.2)$$

Attacks sought the largest Δ and defences the smallest. Unless stated otherwise, accuracies and Δ are proportions in this chapter; 100Δ is a difference in percentage points. D7.5 specifies clean accuracy and runtime as tie-breakers [33]. These do not prevent a low-utility model with a smaller Δ from outranking a more accurate model.

Table 6.1. Planned and reported SAFAIR evaluation procedures.

	Plan [37]	Reported delivery [33, 32]
Participants	attack and defence tracks for both tasks	defence submissions only
Attack selection	organiser baselines, then five selected participant attacks	four named organiser baselines
Defence score	weighted opening Δ ; mean final Δ over opponents	aggregate best and worst perturbed accuracies; Δ from the best value
Final comparison	five attacks against five defences	no participant-attack round
Reproduction	hidden evaluation set and submitted containers	sampling code and tool descriptions; incomplete run-level records

The planned opening round evaluated defences against FGSM, BIM and PGD, with respective weights 0.2, 0.4 and 0.4 [37]. FGSM makes one update in the sign of the input-loss gradient [123]. BIM repeats smaller FGSM-style updates and clips each result to the permitted input range and perturbation neighbourhood [169]. PGD also applies iterative gradient updates, projects each result onto the feasible perturbation set, and can use multiple starting points [124]. Carlini–Wagner instead formulates adversarial-example generation as an optimisation problem that balances the required classification outcome against perturbation size [170]. The five selected attacks would then face the five selected defences, with the final score averaged over opponents [37]. No attack submissions arrived. The organisers used fixed baseline attacks instead; D7.5 lists Carlini–Wagner in addition to FGSM, BIM and PGD [33]. Table 6.1 records the change in protocol.

The lack of independent attacks reduced the diversity of the evaluation. It does not explain all weaknesses of the resulting measurement. As derived in Section 6.8.4, a constant classifier has $\Delta = 0$ against any set of label-preserving attacks, including the opponents in the planned final round. The scoring defect and the attack-coverage defect are therefore distinct.

The contest drew inspiration from the NIPS 2017 competition, whose defence score was the proportion of correctly classified attacked images averaged across attacks [28]. That score differs from Equation 6.2. The competitions also differed in task and participation, so their results do not isolate the effect of independent contestants. Details of the schedule are retained in the cited project records; the scientific comparison here concerns the scoring and attack interfaces.

6.5 Data and Evaluation Measures

6.5.1 Corpora and Hidden Evaluation Samples

The contest used CelebA for model training, a corpus described by Liu et al. as approximately 200,000 images of 10,000 identities, with forty binary attributes per image [171]. For hidden attribute evaluation the organisers used LFWA, the forty-attribute annotation of Labeled Faces in the Wild, which contains 13,233 images of 5,749 identities [172, 171]. A separate corpus

Table 6.2. Evaluation units and data provenance. Stated sample sizes describe the printed procedures, not recovered per-example result files.

Evaluation	Input and prediction	Source and sampling
Contest identity	one image; one identity label	augmented CelebA; 1,000 selected test records [32]
Contest attributes	one image; forty binary labels	LFWA; 1,000 selected images [32]
Partner benchmark	one 50×50 histopathology patch; IDC/non-IDC	training, validation and test subsets described; evaluated subset size and patient separation unreported [31]
PDF comparison	21 tag counts; benign/malicious	20,000 files; 45% defender, 45% proxy, 10% crafting [33]

reduces direct image reuse, but does not establish equality of image distributions or attribute prevalences.

Attribute imbalance can give a constant predictor high mean accuracy, as reported for SD [32]. Attribute prevalences and per-attribute outcomes are therefore needed alongside the aggregate score; the available results do not establish demographic performance differences.

Re-identification required identity labels compatible with the trained classifier. The organisers therefore used CelebA images under a random choice of colour jitter, greyscale conversion, horizontal reflection or rotation of at most five degrees [32]. The printed procedure applies a face-detection check and retries failed transformations before excluding an image. The resulting selection may differ from an unfiltered sample. The report recognises that participants could have used similar augmentations during training. Augmentation alone does not demonstrate independence from training images.

6.5.2 Accuracy and Task-Specific Attack Success

For N examples with identity labels y_i , the decision is $h(x_i) = \arg \max_k f_{\theta,k}(x_i)$. Attribute decisions are $h_j(x_i) = \mathbf{1}[f_{\theta,j}(x_i) \geq 0]$ when the outputs are logits, with equivalent threshold 0.5 for sigmoid probabilities. The corresponding accuracies are

$$A_{\text{id}} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[h(x_i) = y_i], \quad A_{\text{attr}} = \frac{1}{40N} \sum_{i=1}^N \sum_{j=1}^{40} \mathbf{1}[h_j(x_i) = y_{ij}]. \quad (6.3)$$

These conventions follow the task-specific scoring code printed in D7.6. Attribute accuracy counts individual attribute decisions; it does not require all forty predictions for an image to be correct [32].

Accuracy loss does not identify success against a specified target. For identity targets $t_i \neq y_i$, let $I_t = \{i : h(x_i) = y_i, t_i \neq y_i\}$ contain eligible, initially correct examples. Given valid attacked inputs x'_i , conditional targeted success is

$$\text{ASR}_{\text{target}} = \frac{1}{|I_t|} \sum_{i \in I_t} \mathbf{1}[h(x'_i) = t_i]. \quad (6.4)$$

A prediction of a third identity reduces accuracy without satisfying the targeted objective. An empty eligible set makes this rate undefined and its size must be reported.

For a nonempty nominated attribute subset J_i , define $I_J = \{i : h_j(x_i) = y_{ij} \text{ for all } j \in J_i\}$. One explicit success criterion is

$$\text{ASR}_{\text{subset}} = \frac{1}{|I_J|} \sum_{i \in I_J} \prod_{j \in J_i} \mathbf{1}[h_j(x'_i) \neq y_{ij}]. \quad (6.5)$$

This requires errors on every nominated attribute of an initially correct subset. Reporting $|I_J|$ separates induced errors from those already present; the rate is undefined if $|I_J| = 0$. Equations 6.4 and 6.5 specify measurements needed to evaluate the stated objectives; they are not additional scores recovered from the contest. If an attack flips all k initially correct bits in the nominated subset and leaves all other decisions unchanged, the per-image contribution to attribute accuracy falls by only $k/40$. For $k = 1$ this is 2.5 percentage points, although the specified attack succeeds.

6.5.3 Aggregation over Attacks

An attack suite can fail on different examples under each method. Selecting the lowest aggregate accuracy does not account for that complementarity. Following per-example evaluation guidance [14], let $\mathcal{A}_\varepsilon$ be a finite set of attacks whose outputs satisfy Equation 6.1. For a single-label task, define

$$\hat{A}_{\mathcal{A}}(\varepsilon) = \frac{1}{N} \sum_{i=1}^N \min_{a \in \mathcal{A}_\varepsilon \cup \{\text{id}\}} \mathbf{1}[h(a(x_i)) = y_i]. \quad (6.6)$$

The identity transformation includes clean errors in the score. The suite must respect the same threat model for each defence. Because it searches only a subset of valid perturbations, its surviving accuracy is an upper bound on worst-case accuracy on these examples, not a certificate of robustness. Moreover,

$$\hat{A}_{\mathcal{A}}(\varepsilon) \leq \min_{a \in \mathcal{A}_\varepsilon \cup \{\text{id}\}} \frac{1}{N} \sum_i \mathbf{1}[h(a(x_i)) = y_i], \quad (6.7)$$

since each per-example minimum is no greater than its value for any fixed attack. The aggregate tables do not identify which examples fail under which attack, so this suite score cannot be calculated retrospectively from them.

6.6 Benchmark Architecture and Reproducibility

The benchmark implements a common software procedure for adversarial evaluation. It separates attack execution from the task-specific model and data interfaces. D7.3 proposed five modules, which D7.6 reports as implemented: attacks, datasets and models, together with use cases and a wrapper [37, 32]. Figure 6.1 redraws the published architecture.

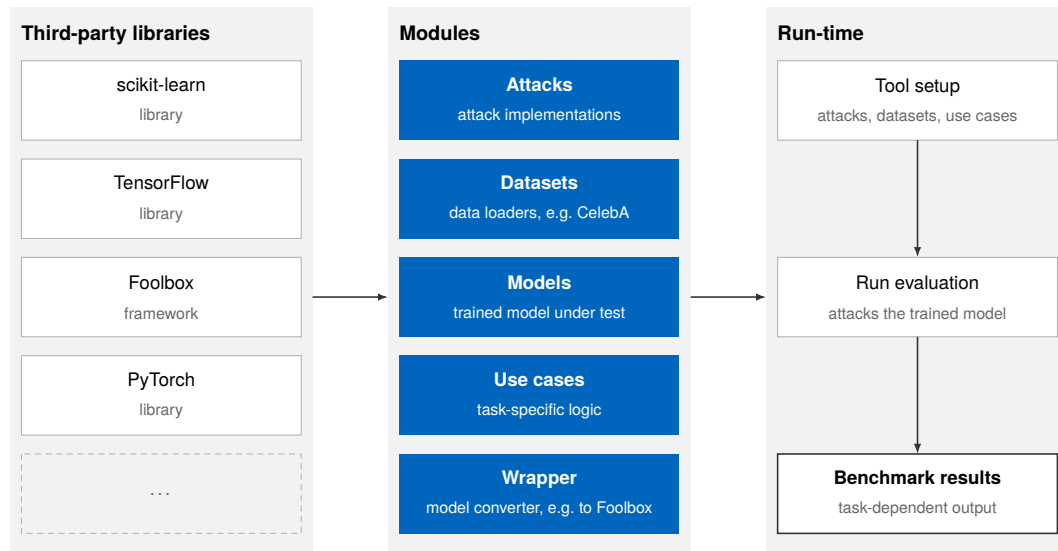


Figure 6.1. Benchmark architecture, redrawn after D7.6 Figure 2, p. 22 [32]. Box labels follow the original. The five modules separate task definitions and data loading from attack execution.

Foolbox supplies reference attack implementations [27]. The SAFAIR framework integrates them into a shared evaluation procedure with task-specific adapters. Identity classification requires a single-label loss, whereas facial attributes require a multi-label loss. The implementation selects cross-entropy or binary cross-entropy for these tasks and passes the selected loss to supported attacks [30]. Researchers can thus change the prediction task while retaining the evaluation loop.

A run loads trained model weights and evaluation data, converts the model to the attack-library interface, and records clean and attacked accuracies in JSON format. New attack classes extend a base class and are discovered by the loader; a configuration list excludes named attacks. PyTorch is the native model interface, and D7.6 describes conversion of TensorFlow or Keras models [32, 30]. The public benchmark source is separate from the contest development toolkit [39].

Reuse requires compatible prediction criteria and data processing. The multi-label adapter handles fixed-budget attacks but omits some minimum-distance interfaces. Vicomtech’s histopathology evaluation in Section 6.8.3 documents use on a different dataset and prediction task [32]. Standardisation here concerns the component interfaces and execution procedure.

Perturbation validation. The public revision identified by commit 2fac62b947ed pins Foolbox 3.3.1. Its attack call returns a raw candidate and a budget-clipped candidate, with a success indicator. However, lines 134–135 of `main.py` pass the raw candidate to the accuracy function [30]. In Foolbox’s minimisation interface the clipped candidate is obtained by restricting the raw perturbation to the requested distance budget; the raw output need not satisfy that budget [173].

A further distinction concerns targeted runs. The same implementation replaces the reference labels with the nominated target before computing its fields labelled accuracy. Those fields

then measure target-label agreement, with a different interpretation from true-label accuracy [30]. Each output therefore needs an explicit task, reference label and denominator.

6.7 Defence Mechanisms and Integration

Vicomtech used the benchmark described in Section 6.6 to evaluate five defence configurations documented in D7.2 [31, 32]. They instantiate the three defence strategies introduced in Section 6.1: adversarial training, transformations of inputs or internal representations, and adversarial-input detection.

The underlying histopathology classifier combines a pretrained VGG16 feature extractor with a dense prediction network [31, 32]. Write its scores as $f_\theta(x) = c_\psi(\phi_\omega(x))$, where ϕ_ω is the convolutional feature extractor and c_ψ the prediction head. The following notation makes the intervention of each defence explicit without supplying missing layer configurations.

6.7.1 Training and Representation Transformations

The adversarial-training configuration retrains the classifier with clean and perturbed examples. D7.6 identifies FGSM, BIM and PGD as the attacks used to generate its adversarial training examples [32]. This method exposes the model to those attacks during training; resistance to other attacks remains an empirical question.

For the dimensionality-reduction variants, let r_η denote an autoencoder. The two configurations have score functions

$$f_{\text{top}}(x) = c_\psi(\phi_\omega(r_{\eta_x}(x))), \quad f_{\text{mid}}(x) = c_\psi(r_{\eta_z}(\phi_\omega(x))). \quad (6.8)$$

The first transforms pixels and the second transforms extracted features. D7.2 states that the original convolutional network and prediction head retain their weights in these autoencoder variants. Its separate encoder-only variant trains a new prediction head, but that variant is not a column of the D7.6 benchmark table [31]. Figure 6.2 maps the initial and middle autoencoder designs of D7.2 to the Top AE and Middle AE configurations compared in Table 6.4.

A smaller reconstruction error alone does not establish robustness of either composition. The relevant attack must optimise against the complete transformed classifier, including its preprocessing. Reusing examples generated against the original classifier tests a different condition from attacking the defended composition [14].

6.7.2 History and Activation Detectors

Figure 6.3 shows the two partner detectors and their response paths.

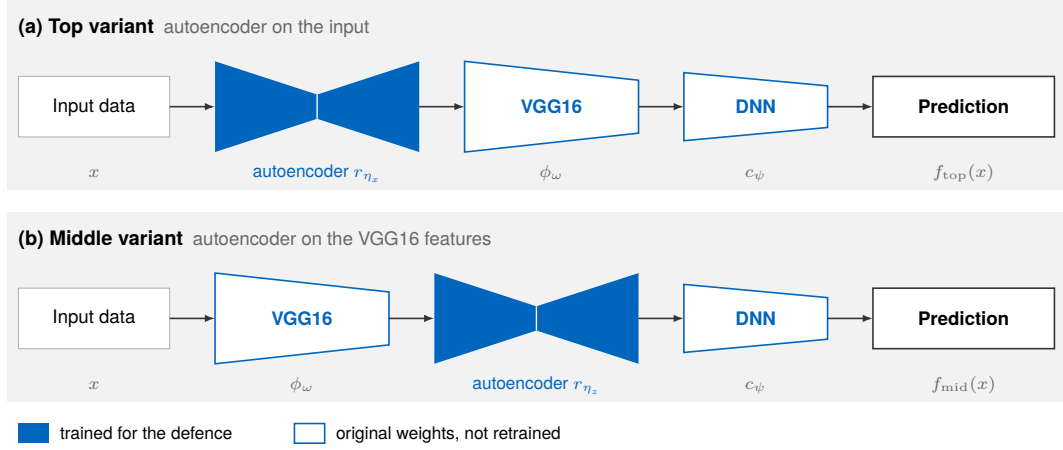


Figure 6.2. Autoencoder placement in the partner classifier, redrawn after D7.2 Figures 13 and 11, p. 14 [31]. In the top variant (a), the autoencoder r_{η_x} transforms the input before feature extraction. In the middle variant (b), r_{η_z} acts between the feature extractor ϕ_ω and the prediction head c_ψ , as in Equation 6.8.

Prediction similarity adds a history of queries and their predictions. The partner study records user information with image similarity and alarm statistics, and reports SSIM as the similarity measure [31]. Its alarm can be written $d_t = d(x_t, H_{t-1})$, where H_{t-1} contains the preceding queries associated with the monitored interaction. A sequence of related optimisation queries may then trigger an alarm. A transferred adversarial example submitted once presents a different observation to this stateful detector.

The activation detector instead characterises the response of the underlying classifier to an input. For each of nineteen hidden layers it calculates nine statistics: mean and median; skewness and kurtosis; standard deviation and interquartile range; minimum and maximum; and the number of nonzero activations [31]. With layer activations $a_\ell(x)$ and the statistic vector s , the representation is

$$z(x) = \text{concat}_{\ell=1}^{19} s(a_\ell(x)) \in \mathbb{R}^{171}. \quad (6.9)$$

After standardisation and feature selection, two support-vector classifiers are trained, one for each class predicted by the original model. The selected detector therefore depends on that prediction. The 94% detector accuracy reported in D7.2 concerns detection of adversarial inputs; it is not the defended task accuracy of the D7.6 table.

6.7.3 From an Alarm to a Classification

To place the detectors in an accuracy table, D7.6 uses the binary response policy of returning the opposite class whenever the detector raises an alarm [32]. Let $h(x) \in \{0, 1\}$ denote the base prediction, $d(x) \in \{0, 1\}$ the alarm and $g(x)$ the returned class. For a fixed history, where applicable,

$$g(x) = h(x) \oplus d(x), \quad (6.10)$$

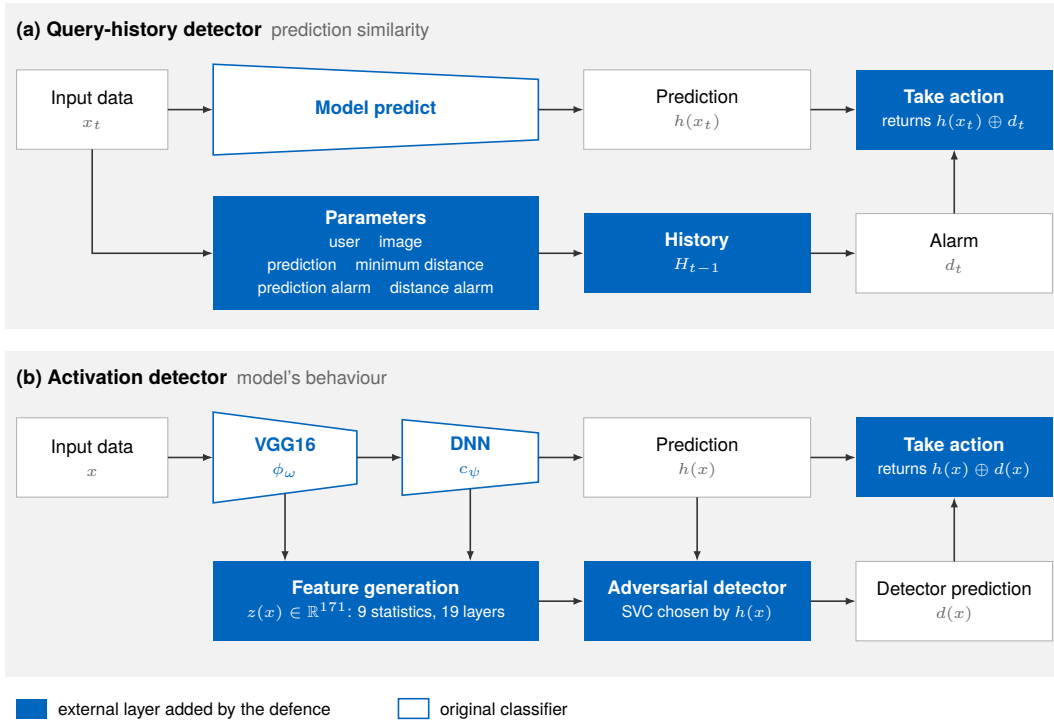


Figure 6.3. Partner detection mechanisms, redrawn after D7.2 Figures 15 and 16, p. 16 [31]. Panel (a) uses preceding queries, whereas panel (b) uses internal activations of the classifier. The recorded parameters, the activation features of Equation 6.9 and the per-class detector follow D7.2. The response is the label reversal of Equation 6.10, which D7.6 applies in the benchmark [32]. The final action is part of the defended system and must be included when task accuracy is measured.

Table 6.3. Reported contest scores, including the worst-perturbed column, from D7.6 Table 1, p. 16 [32]. Task names follow that table. The SD task label conflicts with its detailed attribute-model description; best and worst values are not identified as paired outcomes for one checkpoint under a common attack suite.

Rank	Team	Task	Clean	Best	Worst	Δ
1	SD	re-identification	0.87	0.85	0.67	0.02
2	Vicomtech	re-identification	0.85	0.72	0.72	0.13
	Vicomtech	attribute alteration	0.80	0.65	0.65	0.15
3	BPI	re-identification	0.84	0.68	0.68	0.16

where $\oplus$ denotes exclusive OR. This rule corrects an erroneous binary prediction when the detector raises an alarm, but turns a correct prediction into an error under the same alarm. Consequently, on a common labelled evaluation set,

$$A(g) - A(h) = \Pr(d = 1, h \neq y) - \Pr(d = 1, h = y). \quad (6.11)$$

The probabilities denote empirical fractions on that set. To obtain the identity, partition the observations by the alarm and the correctness of h : unflagged predictions are unchanged, flagged errors are corrected, and flagged correct predictions are reversed. If $d(x) = 1$ for every observation, then $A(g) = 1 - A(h)$, while g can still depend on the input.

Detection accuracy alone does not determine either term in Equation 6.11. A useful evaluation records alarms jointly with base and returned predictions, and measures false alarms on clean inputs. Abstention would be a different response policy, for which coverage and error among accepted predictions must be reported [14]. The historical benchmark reports the reversal policy; its aggregate accuracies do not identify the two joint fractions.

6.8 Results and Metric Analysis

6.8.1 Contest Evidence

Table 6.3 preserves the numerical columns of D7.6 Table 1. The source reports one row per ranked team and task, rather than one row per submitted checkpoint. Its surrounding text describes best and worst scores across submissions, but does not link each value to an attack and checkpoint [32].

The SD row reports 0.85 as best perturbed accuracy and 0.67 as worst, a spread of eighteen percentage points. The printed $\Delta = 0.02$ uses the best value. Subtracting 0.67 from 0.87 would give 0.20, but it would not establish a worst-case loss for the same model without the missing checkpoint association. The report’s prose also substitutes 87% for attacked accuracy and gives 61% and 88% values that do not match its table. The numerical record used here is the table, with those inconsistencies retained as limits on interpretation.

6.8.2 SD Contest Submissions

The contest permitted each participant up to three submissions, and D7.6 describes the three models submitted by SD [37, 32]. Although the report introduces them as re-identification models, their technical descriptions specify forty attribute outputs. The analysis therefore follows the attribute-model description; their mapping to the official SD row in Table 6.3 remains unresolved.

The first submission used a convolutional network trained on PGD examples with $\varepsilon = 0.3$, approximately 9.6 times the contest's 8/255 budget if both use $[0, 1]$ pixels. The team reports that training produced the same output for every input, with approximately 80% clean attribute accuracy and zero accuracy loss. It attributes that accuracy to the prevalence of absent attributes and recognises a weakness in the dataset and contest rules. A constant attribute vector cannot be interpreted as a demonstrated identity-classification result, and neither the checkpoint nor individual predictions accompany the reported collapse.

The second submission combines six EfficientNet and three pruned ECA-ResNet models. Their averaged outputs are thresholded to obscure gradients. For a nonzero scalar attribute logit u , write the transformation as

$$q(u) = \begin{cases} 10, & u > 0, \\ -10, & u < 0. \end{cases} \quad \mathbf{1}[q(u) \geq 0] = \mathbf{1}[u \geq 0], \quad q'(u) = 0 \ (u \neq 0). \quad (6.12)$$

For $u \neq 0$, thresholding preserves the binary decision while suppressing its direct derivative. Adaptive or transfer attacks are therefore needed to assess this construction [13, 15].

The second submission achieved approximately 92% clean accuracy and 79% under transferred attacks in the team's development account. The third submission used one pruned ECA-ResNet, added adversarial training, and retained the threshold operation. Its reported clean accuracy was approximately 87%, with 84–85% under transferred attacks [32]. Figure 6.4 compares the reported values for these two submissions.

The third submission trades five percentage points of reported clean accuracy for five to six points of transferred-attack accuracy relative to the second submission. This is a descriptive comparison between systems: ensemble size and training changed together, and the attacks' individual settings are not given. It does not isolate the effect of adversarial training.

6.8.3 Histopathology Benchmark

D7.6 reports that Vicomtech used the benchmark tool to evaluate five defence configurations on a histopathology classification task [32, Section 3.1.3]. This application documents use of the framework for a prediction problem distinct from the competition's facial image tasks. Table 6.4 reproduces the reported accuracies for binary IDC classification. For the two detectors, these include the response policy in Equation 6.10. The clean row records the prediction cost associated with each configuration.

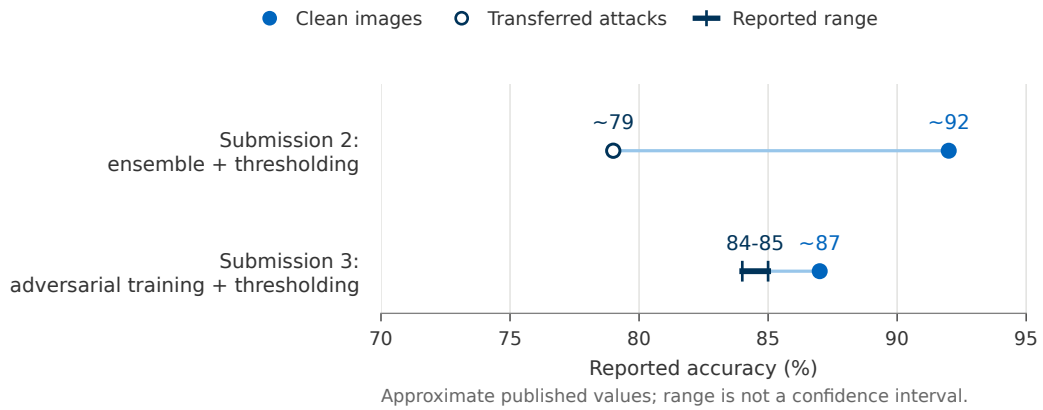


Figure 6.4. Reported clean and transferred-attack accuracies for SD submissions 2 and 3, from D7.6 Sections 2.8.1.2–2.8.1.3, pp. 18–19 [32]. Submission 1 is omitted because D7.6 reports no comparable transferred-attack accuracy for its collapsed model. The 84–85% interval is an approximate reported range, not a confidence interval; these development values are separate from the official contest scores.

Table 6.4. Published accuracies for the five partner defence configurations, from D7.6 Table 2, pp. 28–29 [32]. Attack hyperparameters and the evaluated subset are unspecified. Each entry is a reported aggregate, with no dispersion estimate.

Attack	Adv. training	Top AE	Middle AE	Pred. similarity	Activations
none (clean)	0.844	0.726	0.817	0.844	0.708
FGSM	0.837	0.716	0.812	0.837	0.708
L_2 basic iterative	0.844	0.725	0.817	0.842	0.708
BIM	0.837	0.716	0.812	0.837	0.708
DeepFool	0.003	0	0.001	0.015	0.708
additive noise	0.844	0.725	0.817	0.844	0.708
NewtonFool	0.464	0	0	0.460	0.708
PGD	0.837	0.717	0.813	0.837	0.708

DeepFool iteratively linearises the classifier around the current input and moves towards the closest estimated decision boundary [174]. NewtonFool applies gradient steps that reduce the original-class softmax probability while bounding the perturbation at each iteration [175].

The middle autoencoder exceeds the top autoencoder by 9.1 percentage points on clean data and 9.6 points under the reported PGD condition. Its PGD loss is 0.4 points, compared with 0.9 for the top variant. These values favour the middle configuration within the reported evaluation. They do not establish that placement alone caused the difference, because the complete training configurations and repeated-run outcomes are unavailable.

Adversarial training and prediction similarity have equal reported clean accuracy and equal FGSM, BIM and PGD values. Prediction similarity is higher under DeepFool by 1.2 percentage points, but lower under NewtonFool by 0.4 points. It therefore does not dominate adversarial training across the table. The source’s explanation that similarity detects gradient-based construction is insufficient to explain the results, since DeepFool and NewtonFool also use gradient information. Query sequences and detector alarms would be needed to test that mechanism.

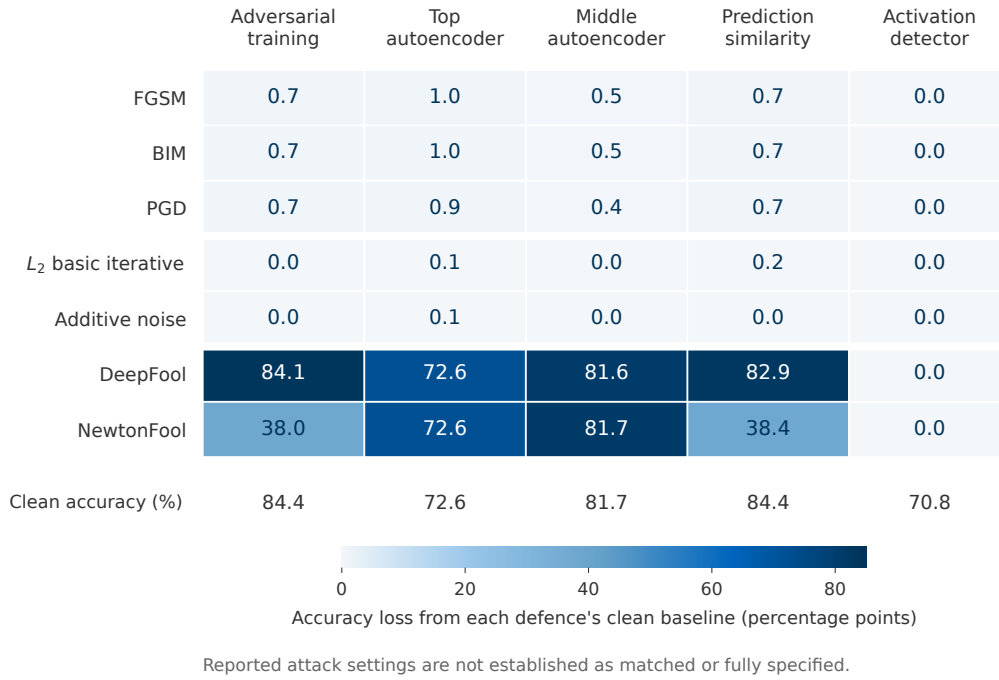


Figure 6.5. Accuracy losses computed from D7.6 Table 2 [32]. Each cell is $100(A_{\text{clean}} - A_{\text{attack}})$ in percentage points; the original accuracies remain in Table 6.4. The separation of attack rows does not imply matched budgets. In particular, zero loss for the activation detector describes its aggregate score, not a demonstrated mechanism or robustness certificate.

The near-zero DeepFool results for four configurations require separate interpretation. DeepFool searches for an adversarial input with low distortion and should be assessed by the distortion required, or by success under a common valid budget [14]. Neither the norm and stopping settings nor achieved distortions are reported for these benchmark runs. The table therefore cannot establish that its attack rows represent equally constrained adversaries. The code finding in Section 6.6 reinforces the need to validate distances, but does not resolve the historical configuration. The additive-noise row is a useful control condition; without its distribution, budget and number of trials, its small changes do not establish a comprehensive random-search check.

6.8.4 Sensitivity to the Scoring Rule

Figure 6.5 subtracts each attacked accuracy from that configuration's clean accuracy. It makes the score dependence visible without changing the original results. Under PGD, the activation detector has the smallest Δ , zero, but its attacked accuracy is 12.9 percentage points below adversarial training. Similarly, the middle autoencoder has a smaller loss than adversarial training while its attacked accuracy is 2.4 points lower. A smaller loss alone therefore does not identify the more useful defended classifier.

For paired clean and attacked examples, let C denote an initially correct prediction and C' a correct prediction after attack. Expanding the two accuracy terms yields

$$\Delta = \Pr(C \cap \neg C') - \Pr(\neg C \cap C'). \quad (6.13)$$

Both accuracies contain the correctly classified observations in $C \cap C'$, which cancel. Thus, Δ measures the net change in correctness. It is not the conditional fraction of initially correct examples made incorrect. Equal numbers of harmful and corrective changes can leave $\Delta = 0$ despite changes in individual outcomes. The aggregate table cannot determine those two counts.

Constant-prediction result. For any constant classifier $h_c(x) = c$ and label-preserving attack a , $h_c(a(x_i)) = h_c(x_i)$ for each observation. Therefore,

$$A_{\text{clean}}(h_c) = A_{\text{attack}}(h_c) = \hat{p}_c, \quad \Delta(h_c) = 0, \quad (6.14)$$

where $\hat{p}_c$ is the evaluated class fraction. The statement holds for each attack and any weighted average over attacks. Independent opponents cannot eliminate this property of the score. The result establishes an indifference to prediction utility when ranking only by loss; it does not imply that the constant classifier correctly classifies the other classes.

For attributes, choose a fixed vector m using training-set majority labels. Its held-out accuracy is

$$A_{\text{attr}}(m) = \frac{1}{40} \sum_{j=1}^{40} [m_j \hat{p}_j + (1 - m_j)(1 - \hat{p}_j)], \quad \Delta(m) = 0, \quad (6.15)$$

where $\hat{p}_j$ is the prevalence of attribute j in the evaluation set. When the training and evaluation majorities agree, each summand is $\max(\hat{p}_j, 1 - \hat{p}_j)$. This explains how a fixed attribute vector can have high average accuracy on an imbalanced corpus. The baseline labels must be fixed before evaluation; selecting them from hidden test labels would use unavailable information.

These identities support reporting clean utility together with attacked performance and a matched constant baseline. Prediction distributions and paired error transitions provide additional checks. None of these quantities can be reconstructed uniquely from an unchanged accuracy column, so the activation result remains an observation rather than a second demonstration of the SD collapse mechanism.

6.8.5 Comparison with Constrained PDF-Malware Features

D7.5 reports a separate partner study of PDF-malware evasion that illustrates domain-specific input constraints [33]. It uses 10,000 benign and 10,000 malicious Contagio files, with 21 PDFiD tag counts as features. The split assigns 45% to defender training, 45% to proxy training and 10% to adversarial crafting. A neural proxy generates examples transferred to neural, SVM and random-forest classifiers without target queries.

Tag counts must remain integer-valued and cannot decrease. For an original vector $v \in \mathbb{N}_0^{21}$, the reported constraints require

$$v' \in \mathbb{N}_0^{21}, \quad v'_j \geq v_j \quad (j = 1, \dots, 21). \quad (6.16)$$

A perturbation bound applies in the standardised representation, followed by inverse transformation and rounding. These feature constraints do not establish that a corresponding

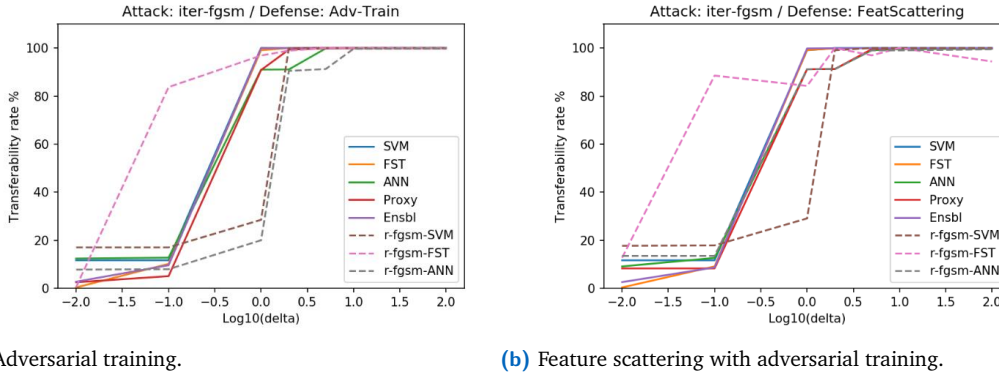


Figure 6.6. Original iterative-FGSM transfer curves from the partner PDF-malware study, D7.5 Figure 15, p. 23 [33]. The horizontal axis is the source's $\log_{10}(\delta)$ perturbation budget, distinct from the accuracy loss Δ . The published feature-space results contain no uncertainty bands.

modified PDF was constructed with its malicious functionality preserved, as required by the problem-space distinction in Section 2.5.

The legends identify the SVM, random forest (FST), neural network (ANN) and ensemble (Ensbl). The prefix r-fgsm denotes adversarial retraining with FGSM examples; Proxy is the attack's source model. The feature-scattering comparison also retrains the SVM and random forest with adversarial examples [33].

Figure 6.6 reports model-dependent transfer success across perturbation budgets. Some curves are non-monotone: they describe the tested searches, not a worst-case envelope over all admissible perturbations. The comparison supports budget-dependent reporting under explicit feature constraints, but not a general ranking of defence families. For Android, the constraints must describe valid application changes and their effect on the detector's representation.

6.9 Threats to Validity

Measurement validity. The contest combines task-dependent accuracies with a net-change score. The identity and attribute objectives require different success criteria, and the targeted software output uses target-label agreement. Aggregate accuracy loss cannot identify induced errors, useful classification or detector alarms. The analytical results in Sections 6.7 and 6.8.4 identify these limits; they do not recover unreported individual outcomes.

Attack coverage and budgets. No independent attack submissions entered the contest. Its fixed gradient-based baselines did not constitute a documented adaptive assessment of every defence. The SD development account includes transfer attacks, but the official table provides no per-attack breakdown. The benchmark additionally mixes attack families without the full configurations needed for comparison. A later evaluation would require valid perturbations in common coordinates and attacks adapted to the complete defended system, with convergence checks [14, 15].

Generalisation and attribution. The four participating teams belonged to SPARTA, so the contest did not test participation beyond the consortium. The evaluated image tasks also differ from Android malware analysis in their inputs and validity constraints. The public PDF-malware study narrows this conceptual difference through constrained count features, but supplies no Android result. Its experiments and the partner defence algorithms retain their attribution. D7.4 is a confidential deliverable; the figures and empirical comparisons in this chapter use public reports.

A focused subsequent study could compare an ordinary classifier with a training-selected constant baseline and the same classifier under output thresholding. For detectors, paired base predictions and alarms would test Equation 6.11. Budget curves and per-example attack aggregation would then measure how the conclusions change under validated perturbations. Those experiments remain separate from the historical evidence presented here.

6.10 Individual Contribution and Project Attribution

The candidate's contribution is stated at section level, following his account of the collaborative project. He wrote D7.1 Sections 2.2.2 and 2.3, and D7.2 Sections 2.1 and 2.2. For D7.2 Sections 2.4 and 2.5 he contributed the concept and definitions. He edited and refined partner text in D7.3, with no authored content claimed for that plan. He wrote Sections 3.1 and 3.2 of D7.4, Chapter 5 of D7.5, and the whole of D7.6 [38, 31, 37, 33, 32]. His D7.4 contribution is recorded as part of this account, without using the confidential report as an empirical source.

Within SAFAIR, the candidate contributed to organising the competition and to developing the reusable benchmark framework. The contest toolkit and benchmark source code are published on GitHub under the candidate's name [39, 30]. The evaluated defensive algorithms remain partner methods. In particular, the histopathology configurations originate in the Vicomtech study of D7.2 Section 2.3.2; the PDF-malware curves belong to the partner study in D7.5 Section 2.8. Authorship of the reporting deliverables does not transfer those experiments or algorithms to the candidate. The mathematical analysis and source comparison in this chapter examine their implications for the evaluation framework.

6.11 Chapter Summary

The SAFAIR work addressed RQ4 through a competition and a reusable benchmark framework. The competition evaluated submitted defences on hidden samples under explicit input constraints. SD's reported constant-prediction submission exposes a weakness of accuracy-loss ranking: a constant predictor can obtain zero loss despite limited predictive utility. Section 6.8.4 proves that this property persists under any label-preserving attack. Logit thresholding raises a separate evaluation problem because it obstructs direct gradients while preserving attribute decisions.

The framework provided a common evaluation procedure with task-specific interfaces. Vi-comtech used it to evaluate histopathology defences, which documents an application outside the contest's facial-image tasks. The resulting table illustrates why the scoring choice matters: a configuration with smaller accuracy loss can have lower attacked accuracy. For detector-based systems, the comparison also depends on the response to an alarm; unchanged aggregate accuracy alone does not identify the underlying mechanism.

Evaluators should therefore report clean and attacked utility together and test attacks against the complete defence. Per-example outcomes and valid perturbation budgets are needed to interpret the scores. The contribution combines a documented evaluation design with an analysis of its measurement properties. The available aggregates do not establish a general relation between clean performance and adversarial robustness, and experiment-level reproduction remains unresolved.

Cross-Contribution Discussion

Chapters 3 to 6 contribute methods for representing security-relevant context and procedures for evaluating learning-based decisions. This discussion relates their design choices to the evidence each study provides. The comparison between Hybroid and HGANN-Mal is conceptual; empirical comparisons remain within the corpus and protocol of each study.

7.1 Contributions of Contextual Representations

Hybroid studies context across evidence sources. Its static graph describes program structure, and its flow features summarise network behaviour observed during execution [5]. The within-system comparison in Table 3.6 reports higher detection F_1 for fusion than for either branch alone. For category classification, fusion improved the ensemble results, while the decision tree favoured static features. The contribution includes a direct measurement of the value of combining these observations for the tested tasks and learners.

HGANN-Mal studies context within the application package. It groups methods by call neighbourhood and semantic similarity; slice-aware hyperedges add relations around security-sensitive call sites [6]. These groups define which methods contribute together to a hyperedge representation. The hypergraph configurations achieved higher mean accuracy than the pairwise baselines for binary detection and for the dataset-specific classification tasks: families on Drebin and categories on CICMalDroid (Tables 4.2 and 4.3). These are comparisons of complete configurations. Comparisons with non-attentive operators vary both construction and weighting; an attention-only effect requires a controlled ablation.

The operator analysis in Section 4.8 gives a separate account of what learned weighting can represent. Different hyperedge groupings can produce different attentive propagation even when they produce the same fixed propagation matrix. This capacity depends on the grouping retained during construction: attention adjusts existing memberships. The distinction connects the model's mathematical properties to the program relations made available by static analysis.

In the industrial work, NADICS represents network connections through protocol features, while its SWaT extension analyses process measurements in multivariate time windows. A common learning interface accommodates both forms, with preprocessing and decision units defined for each application. Chapter 5 connects this modular design to the observations available in the two industrial settings and evaluates each under its own protocol.

The SWaT comparison adds an operating-objective perspective. Full process inputs gave the highest reported recall, while the PCA configurations reduced the false-positive rate

(Table 5.6). The preferred configuration therefore depends on the relative importance of detecting anomalous observations and reducing false-positive observations. Across these studies, the useful question is which context serves the prediction task, with the comparison made under a stated protocol.

7.2 Observation Requirements and Analysis Cost

Static and multimodal analysis place different requirements on data acquisition. Hybroid’s evaluation reused traffic supplied by CICAndMal2017 alongside each application package. Its fused representation therefore requires a recorded execution. HGANN-Mal constructs its representation from the package, so it can be applied without arranging a runtime capture. This separates the effort of obtaining an additional observation from the computational work of modelling static structure.

Chapter 4 also provides a computational account of higher-order representation. The hyperedge cardinality cap bounds retained memberships, and the sparse representation grows linearly with the numbers of methods and slices (Equation 4.19). Construction adds call-graph searches and comparisons between semantic embeddings. The bounds in Section 4.13 distinguish this stage from propagation and make their scaling assumptions explicit. An end-to-end runtime comparison would require measurements under common hardware and workloads.

Observation requirements also define the evaluation cohort. Hybroid compares modalities on the same applications with both representations available, which permits a matched comparison on that cohort. CICMalDroid supplies applications retained by its authors’ dynamic-analysis procedure [160]; HGANN-Mal then extracts static representations from these packages. The processing requirements of a detector and the acquisition history of its benchmark are therefore separate properties. Recording both makes explicit which applications and operating conditions a reported result describes.

7.3 Evaluation Principles Across the Contributions

Representation design determines the evidence available to a detector. Predictive evaluation measures how effectively it uses that evidence, while adversarial evaluation examines its decisions when an attacker deliberately modifies the input. The SAFAIR contest and benchmark in Chapter 6 contribute a procedure for investigating this second question under explicit attacker assumptions.

The Android studies provide matched, within-system comparisons under their own protocols. Hybroid holds the applications and learner fixed when comparing input modalities. The HGANN-Mal evaluation uses common call graphs and node features across its relational configurations and reports five-seed means for each fixed partition. These controls provide a consistent basis for comparing input modalities and relational structures. Time-ordered evaluation addresses performance on later observations [9]. Each protocol examines a distinct evaluation question under stated conditions.

Reusable software provides a second connection between the contribution chapters. NADICS places estimators behind a common interface, so that researchers can change learning methods without rewriting the surrounding pipeline [25]. Its semi-automated model-development objective is to support candidate comparison on a new dataset, with researchers responsible for the selection criterion and validation design. Reuse of the workflow requires fitting and assessment on the new data; it does not establish transfer of a trained detector between installations. SAFAIR applies a corresponding separation to evaluation: attack implementations share an execution procedure, while task-specific components define the model and data interfaces [32]. Vicomtech’s application of the framework to histopathology defences demonstrates reuse on a prediction task outside the facial-image contest [32, Section 3.1.3.1]. Both frameworks separate components that researchers can vary within one configured procedure.

Chapter 6 adds analytical criteria for interpreting evaluation scores. A constant predictor has zero accuracy loss under label-preserving attacks (Equation 6.14), even when its classification utility is limited. Clean and attacked accuracy therefore answer different questions and should be reported together. Under the benchmark’s binary label-reversal response, Equation 6.11 relates the change in task accuracy to the correct and incorrect base predictions that trigger an alarm. These derivations make the scoring procedure part of the scientific contribution. Their assumptions can be examined independently of a particular benchmark, and they specify outcomes needed to assess the complete defended system [14].

7.4 Implications for Android Malware Analysis

For Android analysis, the distinction between acquiring evidence and structuring it gives two design choices that can be investigated together. A common-cohort evaluation could compare static relational encodings and then measure the additional value of traffic for the same prediction task. Acquisition effort and construction time would give those performance differences an operational interpretation.

The SAFAIR evaluation design provides a basis for extending such a comparison to an explicit adversary. Android-specific tests would use valid package transformations that preserve malicious functionality and would assess the complete pipeline, including fusion where applicable [14]. The Android studies establish predictive results; resistance to those transformations remains a question for that evaluation. Together, the contributions relate contextual modelling to the evidence and measurement procedures needed to assess a security-analysis system.

Conclusion and Future Work

8.1 Answers to the Research Questions

RQ1. *How can static program structure and observed network behaviour be jointly represented for Android malware detection and category classification, and how does their combination affect predictive performance compared with either source alone?* Hybroid represents both sources at application level: a 64-dimensional program-structure vector is concatenated with 13 flow features averaged per application. On CICAndMal2017, this fusion improved reported detection F_1 by one or two percentage points over the better single source. For category classification, the difference ranged from minus two to plus three points across learners. These matched comparisons address gap G1 within the evaluated corpus and protocol.

RQ2. *How can higher-order relations within static Android program structure be represented and learned, and to what extent do they improve malware detection and classification over pairwise graph representations?* HGANN-Mal groups methods by call neighbourhood, semantic similarity and security-sensitive slices, and learns membership weights by attention. On Drebin and CICMalDroid, five-run mean accuracy exceeded the stronger pairwise baseline by 3.1 to 10.8 points for every hypergraph configuration under common inputs and fixed splits. HGANN-Mal led the strongest non-attentive baseline in three of four settings by 1.2 to 4.1 points; its CICMalDroid binary accuracy was 97.6% against 97.7%. These complete-system comparisons address gap G2; attention attribution requires a controlled ablation.

RQ3. *How can a modular learning framework analyse network and process features for anomaly detection in industrial control systems?* NADICS separates feature preparation from interchangeable learning components to support researcher-guided comparison and selection of candidate models for a dataset. On the session-disjoint S7-300 evaluation, a random forest achieved 94.50% attack F_1 at a record-level false-positive rate of 0.484%. The GAN-based extension with 24 PCA components achieved 91.89% timestamp recall at a false-positive rate of 5.08% on later SWaT test data. These separate evaluations address gap G3 through a modular framework and observation-specific analysis within the tested industrial settings.

RQ4. *How can an explicit adversary model be translated into a reproducible benchmark for adversarial attacks and defences, and what does such a benchmark reveal about the relation between clean predictive performance and adversarial robustness?* SAFAIR translated attacker assumptions into a contest protocol and a benchmark with task-specific interfaces. The implementation and scoring analysis address the design and measurement aspects of gap G4. Any constant classifier has zero accuracy loss under label-preserving attacks. In Vicomtech's histopathology benchmark, the activation detector had zero loss under PGD but an attacked accuracy 12.9 percentage points below adversarial training. Clean and attacked accuracy

therefore need to be reported together, under an explicit prediction objective and valid perturbation constraints. The four contest rows do not establish a general relation between clean performance and robustness.

8.2 Summary of Contributions

The Android studies form the main contribution. Hybroid combines static program-graph representations with observed network flows and measures the value of fusion on the same applications. The candidate designed the overall pipeline and traffic analysis and conducted the evaluation; his contribution to graph processing was partial. HGANN-Mal represents context within the application package. Chapter 4 formalises its construction and shows that different hyperedge groupings can produce different attentive propagation even when their fixed propagation matrices coincide.

The industrial contribution is a modular framework for semi-automated model development on network and process observations. It combines configurable preparation with comparison of candidate learners, supported by the candidate's extractor and labelled corpus and a GAN-based extension for SWaT. SAFAIR contributes a benchmark design for adversarial robustness evaluation and reusable software with task-specific interfaces. Chapter 6 analyses accuracy-loss scoring and the effect of binary label reversal after an alarm. Partner defences provide applications of the framework.

8.3 Future Work

The first priority is a controlled HGANN-Mal ablation on both corpora. Comparing learned and fixed weighting over identical fused hyperedges requires common preprocessing and normalisation, with reported variation across runs. A comparison with topological hyperedges alone would assess the contribution of construction. A common Android cohort with time-ordered splits would permit comparison of static relational encodings and the additional value of traffic, as proposed in Chapter 7. Acquisition effort and computation cost should be measured separately. Adversarial evaluation would then apply valid package transformations that preserve malicious functionality, under stated attacker access and budgets. It should measure clean and attacked performance of the complete pipeline, including fusion.

The dissertation connects contextual representation design with the evidence and evaluation procedures needed to assess learning-based security decisions.

References

- [1] T. Bilot, N. El Madhoun, K. Al Agha, and A. Zouaoui. “A Survey on Malware Detection with Graph Representation Learning”. In: *ACM Computing Surveys* 56.11 (2024), pp. 1–36. DOI: 10.1145/3664649.
- [2] D. Arp, M. Spreitzenbarth, M. Hübner, H. Gascon, and K. Rieck. “DREBIN: Effective and Explainable Detection of Android Malware in Your Pocket”. In: *Proceedings of the Network and Distributed System Security Symposium (NDSS)*. 2014. DOI: 10.14722/ndss.2014.23247.
- [3] L. Onwuzurike, E. Mariconti, P. Andriotis, E. De Cristofaro, G. Ross, and G. Stringhini. “MaMaDroid: Detecting Android Malware by Building Markov Chains of Behavioral Models (Extended Version)”. In: *ACM Transactions on Privacy and Security* 22.2 (2019), pp. 1–34. DOI: 10.1145/3313391.
- [4] Y. Liu, C. Tantithamthavorn, L. Li, and Y. Liu. “Deep Learning for Android Malware Defenses: A Systematic Literature Review”. In: *ACM Computing Surveys* 55 (2023), pp. 1–36. DOI: 10.1145/3544968.
- [5] M. R. Norouzian, P. Xu, C. Eckert, and A. Zarras. “Hybroid: Toward Android Malware Detection and Categorization with Program Code and Network Traffic”. In: *Information Security: 24th International Conference, ISC 2021*. Vol. 13118. Lecture Notes in Computer Science. Springer, 2021, pp. 259–278. DOI: 10.1007/978-3-030-91356-4_14.
- [6] M. R. Norouzian and C. Eckert. “HGANN-Mal: A Hypergraph Attention Neural Network Approach for Android Malware Detection”. In: *Information Security: 28th International Conference, ISC 2025*. Vol. 16186. Lecture Notes in Computer Science. Springer, 2025, pp. 388–408. DOI: 10.1007/978-3-032-08124-7_23.
- [7] F.-X. Aubet, M.-O. Pahl, S. Liebald, and M. R. Norouzian. *Graph-based Anomaly Detection for IoT Microservices*. Two-page contribution, Passive and Active Measurement Conference (PAM). Berlin, Germany, 2018.
- [8] R. Jordaney, K. Sharad, S. K. Dash, Z. Wang, D. Papini, I. Nouretdinov, and L. Cavallaro. “Transcend: Detecting Concept Drift in Malware Classification Models”. In: *USENIX Security Symposium*. 2017, pp. 625–642.
- [9] F. Pendlebury, F. Pierazzi, R. Jordaney, J. Kinder, and L. Cavallaro. “TESSERACT: Eliminating Experimental Bias in Malware Classification Across Space and Time”. In: *USENIX Security Symposium*. 2019, pp. 729–746. arXiv: 1807.07838.

- [10] D. Arp, E. Quiring, F. Pendlebury, A. Warnecke, F. Pierazzi, C. Wressnegger, L. Cavallaro, and K. Rieck. “Dos and Don’ts of Machine Learning in Computer Security”. In: *Proceedings of the 31st USENIX Security Symposium*. USENIX Association, 2022, pp. 3971–3988. arXiv: 2010.09470. URL: <https://www.usenix.org/conference/usenixsecurity22/presentation/arp>.
- [11] P. Irolla and A. Dey. “The Duplication Issue within the Drebin Dataset”. In: *Journal of Computer Virology and Hacking Techniques* 14.3 (2018), pp. 245–249. DOI: 10.1007/s11416-018-0316-z.
- [12] Y. Zhao, L. Li, H. Wang, H. Cai, T. F. Bissyandé, J. Klein, and J. Grundy. “On the Impact of Sample Duplication in Machine-Learning-Based Android Malware Detection”. In: *ACM Transactions on Software Engineering and Methodology* 30.3 (2021), 40:1–40:38.
- [13] A. Athalye, N. Carlini, and D. Wagner. “Obfuscated Gradients Give a False Sense of Security: Circumventing Defenses to Adversarial Examples”. In: *International Conference on Machine Learning (ICML)*. Vol. 80. Proceedings of Machine Learning Research. 2018, pp. 274–283.
- [14] N. Carlini, A. Athalye, N. Papernot, W. Brendel, J. Rauber, D. Tsipras, I. J. Goodfellow, A. Madry, and A. Kurakin. *On Evaluating Adversarial Robustness*. arXiv:1902.06705. 2019. arXiv: 1902.06705 [cs.LG].
- [15] F. Tramèr, N. Carlini, W. Brendel, and A. Madry. “On Adaptive Attacks to Adversarial Example Defenses”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2020, pp. 1633–1645. arXiv: 2002.08347.
- [16] F. Pierazzi, F. Pendlebury, J. Cortellazzi, and L. Cavallaro. “Intriguing Properties of Adversarial ML Attacks in the Problem Space”. In: *IEEE Symposium on Security and Privacy (S&P)*. 2020, pp. 1332–1349. DOI: 10.1109/SP40000.2020.00073. URL: <https://arxiv.org/abs/1911.02142v2>.
- [17] L. Li, T. F. Bissyandé, M. Papadakis, S. Rasthofer, A. Bartel, D. Octeau, J. Klein, and Y. Le Traon. “Static Analysis of Android Apps: A Systematic Literature Review”. In: *Information and Software Technology* 88 (2017), pp. 67–95. DOI: 10.1016/j.infsof.2017.04.001.
- [18] A. Moser, C. Kruegel, and E. Kirda. “Limits of Static Analysis for Malware Detection”. In: *Proceedings of the 23rd Annual Computer Security Applications Conference (ACSAC)*. 2007, pp. 421–430. DOI: 10.1109/ACSAC.2007.21.
- [19] W. Enck, P. Gilbert, S. Han, V. Tendulkar, B.-G. Chun, L. P. Cox, J. Jung, P. McDaniel, and A. N. Sheth. “TaintDroid: An Information-Flow Tracking System for Realtime Privacy Monitoring on Smartphones”. In: *ACM Transactions on Computer Systems* 32 (2014), pp. 1–29. DOI: 10.1145/2619091.
- [20] L. Onwuzurike, M. Almeida, E. Mariconti, J. Blackburn, G. Stringhini, and E. De Cristofaro. “A Family of Droids – Android Malware Detection via Behavioral Modeling: Static vs Dynamic Analysis”. In: *Annual Conference on Privacy, Security and Trust (PST)*. 2018, pp. 1–10. DOI: 10.1109/PST.2018.8514191.
- [21] D. Zhou, J. Huang, and B. Schölkopf. “Learning with Hypergraphs: Clustering, Classification, and Embedding”. In: *Advances in Neural Information Processing Systems 19 (NIPS)*. MIT Press, 2006, pp. 1601–1608. DOI: 10.7551/mitpress/7503.003.0205.

- [22] S. Agarwal, K. Branson, and S. Belongie. “Higher Order Learning with Graphs”. In: *Proceedings of the 23rd International Conference on Machine Learning (ICML)*. 2006, pp. 17–24. DOI: 10.1145/1143844.1143847.
- [23] D. Zhang, X. Wu, E. He, X. Guo, X. Yang, R. Li, and H. Li. “Android Malware Detection Based on Hypergraph Neural Networks”. In: *Applied Sciences* 13.23 (2023), p. 12629. DOI: 10.3390/app132312629.
- [24] S. Bai, F. Zhang, and P. H. S. Torr. “Hypergraph Convolution and Hypergraph Attention”. In: *Pattern Recognition* 110 (2021), p. 107637. DOI: 10.1016/j.patcog.2020.107637.
- [25] S. Duque Antón, M. R. Norouzian, K. Fischer, D. Friedrich, P. Kasinathan, S. Teuber, R. Hormann, M. Soppa, S. Dukanovic, J. Hachenberger, A. Giehl, S. Plaga, S. Tatschner, and N. Wiedermann. *Anwendungsfall "Visueller Security-Leitstand"*. Abschlussdeliverable, Arbeitspaket 4. IUNO — Nationales Referenzprojekt IT-Sicherheit in Industrie 4.0, 2018.
- [26] A. P. Mathur and N. O. Tippenhauer. “SWaT: A Water Treatment Testbed for Research and Training on ICS Security”. In: *Proceedings of the International Workshop on Cyber-Physical Systems for Smart Water Networks (CySWater)*. IEEE, 2016, pp. 31–36. DOI: 10.1109/CySWater.2016.7469060.
- [27] J. Rauber, W. Brendel, and M. Bethge. *Foolbox: A Python Toolbox to Benchmark the Robustness of Machine Learning Models*. arXiv:1707.04131. 2017.
- [28] A. Kurakin, I. Goodfellow, S. Bengio, Y. Dong, F. Liao, M. Liang, T. Pang, J. Zhu, X. Hu, C. Xie, J. Wang, Z. Zhang, Z. Ren, A. Yuille, S. Huang, Y. Zhao, Y. Zhao, Z. Han, J. Long, Y. Berdibekov, T. Akiba, S. Tokui, and M. Abe. “Adversarial Attacks and Defences Competition”. In: *The NIPS ’17 Competition: Building Intelligent Systems*. The Springer Series on Challenges in Machine Learning. Springer, 2018, pp. 195–231. DOI: 10.1007/978-3-319-94042-7_11.
- [29] M. R. Norouzian. *SAFAIR AI Contest*. Chair for IT Security, Technical University of Munich. 2021. URL: <https://www.sec.in.tum.de/i20/projects/sparta-safair-ai-contest> (visited on 08/15/2026).
- [30] M. R. Norouzian. *Adversarial Machine Learning Benchmark Tool*. Source code, revision 2fac62b (8 November 2021). GitHub. 2021. URL: <https://github.com/reza-norouzian/Adversarial-Benchmark-Tool/tree/2fac62b947ed> (visited on 09/10/2026).
- [31] SPARTA Consortium. *Preliminary Description of AI Systems Security Mechanisms and Tools*. Project Deliverable D7.2. Editor: M. Pawlicki (ITTI). Dissemination level: public. 85 pages. SPARTA Project, EU Horizon 2020 grant agreement No. 830892, July 2020. URL: <https://sparta.eu/assets/deliverables/SPARTA-D7.2-Preliminary-description-of-AI-systems-security-mechanisms-and-tools-PU-M18.pdf>.
- [32] SPARTA Consortium. *Validation and Evaluation Report*. Project Deliverable D7.6. Editor: M. R. Norouzian (TUM). Dissemination level: public. 73 pages. SPARTA Project, EU Horizon 2020 grant agreement No. 830892, Feb. 2022. URL: <https://sparta.eu/assets/deliverables/SPARTA-D7.6-Final-version-of-evaluation-and-validation-plan-PU-M36.pdf>.

- [33] SPARTA Consortium. *Final Version of AI Systems Security Mechanisms and Tools*. Project Deliverable D7.5. Editor: B. Addad (TCS). Contributor: M. R. Norouzian (TUM). Dissemination level: public. 129 pages. SPARTA Project, EU Horizon 2020 grant agreement No. 830892, Aug. 2021. URL: <https://sparta.eu/assets/deliverables/SPARTA-D7.5-Final-version-of-AI-systems-security-mechanisms-and-tools-PU-M30.pdf>.
- [34] J. Goh, S. Adepu, K. N. Junejo, and A. Mathur. “A Dataset to Support Research in the Design of Secure Water Treatment Systems”. In: *Critical Information Infrastructures Security (CRITIS)*. Vol. 10242. Lecture Notes in Computer Science. Springer, 2017, pp. 88–99. DOI: 10.1007/978-3-319-71368-7_8.
- [35] S. Duque Antón, D. Fraunholz, C. Lipps, D. Angermeier, B. Filipovic, A. Giehl, J. Obermaier, P. Schneider, D. Schuster, N. Wiedermann, M. Rudolph, H. Brandl, F. Schreiner, K. Fischer, D. Friedrich, B. Bierbaumer, M. R. Norouzian, J. Müller, G. Fischer, and W. Neifer. *Werkzeuge*. Abschlussdeliverable, Arbeitspaket 7. IUNO — Nationales Referenzprojekt IT-Sicherheit in Industrie 4.0, 2018.
- [36] A. Kleinmann and A. Wool. “Accurate Modeling of the Siemens S7 SCADA Protocol for Intrusion Detection and Digital Forensics”. In: *Journal of Digital Forensics, Security and Law* 9.2 (2014). DOI: 10.15394/jdfs1.2014.1169.
- [37] SPARTA Consortium. *Validation and Evaluation Plan*. Project Deliverable D7.3. Editor: M. R. Norouzian (TUM). Dissemination level: public. 19 pages. SPARTA Project, EU Horizon 2020 grant agreement No. 830892, June 2021. URL: <https://sparta.eu/assets/deliverables/SPARTA-D7.3-Validation-and-evaluation-plan-PU-M21-v1.1.pdf>.
- [38] SPARTA Consortium. *AI Systems Threat Analysis Mechanisms and Tools*. Project Deliverable D7.1. Editor: E. Ríos (TEC). Contributor: M. R. Norouzian (TUM). Dissemination level: public. 95 pages. SPARTA Project, EU Horizon 2020 grant agreement No. 830892, June 2021. URL: https://sparta.eu/assets/deliverables/SPARTA-D7.1-AI-systems-threat-analysis-mechanisms-and-tools-PU-M18_v1.1.pdf.
- [39] M. R. Norouzian. *SAFAIR AI Contest Development Toolkit*. Source code, revision f44ba8c (8 November 2021). GitHub. 2021. URL: https://github.com/reza-norouzian/SPARTA-Adversarial-AI-Contest/tree/f44ba8c4ea13/dev_toolkit (visited on 09/20/2026).
- [40] M.-O. Pahl and F.-X. Aubet. “All Eyes on You: Distributed Multi-Dimensional IoT Microservice Anomaly Detection”. In: *Proceedings of the 14th International Conference on Network and Service Management (CNSM)*. IEEE, 2018, pp. 72–80. URL: <https://ieeexplore.ieee.org/document/8584985>.
- [41] M.-O. Pahl, F.-X. Aubet, and S. Liebold. “Graph-based IoT Microservice Security”. In: *NOMS 2018 – 2018 IEEE/IFIP Network Operations and Management Symposium*. IEEE, 2018, pp. 1–3. DOI: 10.1109/NOMS.2018.8406118.
- [42] R. Hormann, S. Teuber, S. Nikelski, P. Larbig, S. Neff, D. Scheuermann, A. Giehl, S. Plaga, M. R. Norouzian, and A. Malkis. *Anomalieerkennung im Wertschöpfungsnetzwerk; Datenaustauschplattform für das Wertschöpfungsnetzwerk*. Deliverable D4.2-1 / M24. IUNO — Nationales Referenzprojekt IT-Sicherheit in Industrie 4.0, June 2017.
- [43] Android Developers. *Application Fundamentals*. Google. 2026. URL: <https://developer.android.com/guide/components/fundamentals> (visited on 09/20/2026).

- [44] Android Open Source Project. *Dalvik Executable Format*. Android Open Source Project. 2026. URL: <https://source.android.com/docs/core/runtime/dex-format> (visited on 09/20/2026).
- [45] Android Open Source Project. *Android Runtime (ART) and Dalvik*. Android Open Source Project. 2026. URL: <https://source.android.com/docs/core/runtime> (visited on 09/20/2026).
- [46] Android Open Source Project. *Dalvik Bytecode*. Android Open Source Project. 2026. URL: <https://source.android.com/docs/core/runtime/dalvik-bytecode> (visited on 09/20/2026).
- [47] S. Arzt, S. Rasthofer, C. Fritz, E. Bodden, A. Bartel, J. Klein, Y. Le Traon, D. Ocateau, and P. McDaniel. “FlowDroid: Precise Context, Flow, Field, Object-Sensitive and Lifecycle-Aware Taint Analysis for Android Apps”. In: *ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. 2014, pp. 259–269. DOI: 10.1145/2594291.2594299.
- [48] A. P. Felt, E. Chin, S. Hanna, D. Song, and D. Wagner. “Android Permissions Demystified”. In: *ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 2011, pp. 627–638. DOI: 10.1145/2046707.2046779.
- [49] Android Open Source Project. *Permissions on Android*. Android Open Source Project. 2026. URL: <https://developer.android.com/guide/topics/permissions/overview> (visited on 09/20/2026).
- [50] K. W. Y. Au, Y. F. Zhou, Z. Huang, and D. Lie. “PScout: Analyzing the Android Permission Specification”. In: *ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 2012.
- [51] A. Sabbah, R. Jarrar, S. Zein, and D. Mohaisen. “Understanding Concept Drift With Deprecated Permissions in Android Malware Detection”. In: *IEEE Transactions on Dependable and Secure Computing* 23.4 (2026), pp. 8292–8306. DOI: 10.1109/TDSC.2026.3682216.
- [52] A. Desnos. “Android: Static Analysis Using Similarity Distance”. In: *Hawaii International Conference on System Sciences (HICSS)*. 2012, pp. 5394–5403. DOI: 10.1109/HICSS.2012.114.
- [53] R. Vallée-Rai, P. Co, E. Gagnon, L. Hendren, P. Lam, and V. Sundaresan. “Soot: A Java Bytecode Optimization Framework”. In: *CASCON First Decade High Impact Papers*. 2010, pp. 214–224. DOI: 10.1145/1925805.1925818.
- [54] L. K. Yan and H. Yin. “DroidScope: Seamlessly Reconstructing the OS and Dalvik Semantic Views for Dynamic Android Malware Analysis”. In: *USENIX Security Symposium*. 2012, pp. 569–584.
- [55] T. Petsas, G. Voyatzis, E. Athanasopoulos, M. Polychronakis, and S. Ioannidis. “Rage against the Virtual Machine: Hindering Dynamic Analysis of Android Malware”. In: *European Workshop on System Security (EuroSec)*. 2014, pp. 1–6. DOI: 10.1145/2592791.2592796.
- [56] T. Vidas and N. Christin. “Evading Android Runtime Analysis via Sandbox Detection”. In: *ACM Symposium on Information, Computer and Communications Security (AsiaCCS)*. 2014, pp. 447–458. DOI: 10.1145/2590296.2590325.

- [57] M. Lindorfer, M. Neugschwandtner, and C. Platzer. “MARVIN: Efficient and Comprehensive Mobile App Classification through Static and Dynamic Analysis”. In: *Proceedings of the 39th IEEE Annual Computer Software and Applications Conference (COMPSAC)*. 2015, pp. 422–433. DOI: 10.1109/COMPSAC.2015.103.
- [58] D. Wermke, N. Huaman, Y. Acar, B. Reaves, P. Traynor, and S. Fahl. “A Large Scale Investigation of Obfuscation Use in Google Play”. In: *Annual Computer Security Applications Conference (ACSAC)*. 2018, pp. 222–235. DOI: 10.1145/3274694.3274726.
- [59] S. Dong, M. Li, W. Diao, X. Liu, J. Liu, Z. Li, F. Xu, K. Chen, X. Wang, and K. Zhang. “Understanding Android Obfuscation Techniques: A Large-Scale Investigation in the Wild”. In: *International Conference on Security and Privacy in Communication Systems (SecureComm)*. 2018, pp. 172–192. DOI: 10.1007/978-3-030-01701-9_10.
- [60] M. Hammad, J. Garcia, and S. Malek. “A Large-Scale Empirical Study on the Effects of Code Obfuscations on Android Apps and Anti-Malware Products”. In: *International Conference on Software Engineering (ICSE)*. 2018. DOI: 10.1145/3180155.3180228.
- [61] C. Gao, G. Huang, H. Li, B. Wu, Y. Wu, and W. Yuan. “A Comprehensive Study of Learning-Based Android Malware Detectors under Challenging Environments”. In: *International Conference on Software Engineering (ICSE)*. 2024. DOI: 10.1145/3597503.3623320.
- [62] Y. Duan, M. Zhang, A. V. Bhaskar, H. Yin, et al. “Things You May Not Know About Android (Un)Packers: A Systematic Study Based on Whole-System Emulation”. In: *Network and Distributed System Security Symposium (NDSS)*. 2018.
- [63] L. Li, D. Li, T. F. Bissyandé, J. Klein, Y. Le Traon, D. Lo, and L. Cavallaro. “Understanding Android App Piggybacking: A Systematic Study of Malicious Code Grafting”. In: *IEEE Transactions on Information Forensics and Security* 12.6 (2017), pp. 1269–1284. DOI: 10.1109/TIFS.2017.2656460.
- [64] H. Gascon, F. Yamaguchi, D. Arp, and K. Rieck. “Structural Detection of Android Malware Using Embedded Call Graphs”. In: *Proceedings of the 2013 ACM Workshop on Artificial Intelligence and Security (AISec)*. 2013, pp. 45–54. DOI: 10.1145/2517312.2517315.
- [65] H. Dai, B. Dai, and L. Song. “Discriminative Embeddings of Latent Variable Models for Structured Data”. In: *International Conference on Machine Learning (ICML)*. Vol. 48. Proceedings of Machine Learning Research. 2016, pp. 2702–2711.
- [66] X. Xu, C. Liu, Q. Feng, H. Yin, L. Song, and D. Song. “Neural Network-Based Graph Embedding for Cross-Platform Binary Code Similarity Detection”. In: *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 2017, pp. 363–376. DOI: 10.1145/3133956.3134018.
- [67] U. Alon, M. Zilberstein, O. Levy, and E. Yahav. “code2vec: Learning Distributed Representations of Code”. In: *Proceedings of the ACM on Programming Languages* 3.POPL (2019), pp. 1–29. DOI: 10.1145/3290353.
- [68] S. Horwitz, T. Reps, and D. Binkley. “Interprocedural Slicing Using Dependence Graphs”. In: *ACM Transactions on Programming Languages and Systems* 12.1 (1990), pp. 26–60. DOI: 10.1145/77606.77608.
- [69] U. Chitra and B. J. Raphael. “Random Walks on Hypergraphs with Edge-Dependent Vertex Weights”. In: *Proceedings of the 36th International Conference on Machine Learning (ICML)*. 2019. arXiv: 1905.08287.

- [70] Y. Wang and J. Kleinberg. “From Graphs to Hypergraphs: Hypergraph Projection and Its Remediation”. In: *International Conference on Learning Representations (ICLR)*. 2024. URL: <https://arxiv.org/abs/2401.08519>.
- [71] D. Y. Kang, Q. Mei, and S.-W. Kim. “Low Mileage, High Fidelity: Evaluating Hypergraph Expansion Methods by Quantifying the Information Loss”. In: *Proceedings of the ACM Web Conference (WWW)*. 2024, pp. 959–968. DOI: 10.1145/3589334.3645657.
- [72] F. Jiang, Y. Li, Y. Feng, K. Zheng, L. Niu, B. Ramasubramanian, B. Alomair, L. Bushnell, and R. Poovendran. *The WidthWall: A Strict Expressivity Hierarchy for Hypergraph Neural Networks*. 2026. arXiv: 2605.13690 [cs.LG].
- [73] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl. “Neural Message Passing for Quantum Chemistry”. In: *International Conference on Machine Learning (ICML)*. 2017, pp. 1263–1272. arXiv: 1704.01212.
- [74] T. N. Kipf and M. Welling. “Semi-Supervised Classification with Graph Convolutional Networks”. In: *5th International Conference on Learning Representations (ICLR)*. 2017. arXiv: 1609.02907 [cs.LG].
- [75] K. Xu, W. Hu, J. Leskovec, and S. Jegelka. “How Powerful are Graph Neural Networks?” In: *International Conference on Learning Representations (ICLR)*. 2019.
- [76] Q. Li, Z. Han, and X.-M. Wu. “Deeper Insights into Graph Convolutional Networks for Semi-Supervised Learning”. In: *Proceedings of the AAAI Conference on Artificial Intelligence* 32.1 (2018). DOI: 10.1609/aaai.v32i1.11604.
- [77] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio. “Graph Attention Networks”. In: *6th International Conference on Learning Representations (ICLR)*. 2018. arXiv: 1710.10903 [stat.ML].
- [78] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. “Attention Is All You Need”. In: *Advances in Neural Information Processing Systems (NIPS)*. 2017, pp. 5998–6008. arXiv: 1706.03762.
- [79] S. Brody, U. Alon, and E. Yahav. “How Attentive Are Graph Attention Networks?” In: *Proceedings of the 10th International Conference on Learning Representations (ICLR)*. 2022. arXiv: 2105.14491.
- [80] T. Fang, T. Gao, C. Wang, Y. Shang, W. Chow, L. Chen, and Y. Yang. “KAA: Kolmogorov-Arnold Attention for Enhancing Attentive Graph Neural Networks”. In: *International Conference on Learning Representations (ICLR)*. 2025. URL: <https://openreview.net/forum?id=atXCzVSXTJ>.
- [81] Y. Feng, H. You, Z. Zhang, R. Ji, and Y. Gao. “Hypergraph Neural Networks”. In: *Proceedings of the AAAI Conference on Artificial Intelligence* 33.01 (2019), pp. 3558–3565. DOI: 10.1609/aaai.v33i01.33013558.
- [82] Y. Gao, Y. Feng, S. Ji, and R. Ji. “HGNN⁺: General Hypergraph Neural Networks”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 45.3 (2023), pp. 3181–3199. DOI: 10.1109/TPAMI.2022.3182052.
- [83] N. Yadati, M. Nimishakavi, P. Yadav, V. Nitin, A. Louis, and P. Talukdar. “HyperGCN: A New Method for Training Graph Convolutional Networks on Hypergraphs”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2019, pp. 1509–1520. arXiv: 1809.02589.

- [84] E. Chien, C. Pan, J. Peng, and O. Milenkovic. “You Are AllSet: A Multiset Function Framework for Hypergraph Neural Networks”. In: *International Conference on Learning Representations (ICLR)*. 2022. arXiv: 2106.13264.
- [85] J. Kim, S. Oh, S. Cho, and S. Hong. “Equivariant Hypergraph Neural Networks”. In: *Computer Vision – ECCV 2022*. Lecture Notes in Computer Science. 2022, pp. 86–103. DOI: 10.1007/978-3-031-19803-8_6.
- [86] J. Kim, S. Oh, and S. Hong. “Transformers Generalize DeepSets and Can Be Extended to Graphs & Hypergraphs”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2021, pp. 28016–28028. URL: <https://proceedings.neurips.cc/paper/2021/hash/ec0f40c389aeef789ce03eb814facc6c-Abstract.html>.
- [87] M. Li, Y. Fang, D. Shen, H. Feng, X. Zhuang, K. Xia, and P. Liò. “High-Pass Matters: Theoretical Insights and Sheaflet-Based Design for Hypergraph Neural Networks”. In: *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*. Vol. 40. 2026, pp. 23039–23046. DOI: 10.1609/aaai.v40i27.39469.
- [88] Y. Gao, J. Yan, Y. Feng, X. Han, S. Ying, Z. Wu, and H. Hu. “HGNNv2: Stable Hypergraph Neural Networks”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 48.5 (2026), pp. 5311–5327. DOI: 10.1109/TPAMI.2026.3652225.
- [89] V. Paxson. “Bro: A System for Detecting Network Intruders in Real-Time”. In: *Computer Networks* 31.23–24 (1999), pp. 2435–2463. DOI: 10.1016/S1389-1286(99)00112-7.
- [90] V. Chandola, A. Banerjee, and V. Kumar. “Anomaly Detection: A Survey”. In: *ACM Computing Surveys* 41.3 (2009), pp. 1–58. DOI: 10.1145/1541880.1541882.
- [91] R. Sommer and V. Paxson. “Outside the Closed World: On Using Machine Learning for Network Intrusion Detection”. In: *Proceedings of the 2010 IEEE Symposium on Security and Privacy (S&P)*. IEEE, 2010, pp. 305–316. DOI: 10.1109/SP.2010.25.
- [92] M. M. Breunig, H.-P. Kriegel, R. T. Ng, and J. Sander. “LOF: Identifying Density-Based Local Outliers”. In: *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data*. ACM, 2000, pp. 93–104. DOI: 10.1145/342009.335388.
- [93] F. T. Liu, K. M. Ting, and Z.-H. Zhou. “Isolation Forest”. In: *2008 Eighth IEEE International Conference on Data Mining (ICDM)*. IEEE, 2008, pp. 413–422. DOI: 10.1109/ICDM.2008.17.
- [94] L. Ruff, J. R. Kauffmann, R. A. Vandermeulen, G. Montavon, W. Samek, M. Kloft, T. G. Dietterich, and K.-R. Müller. “A Unifying Review of Deep and Shallow Anomaly Detection”. In: *Proceedings of the IEEE* 109.5 (2021), pp. 756–795. DOI: 10.1109/JPROC.2021.3052449.
- [95] G. Pang, C. Shen, L. Cao, and A. van den Hengel. “Deep Learning for Anomaly Detection: A Review”. In: *ACM Computing Surveys* 54.2 (2021), pp. 1–38. DOI: 10.1145/3439950.
- [96] L. Akoglu, H. Tong, and D. Koutra. “Graph Based Anomaly Detection and Description: A Survey”. In: *Data Mining and Knowledge Discovery* 29.3 (2015), pp. 626–688. DOI: 10.1007/s10618-014-0365-y.
- [97] A. Sperotto, G. Schaffrath, R. Sadre, C. Morariu, A. Pras, and B. Stiller. “An Overview of IP Flow-Based Intrusion Detection”. In: *IEEE Communications Surveys & Tutorials* 12.3 (2010), pp. 343–356. DOI: 10.1109/SURV.2010.032210.00054.

- [98] R. R. R. Barbosa, R. Sadre, and A. Pras. “Flow Whitelisting in SCADA Networks”. In: *International Journal of Critical Infrastructure Protection* 6.3–4 (2013), pp. 150–158. DOI: 10.1016/j.ijcip.2013.08.003.
- [99] M.-K. Yoon and G. F. Ciocarlie. “Communication Pattern Monitoring: Improving the Utility of Anomaly Detection for Industrial Control Systems”. In: *Proceedings of the NDSS Workshop on Security of Emerging Networking Technologies (SENT)*. Internet Society, 2014. DOI: 10.14722/sent.2014.23012.
- [100] M. Iliofotou, P. Pappu, M. Faloutsos, M. Mitzenmacher, S. Singh, and G. Varghese. “Network Monitoring Using Traffic Dispersion Graphs (TDGs)”. In: *Proceedings of the 7th ACM SIGCOMM Conference on Internet Measurement (IMC)*. ACM, 2007, pp. 315–320. DOI: 10.1145/1298306.1298349.
- [101] Y. Jin, E. Sharafuddin, and Z.-L. Zhang. “Unveiling Core Network-Wide Communication Patterns through Application Traffic Activity Graph Decomposition”. In: *ACM SIGMETRICS Performance Evaluation Review* 37.1 (2009), pp. 49–60. DOI: 10.1145/2492101.1555356.
- [102] P. Bahl, R. Chandra, A. Greenberg, S. Kandula, D. A. Maltz, and M. Zhang. “Towards Highly Reliable Enterprise Network Services via Inference of Multi-Level Dependencies”. In: *ACM SIGCOMM Computer Communication Review* 37.4 (2007), pp. 13–24. DOI: 10.1145/1282427.1282383.
- [103] X. Chen, M. Zhang, Z. M. Mao, and P. Bahl. “Automating Network Application Dependency Discovery: Experiences, Limitations, and New Solutions”. In: *Proceedings of the 8th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. USENIX Association, 2008, pp. 117–130.
- [104] A. Natarajan, P. Ning, Y. Liu, S. Jajodia, and S. E. Hutchinson. “NSDMiner: Automated Discovery of Network Service Dependencies”. In: *Proceedings of the IEEE INFOCOM*. IEEE, 2012, pp. 2507–2515. DOI: 10.1109/INFCOM.2012.6195642.
- [105] M. Kravchik and A. Shabtai. “Detecting Cyber Attacks in Industrial Control Systems Using Convolutional Neural Networks”. In: *Proceedings of the 2018 Workshop on Cyber-Physical Systems Security and Privacy (CPS-SPC)*. ACM, 2018, pp. 72–83. DOI: 10.1145/3264888.3264896.
- [106] E. Maali, O. Alrawi, and J. A. McCann. “Evaluating Machine Learning-Based IoT Device Identification Models for Security Applications”. In: *Network and Distributed System Security Symposium (NDSS)*. 2025.
- [107] M. Tavallaee, E. Bagheri, W. Lu, and A. A. Ghorbani. “A Detailed Analysis of the KDD CUP 99 Data Set”. In: *2009 IEEE Symposium on Computational Intelligence for Security and Defense Applications (CISDA)*. IEEE, 2009, pp. 1–6. DOI: 10.1109/CISDA.2009.5356528.
- [108] R. Flood, G. Engelen, D. Aspinall, and L. Desmet. “Bad Design Smells in Benchmark NIDS Datasets”. In: *IEEE European Symposium on Security and Privacy (EuroS&P)*. 2024, pp. 658–675. DOI: 10.1109/EUROSP60621.2024.00042.
- [109] D. Kus, E. Wagner, J. Pennekamp, K. Wolsing, I. B. Fink, M. Dahlmanns, K. Wehrle, and M. Henze. “A False Sense of Security? Revisiting the State of Machine Learning-Based Industrial Intrusion Detection”. In: *Proceedings of the 8th ACM Cyber-Physical System Security Workshop (CPSS)*. ACM, 2022, pp. 73–84. DOI: 10.1145/3494107.3522773.

- [110] K. Wolsing, L. Thiemt, C. van Sloun, E. Wagner, K. Wehrle, and M. Henze. “Can Industrial Intrusion Detection Be SIMPLE?” In: *Computer Security (ESORICS 2022)*. Lecture Notes in Computer Science. Springer, 2022, pp. 574–594. DOI: 10.1007/978-3-031-17143-7_28.
- [111] B. Biggio and F. Roli. “Wild Patterns: Ten Years After the Rise of Adversarial Machine Learning”. In: *Pattern Recognition* 84 (2018), pp. 317–331. DOI: 10.1016/j.patcog.2018.07.023.
- [112] B. Biggio, G. Fumera, and F. Roli. “Security Evaluation of Pattern Classifiers under Attack”. In: *IEEE Transactions on Knowledge and Data Engineering* 26.4 (2014), pp. 984–996. DOI: 10.1109/TKDE.2013.57.
- [113] A. Vassilev, A. Oprea, A. Fordyce, H. Anderson, X. Davies, and M. Hamin. *Adversarial Machine Learning: A Taxonomy and Terminology of Attacks and Mitigations*. Tech. rep. NIST AI 100-2e2025. National Institute of Standards and Technology, Mar. 2025. DOI: 10.6028/NIST.AI.100-2e2025.
- [114] N. Papernot, P. McDaniel, I. Goodfellow, S. Jha, Z. B. Celik, and A. Swami. “Practical Black-Box Attacks against Machine Learning”. In: *ACM Asia Conference on Computer and Communications Security (ASIACCS)*. 2017, pp. 506–519. DOI: 10.1145/3052973.3053009.
- [115] K. Grosse, L. Bieringer, T. R. Besold, and A. Alahi. “Towards More Practical Threat Models in Artificial Intelligence Security”. In: *33rd USENIX Security Symposium (USENIX Security 24)*. 2024.
- [116] B. Biggio, I. Corona, D. Maiorca, B. Nelson, N. Šrndić, P. Laskov, G. Giacinto, and F. Roli. “Evasion Attacks against Machine Learning at Test Time”. In: *European Conference on Machine Learning and Knowledge Discovery in Databases (ECML PKDD)*. Vol. 8190. Lecture Notes in Computer Science. Springer, 2013, pp. 387–402. DOI: 10.1007/978-3-642-40994-3_25.
- [117] B. Biggio, B. Nelson, and P. Laskov. “Poisoning Attacks against Support Vector Machines”. In: *International Conference on Machine Learning (ICML)*. 2012, pp. 1807–1814. arXiv: 1206.6389.
- [118] M. Jagielski, A. Oprea, B. Biggio, C. Liu, C. Nita-Rotaru, and B. Li. “Manipulating Machine Learning: Poisoning Attacks and Countermeasures for Regression Learning”. In: *IEEE Symposium on Security and Privacy (S&P)*. 2018, pp. 19–35. DOI: 10.1109/SP.2018.00057.
- [119] T. Gu, K. Liu, B. Dolan-Gavitt, and S. Garg. “BadNets: Evaluating Backdooring Attacks on Deep Neural Networks”. In: *IEEE Access* 7 (2019), pp. 47230–47244. DOI: 10.1109/ACCESS.2019.2909068.
- [120] F. Tramèr, F. Zhang, A. Juels, M. K. Reiter, and T. Ristenpart. “Stealing Machine Learning Models via Prediction APIs”. In: *USENIX Security Symposium*. 2016, pp. 601–618. arXiv: 1609.02943.
- [121] K. Grosse, N. Papernot, P. Manoharan, M. Backes, and P. McDaniel. “Adversarial Examples for Malware Detection”. In: *European Symposium on Research in Computer Security (ESORICS)*. Lecture Notes in Computer Science. Springer, 2017, pp. 62–79. DOI: 10.1007/978-3-319-66399-9_4.

- [122] S. Song, X. Xie, R. Feng, Q. Guo, and S. Chen. “FCGHunter: Towards Evaluating Robustness of Graph-Based Android Malware Detection”. In: *IEEE Transactions on Software Engineering* 52.2 (2026), pp. 428–448. DOI: 10.1109/TSE.2025.3626788.
- [123] I. J. Goodfellow, J. Shlens, and C. Szegedy. “Explaining and Harnessing Adversarial Examples”. In: *International Conference on Learning Representations (ICLR)*. 2015. arXiv: 1412.6572 [stat.ML].
- [124] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu. “Towards Deep Learning Models Resistant to Adversarial Attacks”. In: *International Conference on Learning Representations (ICLR)*. 2018. arXiv: 1706.06083 [stat.ML].
- [125] T. Fawcett. “An Introduction to ROC Analysis”. In: *Pattern Recognition Letters* 27.8 (2006), pp. 861–874. DOI: 10.1016/j.patrec.2005.10.010.
- [126] S. Sørbø and M. Ruocco. “Navigating the Metric Maze: A Taxonomy of Evaluation Metrics for Anomaly Detection in Time Series”. In: *Data Mining and Knowledge Discovery* 38.3 (2024), pp. 1027–1068. DOI: 10.1007/s10618-023-00988-8.
- [127] M. Grandini, E. Bagli, and G. Visani. *Metrics for Multi-Class Classification: An Overview*. White paper. 2020. DOI: 10.48550/arXiv.2008.05756. arXiv: 2008.05756 [stat.ML].
- [128] S. Axelsson. “The Base-Rate Fallacy and the Difficulty of Intrusion Detection”. In: *ACM Transactions on Information and System Security* 3.3 (2000), pp. 186–205. DOI: 10.1145/357830.357849.
- [129] J. Opitz and S. Burst. *Macro F1 and Macro F1*. Version 3, revised 8 February 2021. 2019. DOI: 10.48550/arXiv.1911.03347. arXiv: 1911.03347 [cs.LG].
- [130] Y. Aafer, W. Du, and H. Yin. “DroidAPIMiner: Mining API-Level Features for Robust Malware Detection in Android”. In: *International Conference on Security and Privacy in Communication Systems (SecureComm)*. 2013, pp. 86–103. DOI: 10.1007/978-3-319-04283-1_6.
- [131] N. Peiravian and X. Zhu. “Machine Learning for Android Malware Detection Using Permission and API Calls”. In: *IEEE International Conference on Tools with Artificial Intelligence (ICTAI)*. 2013, pp. 300–305. DOI: 10.1109/ICTAI.2013.53.
- [132] C. Li, K. Mills, D. Niu, R. Zhu, H. Zhang, and H. Kinawi. “Android Malware Detection Based on Factorization Machine”. In: *IEEE Access* 7 (2019), pp. 184008–184019. DOI: 10.1109/ACCESS.2019.2958927.
- [133] C. Chen, Y. Liu, B. Shen, and J.-J. Cheng. “Android Malware Detection Based on Static Behavior Feature Analysis”. In: *Journal of Computers* 29.6 (2018), pp. 243–253. DOI: 10.3966/199115992018122906024.
- [134] G. Canfora, A. De Lorenzo, E. Medvet, F. Mercaldo, and C. A. Visaggio. “Effectiveness of Opcode Ngrams for Detection of Multi Family Android Malware”. In: *International Conference on Availability, Reliability and Security (ARES)*. 2015, pp. 333–340. DOI: 10.1109/ARES.2015.57.
- [135] N. McLaughlin, J. Martinez del Rincon, B. Kang, S. Yerima, P. Miller, S. Sezer, Y. Safaei, E. Trickel, Z. Zhao, A. Doupé, and G.-J. Ahn. “Deep Android Malware Detection”. In: *ACM Conference on Data and Application Security and Privacy (CODASPY)*. 2017, pp. 301–308. DOI: 10.1145/3029806.3029823.

- [136] A. Narayanan, C. Soh, L. Chen, Y. Liu, and L. Wang. “apk2vec: Semi-Supervised Multi-View Representation Learning for Profiling Android Applications”. In: *IEEE International Conference on Data Mining (ICDM)*. 2018, pp. 357–366. DOI: 10.1109/ICDM.2018.00051.
- [137] J. Yan, G. Yan, and D. Jin. “Classifying Malware Represented as Control Flow Graphs Using Deep Graph Convolutional Neural Network”. In: *IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. 2019, pp. 52–63. DOI: 10.1109/DSN.2019.00020.
- [138] N. N. Tran, A. Said, W. Abbas, T. Derr, and X. D. Koutsoukos. *Evaluating Out-of-Distribution Robustness in Graph-Based Android Malware Classification: A New Principled Benchmark*. Accepted at IEEE DSAA 2026. Sept. 16, 2026. DOI: 10.48550/arXiv.2508.06734. arXiv: 2508.06734v3.
- [139] A. Arora, S. Garg, and S. K. Peddoju. “Malware Detection Using Network Traffic Analysis in Android Based Mobile Devices”. In: *International Conference on Next Generation Mobile Apps, Services and Technologies (NGMAST)*. 2014, pp. 66–71. DOI: 10.1109/NGMAST.2014.57.
- [140] A. Zulkifli, I. R. A. Hamid, W. M. Shah, and Z. Abdullah. “Android Malware Detection Based on Network Traffic Using Decision Tree Algorithm”. In: *International Conference on Soft Computing and Data Mining (SCDM)*. 2018, pp. 485–494. DOI: 10.1007/978-3-319-72550-5_46.
- [141] J. Malik and R. Kaushal. “CREDROID: Android Malware Detection by Network Traffic Analysis”. In: *ACM Workshop on Privacy-Aware Mobile Computing (PAMCO)*. 2016, pp. 28–36. DOI: 10.1145/2940343.2940348.
- [142] G. Marín, P. Casas, and G. Capdehourat. “DeepMAL – Deep Learning Models for Malware Traffic Detection and Classification”. In: *Data Science – Analytics and Applications*. Wiesbaden: Springer Vieweg, 2021, pp. 105–112. DOI: 10.1007/978-3-658-32182-6_16.
- [143] W. Wang, M. Zhu, X. Zeng, X. Ye, and Y. Sheng. “Malware Traffic Classification Using Convolutional Neural Network for Representation Learning”. In: *International Conference on Information Networking (ICOIN)*. 2017, pp. 712–717. DOI: 10.1109/ICOIN.2017.7899588.
- [144] A. H. Lashkari, A. F. A. Kadir, L. Taheri, and A. A. Ghorbani. “Toward Developing a Systematic Approach to Generate Benchmark Android Malware Datasets and Classification”. In: *International Carnahan Conference on Security Technology (ICCST)*. 2018, pp. 1–7. DOI: 10.1109/CCST.2018.8585560.
- [145] L. Taheri, A. F. A. Kadir, and A. H. Lashkari. “Extensible Android Malware Detection and Family Classification Using Network-Flows and API-Calls”. In: *International Carnahan Conference on Security Technology (ICCST)*. 2019, pp. 1–8. DOI: 10.1109/CCST.2019.8888430.
- [146] S. Jain, A. Arora, and D. Kumar. “Hybrid Analysis in Android Malware Detection: A Systematic and Comparative Review”. In: *Computer Science Review* 62 (2026), p. 100980. DOI: 10.1016/J.COSREV.2026.100980.
- [147] J. Feng, L. Shen, Z. Chen, Y. Lei, and H. Li. “HGDetecter: A Hybrid Android Malware Detection Method Using Network Traffic and Function Call Graph”. In: *Alexandria Engineering Journal* 114 (2025), pp. 30–45. DOI: 10.1016/j.aej.2024.11.068.

- [148] Canadian Institute for Cybersecurity. *Android Malware Dataset (CICAndMal2017)*. University of New Brunswick. 2026. URL: <https://www.unb.ca/cic/datasets/andmal2017.html> (visited on 09/20/2026).
- [149] T. Mikolov, I. Sutskever, K. Chen, G. S. Corrado, and J. Dean. “Distributed Representations of Words and Phrases and Their Compositionality”. In: *Advances in Neural Information Processing Systems (NIPS)*. 2013, pp. 3111–3119.
- [150] I. U. Haq and J. Caballero. “A Survey of Binary Code Similarity”. In: *ACM Computing Surveys* 54.3 (2021), pp. 1–38. DOI: 10.1145/3446371.
- [151] QoSient, LLC. *Argus: The Network Audit Record Generation and Utilization System*. openargus. 2026. URL: <https://openargus.org/> (visited on 08/14/2026).
- [152] S. Pourali, N. Samarasinghe, and M. Mannan. “Hidden in Plain Sight: Exploring Encrypted Channels in Android Apps”. In: *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 2022, pp. 2445–2458. DOI: 10.1145/3548606.3560665.
- [153] G. Dai, J. Ge, M. Cai, D. Xu, and W. Li. “SVM-Based Malware Detection for Android Applications”. In: *ACM Conference on Security & Privacy in Wireless and Mobile Networks (WiSec)*. 2015, pp. 1–2. DOI: 10.1145/2766498.2774991.
- [154] H. Li, B. Wu, W. Zhou, W. Yuan, C. Gao, X. You, and X. Luo. “An Efficient Adversarial Attack on FCG-Based Android Malware Detection Systems”. In: *IEEE Transactions on Information Forensics and Security* 20 (2025), pp. 9413–9426. DOI: 10.1109/TIFS.2025.3607270.
- [155] Y. He, Y. Liu, L. Wu, Z. Yang, K. Ren, and Z. Qin. “MsDroid: Identifying Malicious Snippets for Android Malware Detection”. In: *IEEE Transactions on Dependable and Secure Computing* 20 (2023), pp. 2025–2039. DOI: 10.1109/TDSC.2022.3168285.
- [156] J. Liu, J. Zeng, F. Pierazzi, Z. Yang, L. Cavallaro, and Z. Liang. “Unraveling the Key of Machine Learning-Based Android Malware Detection”. In: *ACM Transactions on Software Engineering and Methodology* (2026). DOI: 10.1145/3809491.
- [157] N. Daoudi, K. Allix, T. F. Bissyandé, and J. Klein. “Lessons Learnt on Reproducibility in Machine Learning Based Android Malware Detection”. In: *Empirical Software Engineering* 26 (2021), p. 74. DOI: 10.1007/s10664-021-09955-7.
- [158] X. Zhang, Y. Zhang, M. Zhong, D. Ding, Y. Cao, Y. Zhang, M. Zhang, and M. Yang. “Enhancing State-of-the-Art Classifiers with API Semantics to Detect Evolved Android Malware”. In: *ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 2020.
- [159] K. Zhao, H. Zhou, Y. Zhu, X. Zhan, K. Zhou, J. Li, L. Yu, W. Yuan, and X. Luo. “Structural Attack against Graph Based Android Malware Detection”. In: *ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 2021, pp. 3218–3235. DOI: 10.1145/3460120.3485387.
- [160] S. MahdaviFar, A. F. A. Kadir, R. Fatemi, D. Alhadidi, and A. A. Ghorbani. “Dynamic Android Malware Category Classification Using Semi-Supervised Deep Learning”. In: *2020 IEEE International Conference on Dependable, Autonomic and Secure Computing (DASC)*. 2020, pp. 515–522. DOI: 10.1109/DASC-PICOM-CBDCOM-CyberSciTech49142.2020.00094.

- [161] K. Allix, T. F. Bissyandé, J. Klein, and Y. Le Traon. “AndroZoo: Collecting Millions of Android Apps for the Research Community”. In: *IEEE/ACM International Conference on Mining Software Repositories (MSR)*. 2016, pp. 468–471. DOI: 10.1145/2901739.2903508.
- [162] M.-O. Pahl, G. Carle, and G. Klinker. “Distributed Smart Space Orchestration”. In: *NOMS 2016 – 2016 IEEE/IFIP Network Operations and Management Symposium*. IEEE, 2016, pp. 979–984. DOI: 10.1109/NOMS.2016.7502936.
- [163] J. Åkerberg and M. Björkman. “Exploring Security in PROFINET IO”. In: *Proceedings of the 33rd Annual IEEE International Computer Software and Applications Conference (COMPSAC)*. IEEE, 2009, pp. 406–412. DOI: 10.1109/COMPSAC.2009.61.
- [164] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and É. Duchesnay. “Scikit-learn: Machine Learning in Python”. In: *Journal of Machine Learning Research* 12 (2011), pp. 2825–2830.
- [165] D. Li, D. Chen, B. Jin, L. Shi, J. Goh, and S.-K. Ng. “MAD-GAN: Multivariate Anomaly Detection for Time Series Data with Generative Adversarial Networks”. In: *International Conference on Artificial Neural Networks (ICANN)*. Vol. 11730. Lecture Notes in Computer Science. Springer, 2019, pp. 703–716. DOI: 10.1007/978-3-030-30490-4_56.
- [166] A. Geiger, D. Liu, S. Alnegheimish, A. Cuesta-Infante, and K. Veeramachaneni. “TadGAN: Time Series Anomaly Detection Using Generative Adversarial Networks”. In: *IEEE International Conference on Big Data (Big Data)*. 2020, pp. 33–43. DOI: 10.1109/BigData50022.2020.9378139.
- [167] I. J. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio. “Generative Adversarial Nets”. In: *Advances in Neural Information Processing Systems 27 (NIPS)*. 2014, pp. 2672–2680.
- [168] T. Bilot, B. Jiang, Z. Li, N. El Madhoun, K. Al Agha, A. Zouaoui, and T. Pasquier. “Sometimes Simpler is Better: A Comprehensive Analysis of State-of-the-Art Provenance-Based Intrusion Detection Systems”. In: *USENIX Security Symposium*. 2025, pp. 7193–7212.
- [169] A. Kurakin, I. J. Goodfellow, and S. Bengio. “Adversarial Examples in the Physical World”. In: *International Conference on Learning Representations (ICLR) Workshop Track*. 2017. arXiv: 1607.02533 [cs.CV].
- [170] N. Carlini and D. Wagner. “Towards Evaluating the Robustness of Neural Networks”. In: *IEEE Symposium on Security and Privacy (S&P)*. 2017, pp. 39–57. DOI: 10.1109/SP.2017.49.
- [171] Z. Liu, P. Luo, X. Wang, and X. Tang. “Deep Learning Face Attributes in the Wild”. In: *2015 IEEE International Conference on Computer Vision (ICCV)*. 2015, pp. 3730–3738. DOI: 10.1109/ICCV.2015.425.
- [172] G. B. Huang, M. Ramesh, T. Berg, and E. Learned-Miller. *Labeled Faces in the Wild: A Database for Studying Face Recognition in Unconstrained Environments*. Technical Report 07-49. University of Massachusetts, Amherst, Oct. 2007.

- [173] J. Rauber. *Foolbox: Attack Base Classes, Version 3.3.1*. Source file foolbox/attacks/base . py; release of 23 February 2021. GitHub. 2021. URL: <https://github.com/bethgelab/foolbox/blob/v3.3.1/foolbox/attacks/base.py> (visited on 09/10/2026).
- [174] S.-M. Moosavi-Dezfooli, A. Fawzi, and P. Frossard. “DeepFool: A Simple and Accurate Method to Fool Deep Neural Networks”. In: *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2016, pp. 2574–2582. DOI: 10.1109/CVPR.2016.282.
- [175] U. Jang, X. Wu, and S. Jha. “Objective Metrics and Gradient Descent Algorithms for Adversarial Examples in Machine Learning”. In: *Proceedings of the 33rd Annual Computer Security Applications Conference (ACSAC)*. 2017, pp. 262–277. DOI: 10.1145/3134600.3134635.

Feature Importance of the NADICS Extraction Fields

Figure 5.3 in Section 5.4.1 names only the fifteen most important fields of the NADICS extraction schema. Table A.1 lists all 63 in rank order, with the importance of each field and the cumulative importance down to its rank. The cumulative column gives the thresholds of panel (c) of Figure 5.3 exactly: the first ten fields hold 54.47% of the total, the first 24 hold 80.72% and the first 36 hold 90.41%. The remaining 27 fields share 9.59%. Fields of equal importance keep the order of the source record, and the highest process field, `setpoint_change_count`, stands at rank 19.

Table A.1. Importance of the 63 fields of the NADICS extraction schema in the random forest of Section 5.4.2, in rank order. Importance is the normalised share of the total, and the cumulative column sums the importances down to that rank. The letter of the identifier gives the group of Table A.2, and the bar colours follow Figure 5.3.

Rank	ID	Field	Group	Importance (%)	Cumulative (%)	
1	C2	s7_function	Protocol	11.93	11.93	
2	D6	role_violation	Relational	7.95	19.88	
3	B6	period_deviation	Timing	6.76	26.64	
4	B11	unanswered_req_ratio	Timing	4.77	31.41	
5	C12	err_rate	Protocol	4.57	35.98	
6	D1	is_known_edge	Relational	4.37	40.36	
7	C5	write_read_ratio	Protocol	3.78	44.14	
8	A9	rate	Flow	3.58	47.71	
9	D3	n_distinct_dst_1min	Relational	3.48	51.19	
10	C9	addr_entropy	Protocol	3.28	54.47	
11	C11	s7_error_code	Protocol	2.88	57.36	
12	B3	iat_cv	Timing	2.58	59.94	
13	B7	cycle_count_gap	Timing	2.49	62.43	
14	D4	n_distinct_ports_1min	Relational	2.39	64.81	
15	B10	resp_latency_max	Timing	2.19	67.00	
16	C4	n_write_items	Protocol	2.09	69.09	
17	B8	cycle_jitter	Timing	1.79	70.87	
18	B9	resp_latency_mean	Timing	1.79	72.66	
19	E4	setpoint_change_count	Process	1.69	74.35	
20	C10	n_distinct_db	Protocol	1.59	75.94	
21	C15	n_tsap_tried	Protocol	1.29	77.24	
22	A3	service	Flow	1.19	78.43	
23	A4	state	Flow	1.14	79.57	
24	C3	n_read_items	Protocol	1.14	80.72	
25	A1	duration	Flow	1.09	81.81	

continued on the next page

Table A.1. (continued)

Rank	ID	Field	Group	Importance (%)	Cumulative (%)	
26	D5	role_pair	Relational	1.09	82.90	■
27	C6	s7_area	Protocol	0.89	83.80	■
28	D7	dcp_identify_count	Relational	0.84	84.64	■
29	A14	src_loss	Flow	0.80	85.44	■
30	B5	period_power	Timing	0.80	86.23	■
31	A5	src_pkt	Flow	0.75	86.98	■
32	B2	iat_std	Timing	0.75	87.72	■
33	A10	src_load	Flow	0.70	88.42	■
34	C1	s7_rosctr	Protocol	0.70	89.12	■
35	A6	dst_pkt	Flow	0.65	89.76	■
36	D2	edge_age	Relational	0.65	90.41	■
37	A7	src_bytes	Flow	0.60	91.00	■
38	B13	is_new_session	Timing	0.60	91.60	■
39	C18	opcua_session_rate	Protocol	0.60	92.20	■
40	A15	dst_loss	Flow	0.55	92.74	■
41	B4	dominant_period	Timing	0.55	93.29	■
42	A8	dst_bytes	Flow	0.50	93.79	■
43	D9	new_mac_seen	Relational	0.50	94.28	■
44	A12	src_mean_sz	Flow	0.45	94.73	■
45	B1	iat_mean	Timing	0.45	95.18	■
46	C7	s7_db_number	Protocol	0.45	95.63	■
47	E2	value_delta	Process	0.45	96.07	■
48	A11	dst_load	Flow	0.40	96.47	■
49	E5	value_freeze_len	Process	0.40	96.87	■
50	B12	session_age	Timing	0.35	97.22	■
51	E3	value_out_of_range	Process	0.35	97.56	■
52	A13	dst_mean_sz	Flow	0.30	97.86	■
53	C8	s7_start_addr	Protocol	0.30	98.16	■
54	C14	cotp_tsap_slot	Protocol	0.30	98.46	■
55	A2	protocol	Flow	0.25	98.71	■
56	C13	pdu_ref_gap	Protocol	0.25	98.96	■
57	C17	opcua_sec_policy_none	Protocol	0.25	99.20	■
58	A18	tcp_rtt	Flow	0.20	99.40	■
59	C16	opcua_msg_type	Protocol	0.15	99.55	■
60	D8	arp_conflict_count	Relational	0.15	99.70	■
61	A16	syn_ack	Flow	0.12	99.82	■
62	E1	tag_value	Process	0.10	99.92	■
63	A17	ackdat	Flow	0.08	100.00	■

Table A.2 gives the group totals behind panel (b) of Figure 5.3, together with the importance per field that Section 5.4.1 discusses and the rank of the highest field in each group.

Table A.2. Importance by field group. Both shares are given in % of the 63-field total. Importance per field is the group share divided by the number of fields in the group, and the last column gives the rank of the group's highest field in Table A.1.

Group	Fields	Importance share (%)	Field share (%)	Importance per field (%)	Highest rank
A ■ Flow base	18	13.32	28.57	0.74	8
B ■ Timing / periodicity	13	25.84	20.63	1.99	3
C ■ Protocol semantics	18	36.43	28.57	2.02	1
D ■ Relational	9	21.42	14.29	2.38	2
E ■ Process derived	5	2.98	7.94	0.60	19
All groups	63	100.00	100.00	1.59	